Langage C

Laurent Bellon

2025-2026

Introduction au Langage C et Environnement de Développement

Langage C : Un Pilier de la Programmation Moderne

- **Naissance aux Bell Labs (début 1970)**
  - Créé par Dennis Ritchie entre 1972 et 1973.
  - Objectif principal : Faciliter le développement du système d'exploitation Unix.
- **Un Contexte Historique Clé**
  - Avant C, les systèmes étaient souvent en assembleur : complexe et peu portable.
  - Unix, initié par Ken Thompson en assembleur, cherchait une meilleure portabilité.

La Genèse du C : Répondre à un Besoin Crucial

- **Évolution du langage B**
  - Dennis Ritchie a conçu C en ajoutant des innovations majeures comme les "structures" (`struct`).
  - Cette nouveauté a permis une manipulation plus efficace des données complexes, essentielle pour un OS.
- **Révolution pour Unix**
  - C a transformé Unix en le premier système d’exploitation majoritairement écrit dans un langage de haut niveau.
  - Résultat : Une portabilité grandement facilitée sur diverses machines.

L’Impact et l’Héritage du C

- **Diffusion Rapide**
  - À partir de 1975, Unix et C ont été licenciés, se propageant dans le monde universitaire et industriel.
- **Un Parent Influential**
  - C a directement inspiré de nombreux langages fondamentaux modernes : C++, C#, Java, Objective-C.
- **Importance Actuelle**
  - Le langage C reste incontournable pour la programmation système, les systèmes embarqués, et bien d’autres domaines.

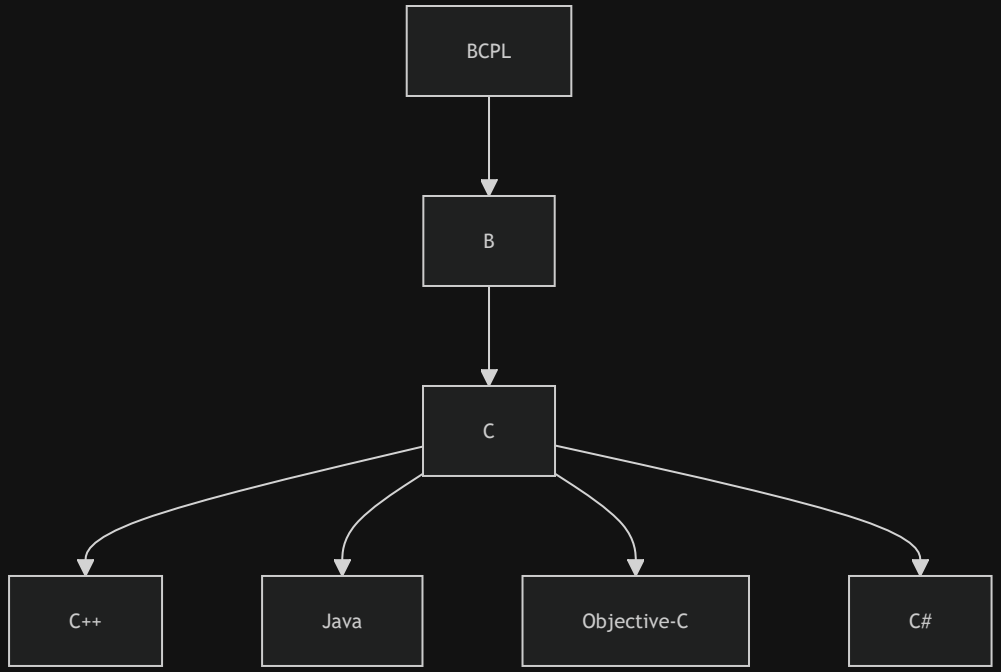
Le C en Pratique : "Hello, World!"

Ce programme illustre la simplicité et la puissance du langage C pour afficher un message.

```c
#include <stdio.h> // Inclusion de la bibliothèque standard d'entrée/sortie

int main() {        // Fonction principale, point d'entrée du programme
    printf("Hello, World!\n"); // Affiche le message "Hello, World!"
    return 0;       // Indique que le programme s'est exécuté sans erreur
}
```

L’Arbre Généalogique des Langages de Programmation



- Ce diagramme montre comment le langage C est issu de B (lui-même dérivé de BCPL) et a donné naissance à de nombreux langages modernes que nous utilisons aujourd’hui.

Ce qu’il faut retenir & Références

- Conclusion
  - La genèse du langage C est intrinsèquement liée au développement d’Unix.
  - Cette relation explique sa robustesse, sa portabilité et son importance continue dans le monde informatique.
- Références
  - Wikipedia - C (programming language)
  - GeeksforGeeks - History and Applications of C
  - Medium - Dennis Ritchie: A journey through C and Unix
  - Ars Technica - The origins of C

Le Langage C : Une Pertinence Toujours d’Actualité

Bien que ce langage ait été inventé dans les années 1970, le C continue d’être massivement utilisé dans des domaines où la maîtrise fine des ressources matérielles et la performance sont essentielles.

Pourquoi le C est-il encore pertinent aujourd’hui ? Nous explorerons les raisons fondamentales de sa pérennité, notamment dans les systèmes d’exploitation, la programmation embarquée et les applications à haute performance.

Cœur des Systèmes d’Exploitation

Le noyau des principaux systèmes d’exploitation (Linux, Windows, macOS) est majoritairement écrit en C.

- **Contrôle sur la mémoire** : Accès direct à la mémoire via les pointeurs, essentiel pour gérer les ressources critiques.
- **Portabilité** : Facilité d’adaptation à différentes architectures matérielles.
- **Performance** : Programmes compilés en code machine efficace, vital pour la réactivité et la stabilité.

> **Exemple : Le noyau Linux** Écrit à plus de 90% en C, avec un peu d’assembleur. Cette base en C assure une évolutivité et une maintenance robustes.

Le C pour l’Embarqué et l’IoT

Dans les systèmes embarqués, qui fonctionnent avec des ressources limitées (mémoire, CPU), le C est indispensable pour :

- Un **contrôle fin des ressources**.
- Générer du code extrêmement **léger et rapide**.
- Une exécution **proche du matériel**.

Le C est le langage de prédilection pour les microcontrôleurs, les systèmes temps réel et les dispositifs IoT.

> **Exemple : Arduino** La plateforme Arduino utilise le C/C++ pour programmer des microcontrôleurs, nécessitant un code très performant avec un contrôle rigoureux du matériel.

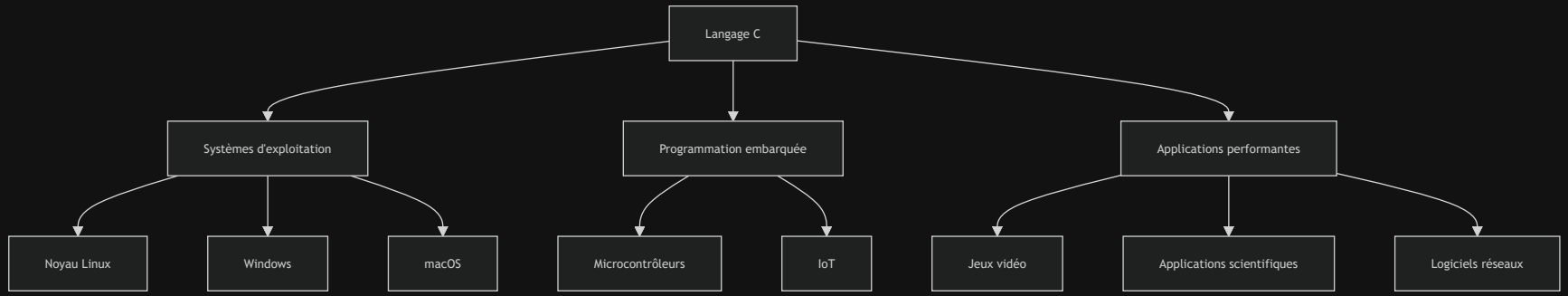
Performance Native et Optimisation

Le C, en tant que langage **compilé**, produit du code machine natif qui s'exécute très rapidement.

- **Absence de surcharge** : Évite le poids d'un interpréteur ou d'une machine virtuelle (contrairement à Python ou JavaScript).
- **Optimisations poussées** : Le compilateur peut effectuer des optimisations complexes.
- **Calculs lourds** : Permet d'écrire des routines très performantes pour des calculs intensifs ou du traitement bas niveau.

Applications et Maîtrise Matérielle

Domaines d’utilisation du langage C



Exemples de Contrôle Bas-Niveau

1. **Allocation manuelle de mémoire** : Le C offre `malloc` / `free` pour un contrôle explicite de la mémoire, crucial dans les systèmes embarqués.
2. **Gestion directe des bits** : Permet d’utiliser des opérations bit à bit rapides pour la programmation bas niveau.

En Résumé : Un Outil Indispensable

Ce panorama illustre que le langage C reste un choix privilégié, notamment dans les domaines critiques où la maîtrise du matériel et la performance sont déterminantes. Sa simplicité, son efficacité et sa souplesse en font un outil irremplaçable.

Sources utilisées

- GNU Project - Why use C?
- Embedded.com - Why is C still the best language for embedded systems?
- Linux Kernel Archives - About the Linux Kernel
- Arduino - Arduino and C/C++
- Stack Overflow Insights - Programming Language Trends 2023

Le Langage C : Une Évolution Standardisée

Le langage C est normalisé par l'ISO, ce qui lui permet d'évoluer progressivement. Cette standardisation vise à répondre aux besoins modernes, tout en restant fidèle à :

- Sa simplicité
- Sa performance

Quatre versions majeures du standard ont marqué son histoire récente, garantissant sa pertinence et sa compatibilité.

Standards C : Chronologie et Apports Majeurs

Les versions successives du standard C ont enrichi le langage avec des fonctionnalités clés :

Version	Année	Nouveautés principales
C99	1999	Types entiers fixes (`int32_t`), déclaration au milieu du code, support du booléen (`<stdbool.h>`), améliorations des macros, initialisation compound literals
C11	2011	Multithreading (`_Thread_local` , atomics), améliorations de la sécurité (`static_assert`), fonctions anonymes, standardisation Unicode (UTF-16, UTF-32)
C18	2018	Correction de bugs et de clarifications, pas de nouvelle fonctionnalité majeure
C23	2023	Nouveaux mots-clés, renforcement du type booléen natif, amélioration de `const` , attributs pour optimisation, support amélioré des littéraux

C23 : Les Nouveautés Clés pour un C Moderne

La dernière version, C23, apporte des ajouts significatifs qui modernisent le langage tout en conservant sa compatibilité.

1. **Type booléen natif amélioré :**
   - Intégration directe d'un type `bool` sans nécessiter `<stdbool.h>`.
   - Simplification de l'écriture : `bool a = true;`, `bool b = false;`
2. **Amélioration de `const` :**
   - Gestion renforcée pour mieux exprimer les intentions sur la mutabilité des données.
   - Nouveaux attributs pour accroître la sécurité et faciliter les optimisations du compilateur.
   - Exemple : `const int x = 10;`
3. **Nouveaux mots-clés et attributs :**
   - Mots-clés réservés supplémentaires et *attributs* (annotations).
   - Ces éléments guident le compilateur pour optimiser le code ou générer des alertes.

C23 en Pratique : Utilisation de `bool` et `const`

Voici un exemple concret illustrant l'utilisation du type booléen natif et de `const` en C23 :

```c
#include <stdio.h> // Nécessaire pour printf

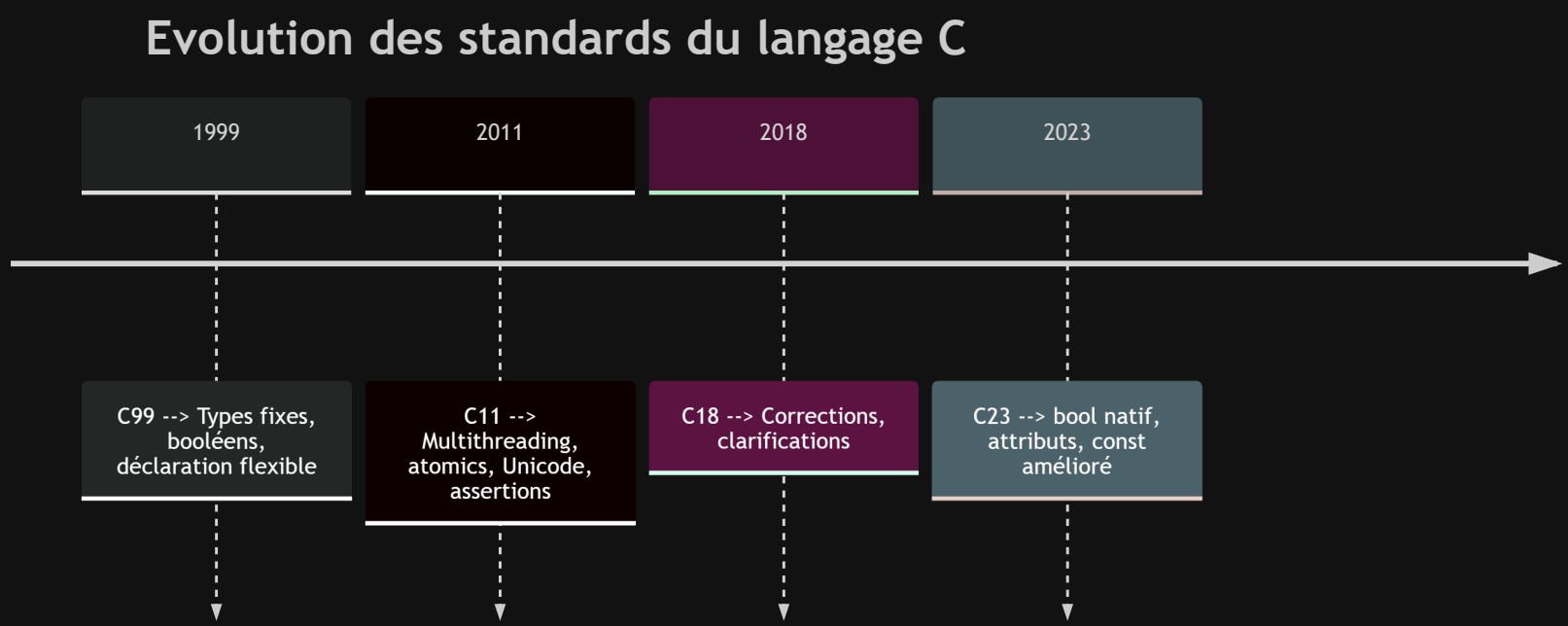
int main(void) {
    bool is_valid = true;  // Utilisation du booléen natif C23
    const int max_value = 100; // Variable constante

    if (is_valid) { // Condition basée sur le booléen
        printf("La valeur maximale est %d.\n", max_value); // Affichage de la constante
    }

    return 0;
}
```

Ce code compile et exécute en C23, démontrant la simplicité d'utilisation des nouvelles fonctionnalités.

L’Évolution des Standards C : Une Ligne du Temps



Bilan et Sources pour Approfondir

Ce qu'il faut retenir : Les versions modernes du langage C enrichissent ses possibilités sans perdre la simplicité et la performance qui font sa force. Avec C23, les avancées se confirment vers un code plus sûr, plus clair et plus performant.

Sources utilisées :

- ISO C standards overview - ISO/IEC JTC1/SC22/WG14 standard documents
- GNU C Library - C99 and C11 improvements
- WG14 Draft Proposal - C2x New Features (C23)
- ISO C23 summary - https://dev.sololearn.com/Discussion/3133636/c23-language/
- CPP Reference - C23 new features

Préparation de l’Environnement de Développement C

Outils essentiels pour coder, compiler et déboguer

L’environnement de développement est un ensemble d’outils indispensables pour écrire, compiler, déboguer et exécuter des programmes en langage C.

Cette présentation explore :

- Le compilateur **GCC**
- L’éditeur de texte **VS Code**
- L’environnement de développement intégré **Code::Blocks**

GCC : Le Compilateur Standard du C

GCC (GNU Compiler Collection) est un compilateur libre, open-source et standard de facto pour le langage C, développé par la Free Software Foundation.

Caractéristiques :

- **Compatibilité étendue :** Linux, Windows, macOS, architectures variées.
- **Performance :** Génère du code machine optimisé.
- **Diagnostics :** Options puissantes pour avertissements et débogage.
- **Intégration :** Facilité d'intégration avec éditeurs et IDE.

Installation rapide :

- **Linux :** `sudo apt install gcc` (souvent préinstallé)
- **Windows :** Inclus dans MinGW ou WSL
- **macOS :** `xcode-select --install` (via Xcode Command Line Tools)

Compilation et exécution de base :

```
# Exemple : un fichier `hello.c`
gcc hello.c -o hello
./hello
```

Le compilateur transforme votre code source en un programme exécutable.

Visual Studio Code : L’Éditeur Léger et Extensible

VS Code est un éditeur de texte léger, gratuit et open-source, très populaire et extensible.

Fonctionnalités clés pour le C :

- **Support multi-langages :** via des extensions, notamment l’extension officielle "C/C++" par Microsoft.
- **Intégration complète :** Débogueur, gestionnaire de tâches (pour la compilation), terminal intégré.
- **Productivité :** Intellisense (autocomplétion et suggestions de code).
- **Interface :** Moderne et hautement personnalisable.

Compilation intégrée avec `tasks.json` : VS Code permet d'automatiser la compilation en un clic, par exemple avec GCC.

```
{
    "version": "2.0.0",
    "tasks": [{
        "type": "shell",
        "label": "compiler C",
        "command": "gcc",
        "args": ["-g", "${file}", "-o", "${fileBasenameNoExtension}"],
        "group": "build"
    }]
}
```

Ce fichier configure VS Code pour appeler GCC automatiquement.

Code::Blocks : L'IDE Complet pour C/C++

Code::Blocks est un Environnement de Développement Intégré (IDE) open-source, conçu spécifiquement pour le développement en C/C++.

Caractéristiques principales :

- **Interface complète :** Regroupe gestionnaire de projets, éditeur de code et compilateur intégré.
- **Débogueur graphique :** Facilite la recherche et la correction d'erreurs.
- **Multi-plateforme :** Disponible sur Windows, Linux et macOS.
- **Configuration simplifiée :** Intégration transparente avec GCC.

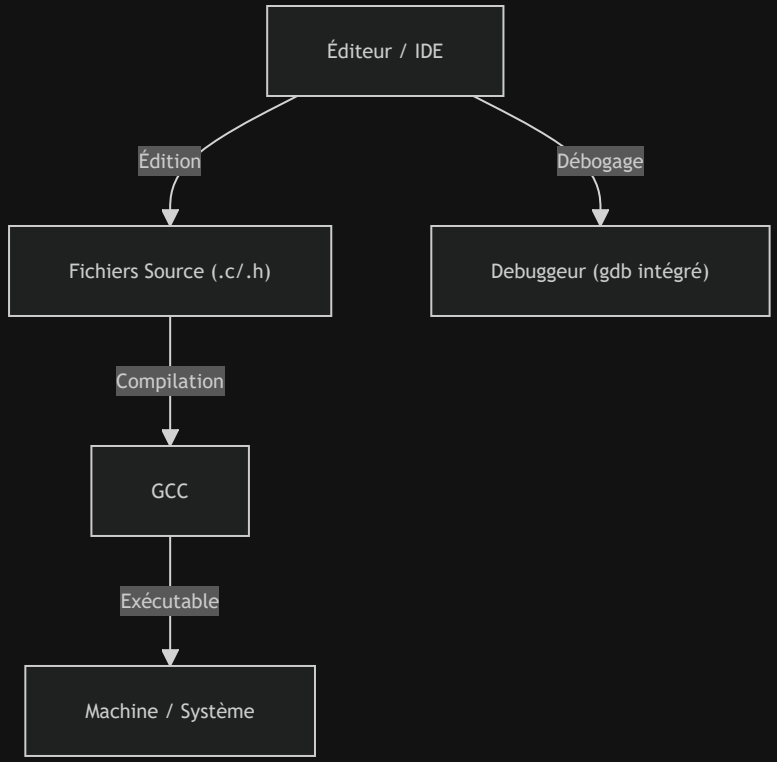
Utilisation :

- Création de projets C/C++ simplifiée.
- Compilation et débogage via des menus graphiques intuitifs.

Idéal pour un développement structuré avec tous les outils à portée de main.

Comprendre l’Écosystème des Outils C

Voici comment les différents outils interagissent au sein de votre environnement de développement :



Résumé des outils :

Outil	Type	Points forts	Usage typique
GCC	Compilateur	Performance, portabilité	Compilation rapide en ligne de commande
VS Code	Éditeur léger	Très personnalisable, extensions	Programmeurs préférant flexibilité
Code::Blocks	IDE complet	Intégration, facilité d’utilisation	Développement structuré projets C

Conclusion & Ressources

Cette présentation fixe les fondations matérielles pour coder efficacement en C à travers des outils modernes polyvalents et accessibles.

Sources utilisées :

- GNU GCC official website - https://gcc.gnu.org/
- Visual Studio Code - https://code.visualstudio.com/docs/languages/cpp
- Code::Blocks - http://www.codeblocks.org/
- TutsMake - How to compile C program using GCC
- Microsoft Docs - VS Code C++ Tutorial

Préparation de l’Environnement de Développement C

Introduction à l’Environnement C

Pourquoi est-ce crucial ?

- L’installation d’un compilateur et d’un IDE est une étape incontournable pour programmer en C.
- Cela permet de transformer votre code source en programme exécutable et de gérer vos projets.

Nous aborderons :

1. L’installation de GCC (le compilateur C incontournable).
2. L’installation de Code::Blocks (un IDE populaire et adapté aux débutants).

Installation de GCC (GNU Compiler Collection)

GCC : Le Compilateur C Incontournable

- Transforme le code source C en exécutable.

1. Sur Windows (via MinGW-w64)

- **Téléchargement :** Site officiel MinGW-w64 (https://www.mingw-w64.org/) ou installateur winlibs.com.
- **Installation :** Choisissez la version (64/32 bits) et l'architecture (x86_64), spécifiez un dossier (ex: `C:\mingw-w64`).
- **PATH :** Ajoutez le chemin du dossier `bin` de MinGW (ex: `C:\mingw-w64\bin`) à la variable d'environnement PATH.
- **Vérification :** Ouvrez `cmd` et tapez `gcc --version`.

2. Sur Linux

- Ouvrez un terminal et lancez :

```
sudo apt update
sudo apt install build-essential
```

- `build-essential` installe GCC et les outils de compilation.
- Vérification : `gcc --version`.

3. Sur macOS

- Installez les outils en ligne de commande Xcode :

```
xcode-select --install
```

- Ou via Homebrew :

```
brew install gcc
```

Installation de Code::Blocks (IDE)

Code::Blocks : L'Environnement de Développement Intégré (IDE)

- Open-source, facilite la gestion de projets C/C++.
- Intègre facilement GCC.

1. Sur Windows

- **Téléchargement :** Site officiel Code::Blocks (http://www.codeblocks.org/downloads).
- **Recommandé :** Choisissez le package complet avec MinGW intégré (ex: `codeblocks-20.03mingw-setup.exe`).
- **Installation :** Exécutez le fichier, suivez l'assistant (options par défaut).

2. Sur Linux

- Installer via gestionnaire de paquets :

```
sudo apt install codeblocks
```

- Code::Blocks détectera automatiquement GCC s'il est déjà installé.

3. Sur macOS

- **Téléchargement :** Fichier `.dmg` officiel de Code::Blocks (http://www.codeblocks.org/downloads).
- Suivez les instructions d'installation.

Premier Lancement et Test

Après l'installation de Code::Blocks (Windows)

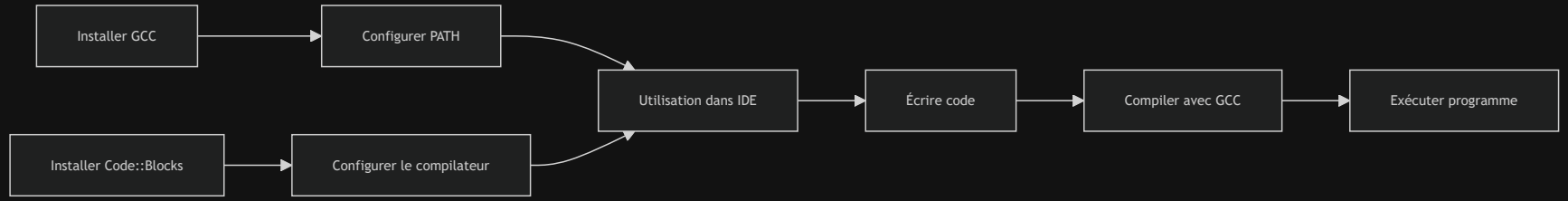
1. Lancement et configuration :
   - Lancez Code::Blocks.
   - Dans `Settings > Compiler`, vérifiez que le compilateur GNU GCC est sélectionné.
2. Création d'un projet simple :
   - `File > New > Project > Console Application > C`.

3. Compilation et Exécution :
   - L'IDE propose deux boutons principaux :
     - **Build** (ou "Compiler") : transforme le code source en exécutable.
     - **Run** : exécute le programme généré.

Exemple de code à tester (dans `main.c`) :

```c
#include <stdio.h>

int main() {
    printf("Hello, Code::Blocks and GCC!\n");
    return 0;
}
```



Description du flux :

1. **Installation de GCC** : Le compilateur est mis en place.
2. **Configuration du PATH** : Permet au système de trouver GCC.
3. **Installation de Code::Blocks** : L'IDE est installé.
4. **Configuration du compilateur** : L'IDE est connecté à GCC.
5. **Utilisation dans l'IDE** : L'environnement est prêt.
6. **Écriture du code** : Le développeur rédige son programme.
7. **Compilation avec GCC** : L'IDE utilise GCC pour transformer le code en exécutable.
8. **Exécution du programme** : Le programme est lancé.

Conclusion & Ressources

Ce qu'il faut retenir :

- Vous disposez maintenant d'un environnement stable et complet.
- Cet environnement est prêt pour :
  - Écrire du code C.
  - Compiler vos programmes.
  - Exécuter vos applications.
  - Déboguer vos projets.

Sources utilisées :

- GCC official site - https://gcc.gnu.org/install/
- MinGW-w64 - https://www.mingw-w64.org/
- Code::Blocks download - http://www.codeblocks.org/downloads
- Ubuntu Documentation - https://help.ubuntu.com/community/InstallingCompilers
- Homebrew for macOS - https://brew.sh/

Configuration de Code::Blocks pour la compilation C

Prérequis :

- Code::Blocks installé avec MinGW (GCC) intégré.

Étapes de vérification et configuration du compilateur GCC :

1. **Accéder aux paramètres :**
   - Menu : `Settings` > `Compiler...`
2. **Vérifier le compilateur :**
   - Sélectionner « GNU GCC Compiler » dans la liste déroulante.
3. **Vérifier le chemin vers le compilateur :**
   - Onglet « Toolchain executables ».
   - Le chemin doit pointer vers le dossier de `gcc.exe`, `g++.exe`, `gdb.exe` (Ex: `C:\Program Files (x86)\CodeBlocks\MinGW\bin`).
4. **Tester la configuration :**
   - Créer un projet Console en C, compiler et exécuter un code simple (ex: `printf("Hello world\n");`).

Configuration de Visual Studio Code (VS Code) pour C avec GCC

Prérequis :

- GCC installé (MinGW sur Windows, GCC sous Linux/macOS).
- VS Code installé.
- Extension officielle Microsoft C/C++ installée.

Configuration pas à pas :

1. **Installer l'extension C/C++ :**
   - Ouvrir le Marketplace (`Ctrl+Shift+X`).
   - Rechercher et installer l'extension « C/C++ » de Microsoft.
2. **Configurer les fichiers projet (`.vscode/`) :**
   - `tasks.json` : Pour la tâche de compilation.
   - `launch.json` : Pour la tâche de débogage.

VS Code : Configuration de la tâche de compilation (`tasks.json`)

Exemple minimal pour compiler `main.c` :

```
{
    "version": "2.0.0",
    "tasks": [
        {
            "label": "Build C program",
            "type": "shell",
            "command": "gcc",
            "args": [
                "-g",
                "main.c",
                "-o",
                "main"
            ],
            "group": {
                "kind": "build",
                "isDefault": true
            },
            "problemMatcher": ["$gcc"]
        }
    ]
}
```

VS Code : Configuration du débogage (`launch.json`) et Exemple

Exemple minimal pour déboguer un programme C :

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Debug C program",
            "type": "cppdbg",
            "request": "launch",
            "program": "${workspaceFolder}/main",
            "MIMode": "gdb",
            "setupCommands": [
                {
                    "description": "Enable pretty-printing for gdb",
                    "text": "-enable-pretty-printing",
                    "ignoreFailures": true
                }
            ]
        }
    ]
}
```

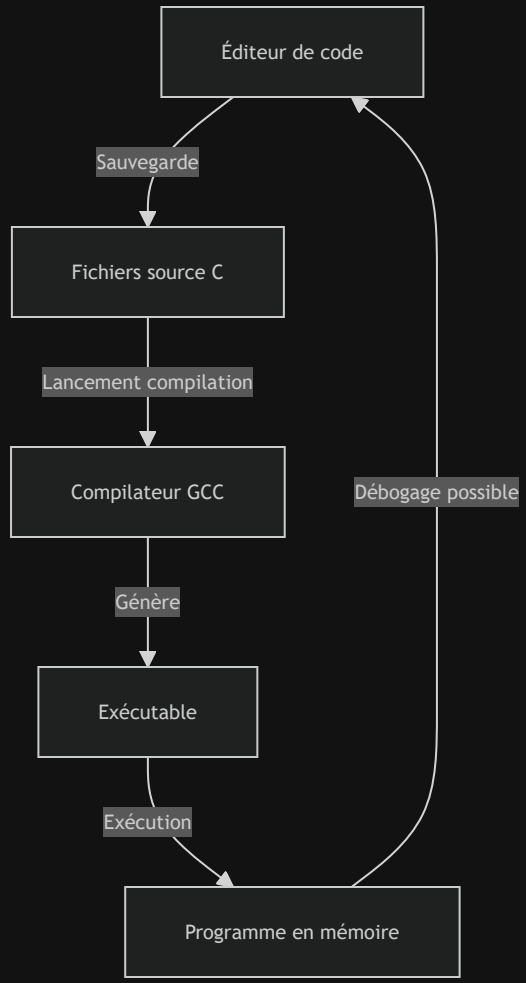
Démarrage du débogage : `F5`

Exemple de code C (`main.c`) et commandes :

```c
#include <stdio.h>

int main() {
    printf("Hello from VS Code!\n");
    return 0;
}
```

- Compilation : `gcc -g main.c -o main`
- Exécution : `./main`


Éditeur de code
Sauvegarde
Fichiers source C
Lancement compilation
Compilateur GCC
Génère
Exécutable
Exécution
Programme en mémoire
Débogage possible

Ce qu’il faut retenir :

Une configuration précise du compilateur dans l’IDE garantit un cycle de développement fluide, de l’écriture à l’exécution et au débogage. Ces étapes sont essentielles pour exploiter pleinement les outils de développement C modernes.

Sources utilisées :

- Code::Blocks Wiki - Compilers setup
- Visual Studio Code Docs - C/C++ development
- Microsoft Docs - Debugging configurations
- GCC Documentation - Building and using GCC

Introduction au Langage C : Mon Premier Programme

Les fondations d'un code fonctionnel

- Le langage C, malgré sa simplicité apparente, repose sur une structure claire et codifiée.
- Cette structure permet d'écrire des programmes efficaces et compréhensibles.
- Nous allons explorer les éléments fondamentaux qu'un programme C doit contenir pour être fonctionnel : `#include`, `main`, `return 0`.

Intégrer les Outils Nécessaires : la Directive `#include`

- **Rôle :** Permet d'inclure des fichiers d'en-tête (headers) contenant des définitions et des déclarations de fonctions, macros, types, essentielles au programme.
- **Syntaxe classique :**

```
#include <nom_du_fichier.h>
```

- **Exemple :** Pour utiliser la fonction `printf` (affichage à l'écran), on inclut la bibliothèque standard d'entrée/sortie :

```
#include <stdio.h>
```

- **Mécanisme :** Permet au compilateur de connaître les signatures des fonctions utilisées.

Le Cœur du Programme : la Fonction `main()`

- **Point d'entrée :** La fonction `main` est le point d'entrée **obligatoire** de tout programme C.
- **Syntaxe minimale :**

```c
int main(void) {
    // corps du programme
}
```

- **Valeur de retour :** Elle doit toujours retourner un entier (`int`), qui est le code de sortie du programme.
- **Convention :** Le `return 0;` signale au système d'exploitation que le programme s'est terminé correctement.

"Bonjour le monde !" : Un Exemple Concret

- Mon premier programme C :

```c
#include <stdio.h>     // Inclusion de la bibliothèque standard d'entrée/sortie

int main() {
    printf("Bonjour le monde !\n");  // Affiche "Bonjour le monde !"
    return 0;                        // Fin normale du programme
}
```

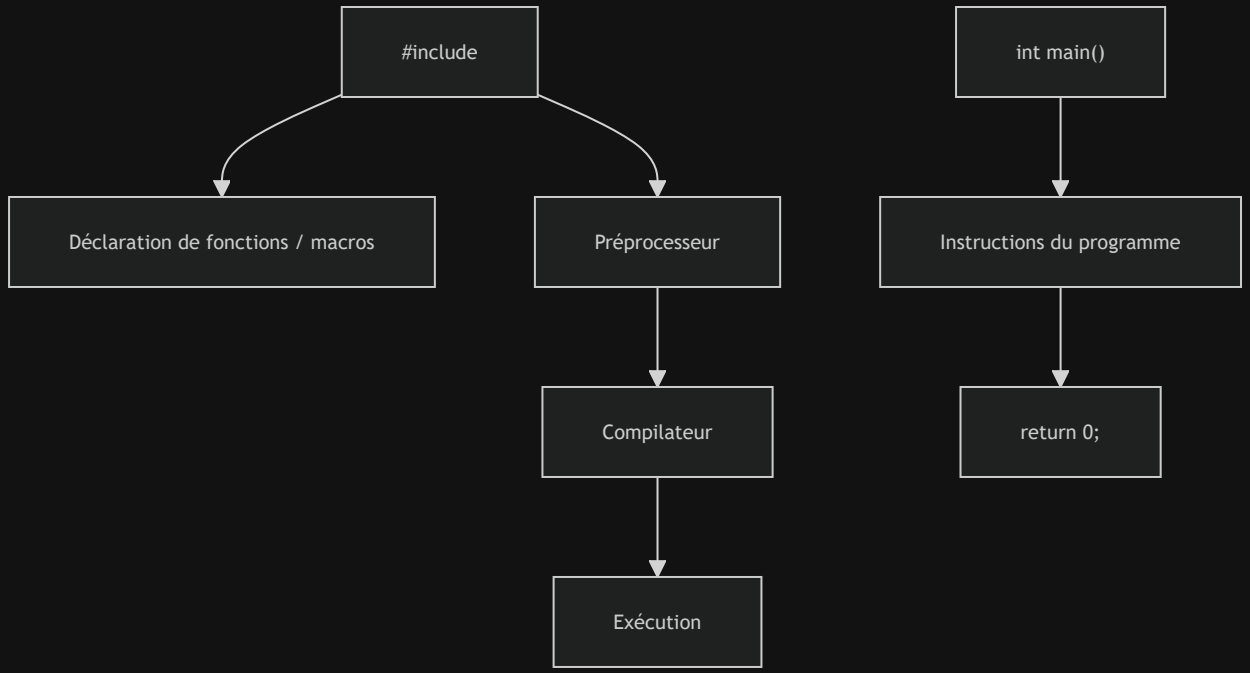
Ce programme affiche le texte "Bonjour le monde !" à l'écran.

- Déroulement du programme :

1. Le **préprocesseur** traite la directive `#include`.
2. Le **compilateur** transforme le code en langage machine.
3. À l'exécution, la fonction `main` **est appelée** automatiquement.
4. La fonction `printf` est exécutée : le message est affiché.
5. Le programme **retourne 0** et se termine.

Structure Visualisée et Options de `main()`

- Diagramme : Structure d’un programme C



- Compléments sur `main()` :
  - La fonction `main` peut recevoir des arguments `argc` (nombre d'arguments) et `argv` (tableau des arguments) pour gérer les paramètres de ligne de commande :

    ```
    int main(int argc, char *argv[]) { /* ... */ }
    ```

  - En C99 et suivants, si le `return 0;` est omis dans `main`, la fonction retourne implicitement 0.

Fondations Solides & Pour Aller Plus Loin

- **Ce qu'il faut retenir :**
  - La maîtrise de cette structure simple (`#include` , `main` , `return 0`) est la première étape.
  - Elle permet d'écrire un programme fonctionnel, structurellement conforme aux spécifications du langage C.
- **Sources Utilisées :**
  - ISO/IEC 9899:2018 – Programming Languages – C (C18 standard)
  - cppreference.com - main function
  - TutorialsPoint - Structure of C program
  - GNU GCC documentation - Standard C Library

L’instruction `printf()` en C : Premiers Pas

- **Qu’est-ce que `printf()` ?**
  - Fonction clé du langage C pour afficher du texte sur la console.
  - Appartient à la bibliothèque standard d’entrée/sortie (`<stdio.h>`).
- **Son Rôle et son Prototype**
  - `int printf(const char *format, ...);`
  - Fonction **variadique** : accepte une chaîne de format suivie d’un nombre variable d’arguments.
  - La chaîne de format peut contenir du texte fixe et des **spécificateurs de format** pour afficher les arguments.

Votre Premier Affichage : "Bonjour le monde !"

- Structure d'un programme simple :

```c
#include <stdio.h> // Indispensable pour printf()

int main() {
    printf("Bonjour le monde !\n"); // Affiche la chaîne de caractères
    return 0;
}
```

- Comprendre l'exemple :
  - `"Bonjour le monde !\n"` est une **chaîne littérale** affichée telle quelle.
  - Le caractère spécial `\n` insère un **saut de ligne** (nouvelle ligne) après le texte.

Les Spécificateurs de Format : Parlez à `printf()` !

- **Comment afficher différents types de données ?**
  - Utilisez des "placeholders" (marqueurs de position) appelés spécificateurs de format.

Spécificateur	Type attendu	Description	Exemple
`%d`	entier (int)	Affiche un entier en base décimale	`printf("Nombre: %d", 42);`
`%c`	caractère	Affiche un caractère	`printf("Lettre: %c", 'A');`
`%f`	flottant	Affiche un nombre flottant	`printf("Valeur: %f", 3.14);`
`%s`	chaîne	Affiche une chaîne de caractères	`printf("Nom: %s", "Jean");`
`%x`	entier	Affiche en hexadécimal	`printf("Hexa: %x", 255);`

Afficher des Variables : Dynamisme et Précision

- Intégrer des valeurs de variables dans votre texte :

```c
#include <stdio.h>

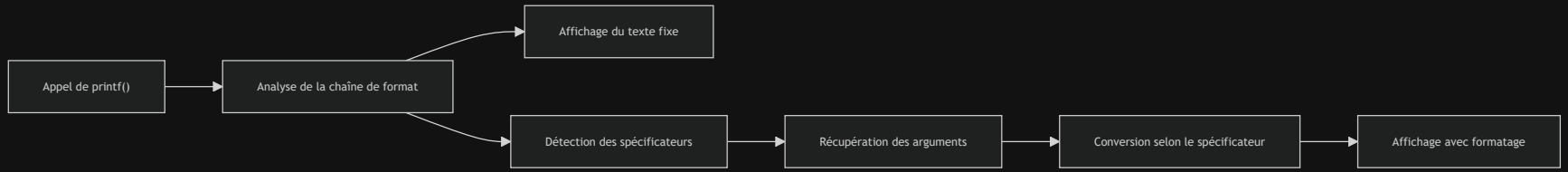
int main() {
    int age = 30;
    char lettre = 'C';
    float pi = 3.1416;

    printf("Votre âge est : %d ans\n", age);
    printf("La première lettre est : %c\n", lettre);
    printf("La valeur de Pi (approx.) : %.2f\n", pi); // Affichage à 2 décimales !
    return 0;
}
```

- `%.2f` : Contrôle de la précision
  - C'est un format de spécificateur qui permet de limiter le nombre de décimales affichées (ici, deux après la virgule) pour un nombre flottant.

Au-delà des Bases : Formatage Avancé et Fonctionnement

- Contrôler la présentation de l'affichage :
  - Précision : `printf("%.2f", 3.14159);` // Affiche `3.14`
  - Largeur : `printf("%5d", 42);` // Affiche `42` (avec 3 espaces devant pour atteindre 5 caractères)
- Comment `printf()` traite votre demande ?



- Le retour de `printf()` :
  - La fonction retourne un entier représentant le **nombre de caractères affichés**.
  - En cas d'erreur, elle retourne un nombre négatif.

- Ce qu’il faut retenir :
  - `printf()` est un outil puissant pour le débogage et l’interaction utilisateur en C.
  - Elle permet d’afficher à la fois du texte statique et des données dynamiques selon un format précis et contrôlé.
- Pour aller plus loin (Sources) :
  - GNU C Library Documentation - printf
  - cppreference.com - printf
  - TutorialsPoint - C - printf() Function
  - ISO/IEC 9899:2018 (C18) - Sections sur `<stdio.h>` fonctions d’entrée/sortie

Mon Premier Programme C : Le "Hello, World!"

Introduction au Langage C : Structure minimale et affichage

Le programme "Hello, World!" est l'exemple classique et universel pour démarrer en langage C.

Il illustre :

- La structure minimale d'un programme fonctionnel.
- L'usage de la fonction `printf()` pour afficher un texte simple.

Anatomie du "Hello, World!"

```c
#include <stdio.h>  // Inclusion de la bibliothèque standard d'entrée/sortie

int main() {
    printf("Hello, World!\n");  // Affiche la chaîne suivie d'un retour à la ligne
    return 0;                   // Terminaison normale du programme
}
```

Explications des composants clés :

- `#include <stdio.h>` : Permet d'accéder à la fonction `printf` pour l'affichage.
- `int main()` : Point d'entrée obligatoire de votre programme.
- `printf("Hello, World!\n");` : Affiche le texte sur la console. `\n` insère un saut de ligne.
- `return 0;` : Indique que le programme s'est terminé sans erreur.

Mettre le Code en Action : Compilation & Exécution

Pour transformer votre code source en un programme exécutable, une étape de compilation est nécessaire.

Sous Linux / macOS / environnement équipé GCC :

1. Enregistrer le code sous `hello.c`
2. Compiler :

   ```
   gcc hello.c -o hello
   ```

   (`gcc` est le compilateur, `-o hello` indique le nom du fichier exécutable.)
3. Exécuter :

   ```
   ./hello
   ```

Sous Windows avec MinGW ou Code::Blocks :

- Compiler directement depuis l'IDE via l'option "Build".
- Exécuter le programme généré via l'option "Run" de l'IDE ou directement le fichier `.exe` .



Ce schéma illustre les étapes :

1. Le **préprocesseur** prépare le code (gère les `#include`).
2. Le **compilateur** (GCC ici) traduit le code source en langage machine.
3. Le **fichier exécutable** est créé.
4. L'**OS** lance l'exécution de l'exécutable.
5. Le programme affiche son résultat sur la console.

Variantes du "Hello, World!"

Il existe plusieurs façons d'écrire un programme simple ou de déclarer la fonction `main`.

Utilisation de la fonction `puts()` pour un affichage simplifié :

```c
#include <stdio.h>

int main() {
    puts("Hello, World!"); // Affiche la chaîne et ajoute automatiquement un retour à la ligne
    return 0;
}
```

Note : `puts()` ajoute automatiquement un saut de ligne, contrairement à `printf()` qui le nécessite via `\n`.

Variante avec les arguments `main` :

```c
int main(int argc, char *argv[]) { // Déclaration complète de main
    printf("Hello, World!\n");
    return 0;
}
```

Ces arguments (`argc`, `argv`) permettent au programme de recevoir des informations lors de son lancement, concept clé pour des applications plus complexes.

Ce qu’il Faut Retenir & Pour Aller Plus Loin

Les points essentiels à retenir :

- Le "Hello, World!" est un excellent point de départ pour tester son environnement de développement.
- Il permet de comprendre les bases de la compilation et du lancement d’un premier code C.
- Les notions abordées ici (inclusion, `main`, `printf`, `return`) se retrouveront dans tous les programmes C, même les plus complexes.

Sources utilisées pour cette introduction :

- GNU GCC official docs – https://gcc.gnu.org/onlinedocs/
- cppreference.com – main function
- TutorialsPoint – Hello World in C
- ISO/IEC 9899:2018 – C Standard

Un Premier Programme C : De la Source à l'Exécutable

La Compilation et l'Exécution

La compilation et l'exécution sont les étapes clés pour transformer votre code C en programme fonctionnel. Elles vous permettent de passer d'un fichier source textuel à une application concrète.

Processus global :

1. **Écriture du code source** (ex: `hello.c`)
2. **Compilation** (`gcc`) : Traduction en code machine.
3. **Exécution** (`./nom_executable`) : Lancement du programme.

Compiler avec GCC

`gcc` est le compilateur standard pour le langage C.

Commande de base :

```
gcc nom_du_fichier.c -o nom_executable
```

- `gcc` : Le compilateur GNU C.
- `nom_du_fichier.c` : Votre fichier de code source.
- `-o nom_executable` : Option pour définir le nom du fichier exécutable généré.
  - *Par défaut :* `a.out` (sous Linux/macOS)

Exemple : Pour un code dans `hello.c` :

```
gcc hello.c -o hello
```

Ceci crée un exécutable nommé `hello`.

Les Étapes de Compilation et l'Exécution

Le compilateur `gcc` gère plusieurs phases distinctes pour créer l'exécutable.

Étapes de compilation par `gcc` :

1. **Préprocesseur** : Traite les `#include`, macros, etc.
2. **Compilation** : Traduit le code C en code assembleur.
3. **Assemblage** : Convertit l'assembleur en code machine objet.
4. **Édition des liens (linking)** : Assemble les objets et les bibliothèques pour former l'exécutable final.

Exécution du programme : Après compilation, lancez votre programme dans le terminal.

```
./nom_executable
```

Exemple :

```
./hello
```

Affiche le résultat du programme à l'écran.

Remarques Importantes & Options

Compatibilité OS :

- **Windows (avec MinGW) :** L'exécutable porte automatiquement l'extension `.exe` .

```
gcc hello.c -o hello.exe
hello.exe
```

- **Linux/macOS :** Pas d'extension par défaut, l'exécutable est directement exécutable.

Options de compilation :

- **Nom par défaut :** Si `-o` est omis, GCC crée un exécutable nommé `a.out` .
- **Mode debug :** Pour inclure les symboles de débogage (utile pour le diagnostic).

```
gcc -g hello.c -o hello
```

Exemple Concret : "Hello, World!"

Code source `hello.c` :

```c
#include <stdio.h>

int main() {
    printf("Hello, Compilation and Execution!\n");
    return 0;
}
```

Compilation :

```
gcc hello.c -o hello
```

Exécution :

```
./hello
```

Sortie affichée :

```
Hello, Compilation and Execution!
```

Ce qu’il faut retenir & Ressources

Conclusion : La maîtrise des commandes `gcc` pour compiler et exécuter vos programmes C est indispensable. Elle permet de développer efficacement et d’affiner rapidement votre code source.

Sources utilisées :

- GNU GCC Online Documentation - https://gcc.gnu.org/onlinedocs/gcc-12.1.0/gcc/Invocation.html
- TutorialsPoint - C - Compilation and Execution
- Linuxize - How to Compile and Run C Program Using GCC Compiler
- Microsoft Docs - Setting up GCC on Windows

L’importance des commentaires en C

- **Définition**
  - Blocs ou lignes de texte dans le code source.
  - Non exécutés par le compilateur.
- **Rôle fondamental**
  - **Documenter** le code.
  - Faciliter sa **compréhension**.
  - Assurer son **maintien** dans le temps.

Types de commentaires en langage C

Commentaires sur une seule ligne (`//`)

- Introduits en C99.
- Commencent avec `//` et s'étendent jusqu'à la fin de la ligne.
- **Utilité** : remarques brèves, explications concises.

```c
int x = 5;  // Déclaration d'une variable entière x
```

Commentaires multilignes (`/* ... */`)

- Encadrent un bloc de texte.
- Peuvent s'étendre sur plusieurs lignes.
- **Utilité** : explications plus longues, désactivation temporaire de code.

```c
/*
   Ce programme affiche un message de bienvenue
   et initialise une variable.
*/
int main() {
    printf("Bienvenue!\n");
    return 0;
}
```

Bonnes pratiques pour des commentaires efficaces

- **Soyez clairs et concis**
  - Apportez une valeur ajoutée sans répéter le code.
- **Expliquez le *pourquoi*, pas le *comment***
  - Mettez en lumière la logique ou la raison d'un choix.
- **Tenez-les à jour**
  - Un commentaire obsolète est plus nuisible qu'utile.
- **Utilisez les commentaires pour structurer votre code**
  - Séparez les sections, expliquez les étapes importantes.

Exemple pratique et rôle du compilateur

```c
#include <stdio.h>

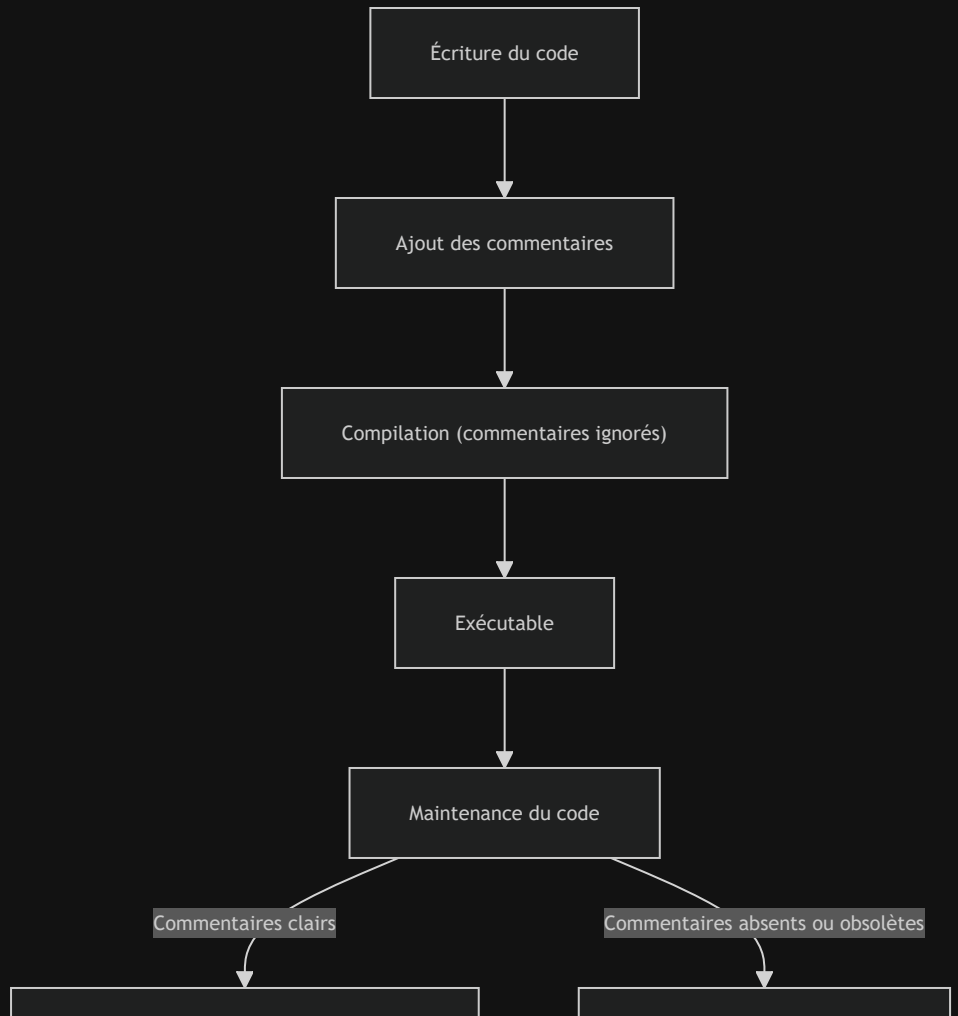
int main() {
    int compteur = 0;          // Initialisation du compteur à zéro

    /* Boucle qui s'exécute 10 fois */
    while (compteur < 10) {
        printf("Valeur : %d\n", compteur);
        compteur++;            // Incrémentation du compteur
    }

    return 0;                  // Fin normale du programme
}
```

Impact sur la compilation

- Le compilateur **ignore complètement** les commentaires.
- Les commentaires **n'occupent pas d'espace en mémoire** lors de l'exécution du programme.


Écriture du code
Ajout des commentaires
Compilation (commentaires ignorés)
Exécutable
Maintenance du code
Commentaires clairs
Commentaires absents ou obsolètes

Synthèse et ressources pour approfondir

Ce qu'il faut retenir

Les commentaires sont un outil indispensable pour un code :

- **Lisible**
- **Partageable**
- **Maintenable**

Une bonne utilisation améliore la collaboration et réduit le risque d'erreurs lors des évolutions ultérieures.

Sources utilisées

- ISO/IEC 9899:2018 – Langage C standard
- cppreference.com - Commentaires en C
- GNU GCC documentation - Preprocessing and comments
- TutorialsPoint - C Comments

Conventions de Nommage en C : Clarté et Cohérence

Les Fondamentaux pour un Code Lisible et Maintenable

Les conventions de nommage définissent des règles pour nommer variables, fonctions, constantes ou types dans un programme.

- **Objectifs clés :**
  - Améliorer la lisibilité du code.
  - Faciliter sa maintenance.
  - Éviter les erreurs.
- **En langage C :**
  - Bien que non imposées par le standard, l'adoption de conventions cohérentes est **fortement recommandée**.

Règles Générales du Nommage

Des principes de base pour tous les identifiants :

- **Caractères autorisés :**
  - Lettres (a-z, A-Z)
  - Chiffres (0-9)
  - Underscore (`_`)
- **Premier caractère :**
  - Doit être une lettre ou un underscore.
  - Ne peut pas être un chiffre.
- **Respect de la casse :**
  - Le C est sensible à la casse (`Variable` et `variable` sont différents).
- **Interdits :**
  - Pas d'espaces.
  - Pas de caractères spéciaux (`!` , `@` , etc.).

Conventions Courantes en C

Élément	Convention recommandée	Exemple
Variables et fonctions	Minuscules, mots séparés par underscore (`_`)	`nombre_utilisateurs` , `calcul_moyenne`
Constantes (macros)	Lettres majuscules avec underscores	`MAX_TAILLE` , `NOMBRE_MAX`
Types définis (`typedef`)	Nom en `PascalCase` ou `CamelCase` (majuscule début de mot)	`Point2D` , `ListeChaînée`
Variables locales	Nom court et descriptif	`i` , `compteur` , `temp`
Variables globales	Préfixe `g_` ou suffixe `_global` pour indiquer la portée	`g_total` , `config_global`

Exemples Illustrés en Code C

```c
#include <stdio.h>

#define MAX_ITERATIONS 100       // Constante macro en MAJUSCULES

typedef struct Point {
    int x_coord;                // Variable membre en minuscules avec underscore
    int y_coord;
} Point2D;                      // Type défini en PascalCase

int calcul_distance(Point2D p1, Point2D p2) { // Fonction en minuscules avec underscore
    int delta_x = p2.x_coord - p1.x_coord;    // Variable locale courte et descriptive
    int delta_y = p2.y_coord - p1.y_coord;
    int distance = delta_x * delta_x + delta_y * delta_y;
    return distance;
}

int main() {
    Point2D pointA = {2, 3};     // Variable locale
    Point2D pointB = {5, 7};

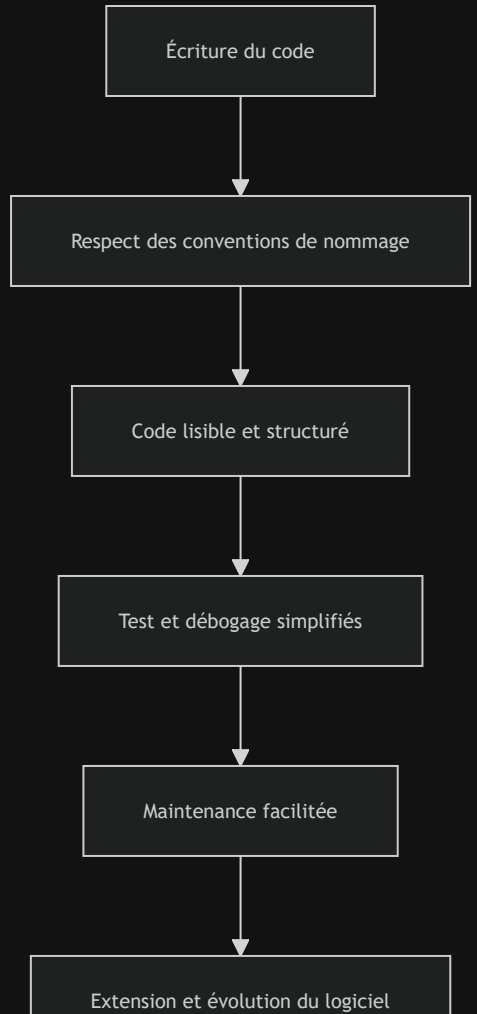
    int dist = calcul_distance(pointA, pointB);
    printf("Distance au carré: %d\n", dist);

    return 0;
}
```

Impact sur le Cycle de Développement

Adopter des conventions de nommage renforce la qualité et l'efficacité du travail.

- Avantages concrets :
  - **Lisibilité :** Un code plus clair et intuitif.
  - **Maintenance facilitée :** Repérer rapidement le rôle d'un élément.
  - **Prévention d'erreurs :** Différencier facilement constantes, variables, types.
  - **Cohérence en équipe :** Travail collaboratif plus efficace.


Écriture du code
Respect des conventions de nommage
Code lisible et structuré
Test et débogage simplifiés
Maintenance facilitée
Extension et évolution du logiciel

Ce qu’il faut retenir & Ressources

L’application de conventions simples de nommage dès les premiers programmes en C contribue à la qualité globale du code et facilite la progressivité dans l’apprentissage et la pratique.

Sources utilisées :

- GNU Coding Standards - https://www.gnu.org/prep/standards/standards.html
- Linux Kernel Coding Style (extraits) - https://www.kernel.org/doc/html/latest/process/coding-style.html
- Stack Overflow - Conventions de nommage en C
- Wikipedia - Variable naming conventions

Qu’est-ce que la Lisibilité du Code ?

La lisibilité du code est la facilité avec laquelle un programme peut être lu et compris par un développeur.

Pourquoi est-elle indispensable en C ?

- La proximité avec la machine impose parfois une syntaxe compacte.
- Adopter des qualités rédactionnelles claires est crucial pour écrire un code maintenable.

Les Fondamentaux pour un Code Clair

Pour améliorer la lisibilité de votre code :

- **Indentation régulière** : Structurez visuellement les blocs de code.
- **Commentaires pertinents** : Expliquez le "pourquoi", pas le "quoi".
- **Noms explicites** : Variables et fonctions aux noms clairs et descriptifs.
- **Simplicité et concision** : Évitez les constructions complexes inutiles.
- **Organisation logique** : Regroupez les fonctions et les déclarations.
- **Utilisation des espaces** : Séparez les opérateurs et les éléments pour aérer le texte.

Exemples : Avant et Après Amélioration

1. Mauvaise lisibilité

```
int f(int a,int b){if(a>b){return a;}else{return b;}}
```

2. Lisibilité améliorée

```
int maximum(int a, int b) {
    if (a > b) {
        return a;
    } else {
        return b;
    }
}
```

3. Commentaires et formatage

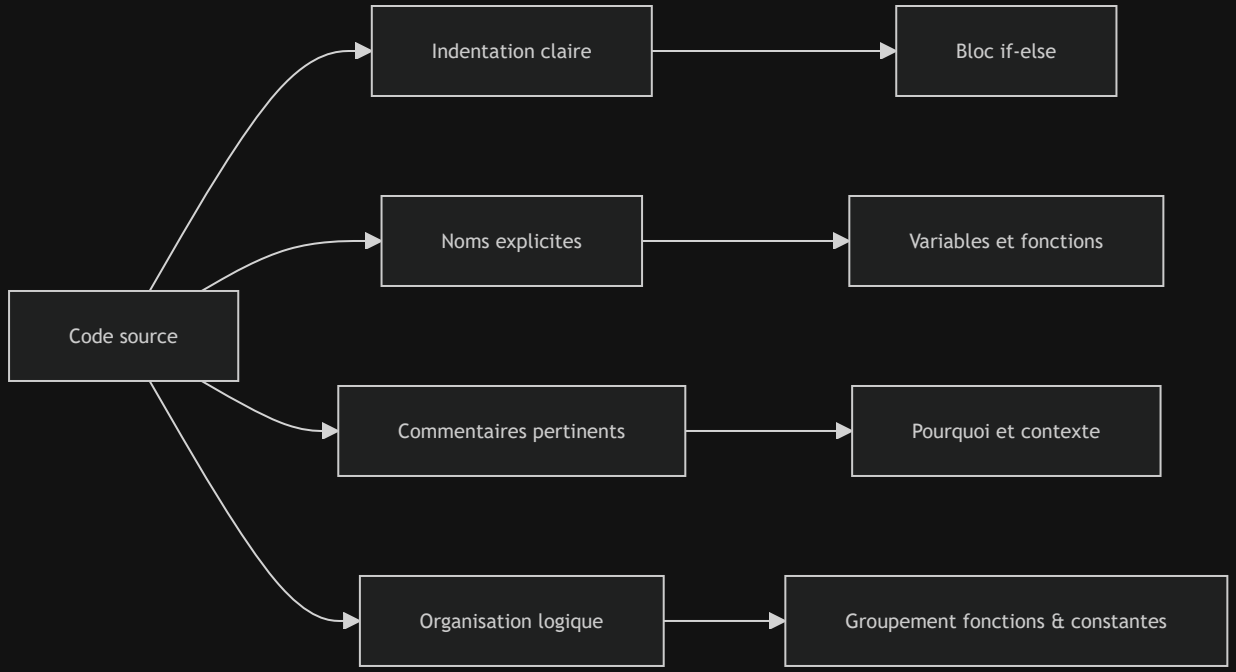
```
// Renvoie la valeur maximale entre a et b
int maximum(int a, int b) {
    if (a > b) {
        return a;  // a est plus grand
    } else {
        return b;  // b est plus grand ou égal à a
    }
}
```

Outils et Bonnes Pratiques

Pour maintenir un code lisible :

- **Utilisation d'un linter** : Détecte et corrige automatiquement le style (ex : `clang-format`).
- **Respect des conventions de style** (`K&R` , `Allman`) pour l'indentation et les accolades.
- **Séparation des responsabilités** dans les fonctions : une fonction = une tâche unique.

La Structure d’un Code Lisible et ses Bénéfices



Impact sur la productivité et la maintenance

Un code lisible permet :

- De réduire le temps de relecture et debugging.
- D’augmenter la collaboration entre développeurs.
- D’assurer une évolution plus sûre et rapide du logiciel.

En Bref : Un Investissement Durable & Sources

Ce qu'il faut retenir

Adopter des règles simples pour améliorer la lisibilité est un investissement durable : il facilite le travail individuel et collectif tout au long du cycle de développement logiciel.

Sources utilisées

- GNU Coding Standards - https://www.gnu.org/prep/standards/standard.html#Formatting
- Clang-Format Documentation – https://clang.llvm.org/docs/ClangFormat.html
- Coding Horror - https://blog.codinghorror.com/code-tells-you-how-comments-tell-you-why/
- Stack Overflow - How to write readable C code?

Les Fondamentaux du C : Variables & Types de Données

Qu’est-ce qu’une variable ?

- Une **variable** est une zone mémoire nommée utilisée pour stocker une donnée.
- Chaque variable a un **type** qui définit :
  - La nature de la donnée qu’elle peut contenir (nombre entier, caractère, nombre à virgule, etc.)
  - Sa taille en mémoire

Déclarer une variable

- Informer le compilateur de son type et de son nom.
- La déclaration ne réserve pas forcément une valeur à la variable.

Syntaxe :

```
type nom_variable;
```

Exemples :

```
int age;           // variable entière appelée age
float temperature; // variable décimale appelée temperature
char initiale;     // variable caractère appelée initiale
```

Initialiser une variable & Pourquoi ?

- Consiste à assigner une valeur à la variable.

Initialisation lors de la déclaration :

```
int age = 25;
float pi = 3.14;
char lettre = 'A';
```

Initialisation après déclaration :

```
int age;
age = 30;
```

Pourquoi initialiser ses variables ?

- Éviter les valeurs indéfinies (contenu mémoire aléatoire) qui peuvent provoquer des comportements imprévisibles.
- Faciliter la lecture et le suivi du programme.

Durée de vie et Portée (Scope)

- Les variables déclarées dans une fonction ont une **portée locale**.
- Les variables globales sont accessibles dans tout le programme.
- La **durée de vie** correspond au temps pendant lequel la variable reste accessible.

Exemple Pratique & Cycle de vie

Exemple complet illustré :

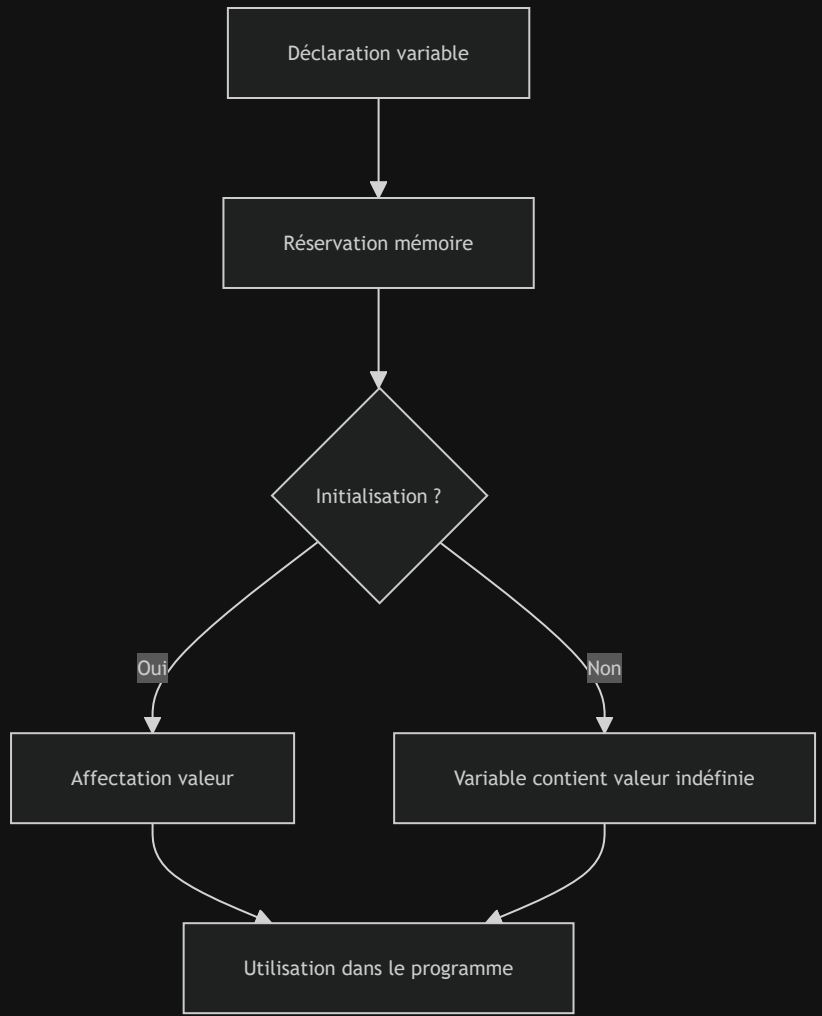
```c
#include <stdio.h>

int main() {
    int compteur = 0;            // déclaration et initialisation
    float moyenne;               // déclaration

    moyenne = 15.5;              // initialisation plus tard
    char grade = 'B';            // initialisation directe

    printf("Compteur: %d\n", compteur);
    printf("Moyenne: %.2f\n", moyenne);
    printf("Grade: %c\n", grade);

    return 0;
}
```


Déclaration variable
Réservation mémoire
Initialisation ?
Oui
Non
Affectation valeur
Variable contient valeur indéfinie
Utilisation dans le programme

Bonnes Pratiques & Ressources

Ce qu'il faut retenir :

- Les noms de variables doivent respecter les règles de nommage.
- En C, les variables non initialisées ont une valeur indéterminée.
- La bonne pratique est de **toujours initialiser les variables** avant utilisation.

La déclaration et l'initialisation des variables constituent le socle fondamental de la programmation en C. Les maîtriser permet de gérer efficacement les données manipulées dans vos applications.

Sources utilisées :

- cppreference.com - Variables in C
- TutorialsPoint - C Variables
- ISO/IEC 9899:2018 – Langage C standard
- GNU GCC documentation - https://gcc.gnu.org/onlinedocs/gcc-12.1.0/gcc/Variable-Attributes.html

Les Fondamentaux du C

Variables et Types de Données : Les Entiers

Introduction aux Types Entiers en C

- **Qu’est-ce qu’un type entier ?**
  - Utilisé pour stocker des nombres sans partie décimale.
- **Pourquoi plusieurs types ?**
  - Optimiser l’espace mémoire.
  - S’adapter à différentes plages de valeurs.
- **Concept de "Signé" vs. "Non Signé"**
  - **Signé (par défaut) :** Permet de stocker des nombres positifs ET négatifs.
  - `unsigned` **(modificateur) :** Permet de ne stocker QUE des valeurs positives, doublant la valeur maximale possible.

Types Entiers Standard et Leurs Plages

Type	Taille typique (octets)[1]	Plage de valeurs signée	Plage de valeurs non signée (unsigned)
`short`	2	-32,768 à 32,767	0 à 65,535
`int`	4	-2,147,483,648 à 2,147,483,647	0 à 4,294,967,295
`long`	4 ou 8	-2,147,483,648 à 2,147,483,647 (4 octets) ou -9... à 9... (8 octets)	0 à 4,294,967,295 (4 octets) ou 0 à 18... (8 octets)
`long long`	8	-9,223,372,036,854,775,808 à 9,223,372,036,854,775,807	0 à 18,446,744,073,709,551,615

[1]_La taille dépend de la plateforme et du compilateur._

Le Modificateur `unsigned` : Usage et Précautions

- **Bénéfice majeur :** Enlever le signe élargit la plage des valeurs positives.
  - Exemple : un `unsigned int` peut représenter des nombres de 0 à plus de 4 milliards.
- **Point d'attention crucial :**
  - Ne pas mélanger des variables signées et non signées dans des opérations arithmétiques.
  - Cela peut entraîner des comportements inattendus ou des bugs difficiles à détecter.

Exemples Pratiques en C

```c
#include <stdio.h>

int main() {
    short s = -10;
    unsigned short us = 60000;

    int i = -1000;
    unsigned int ui = 3000000000U;  // Suffixe U pour unsigned

    long l = 100000L;              // Suffixe L pour long
    unsigned long ul = 4000000000UL;

    long long ll = -9000000000000000000LL; // Suffixe LL pour long long
    unsigned long long ull = 18000000000000000000ULL;

    printf("short s = %d\n", s);
    printf("unsigned short us = %u\n", us);
    printf("int i = %d\n", i);
    printf("unsigned int ui = %u\n", ui);
    printf("long l = %ld\n", l);
    printf("unsigned long ul = %lu\n", ul);
    printf("long long ll = %lld\n", ll);
    printf("unsigned long long ull = %llu\n", ull);
```

Points Essentiels à Retenir

- **Variabilité de la taille :** La taille des types entiers (`sizeof`) peut varier selon la machine et le compilateur.
- **Adéquation des données :** Toujours vérifier la capacité nécessaire pour les données que vous manipulez afin d'éviter les dépassements.
- **Considération du signe :** Prendre en compte la nature signée ou non signée est essentiel pour prévenir les erreurs dans les comparaisons et les calculs.


Types entiers
int (4 octets)
plage: -2 147 483 648 à 2 147 483 647
unsigned: 0 à 4 294 967 295
short (2 octets)
plage: -32 768 à 32 767
unsigned: 0 à 65 535
long (4 ou 8 octets)
plage: variable selon plateforme
unsigned: variable

Conclusion & Ressources

Comprendre ces types entiers et leurs limites permet d'adapter efficacement la gestion des données numériques selon la nature des besoins et des architectures cibles.

Sources utilisées :

- cppreference.com - Integer types ©
- ISO/IEC 9899:2018 – Langage C standard
- TutorialsPoint - C Data Types
- GNU GCC documentation - Integer Types and Sizes

Les Fondamentaux du C : Variables et Types de Données

Types flottants : `float`, `double`

Introduction aux Types Flottants

- **Représentation des nombres réels :** Les types flottants permettent de stocker des nombres avec une partie fractionnaire.
- **Essentiels pour :** Mesures physiques, calculs scientifiques, données financières.
- **Deux types principaux en C :**
  - `float` : simple précision
  - `double` : double précision

`float` et `double` : Caractéristiques et Déclaration

Type	Taille typique (octets)	Précision approximative	Plage typique
`float`	4	6 à 9 chiffres décimaux	±1.5 × 10^−45 à ±3.4 × 10^38
`double`	8	15 à 17 chiffres décimaux	±5.0 × 10^−324 à ±1.7 × 10^308

Ces caractéristiques sont conformes à la norme IEEE 754.

Syntaxe de déclaration et initialisation :

```
float vitesse = 12.5f;   // Suffixe 'f' recommandé pour float
double distance = 12345.6789; // Par défaut, un littéral flottant est double
```

Particularités du Calcul en Virgule Flottante

- **Imprécisions :** Les opérations peuvent engendrer des erreurs dues à la représentation binaire.
- **Comparaisons directes :** Déconseillées entre flottants.
- **Comparaison avec tolérance :** Utiliser une marge d'erreur (`epsilon`) pour vérifier l'égalité.

Exemple Pratique

```c
#include <stdio.h>
#include <math.h> // Pour fabs()

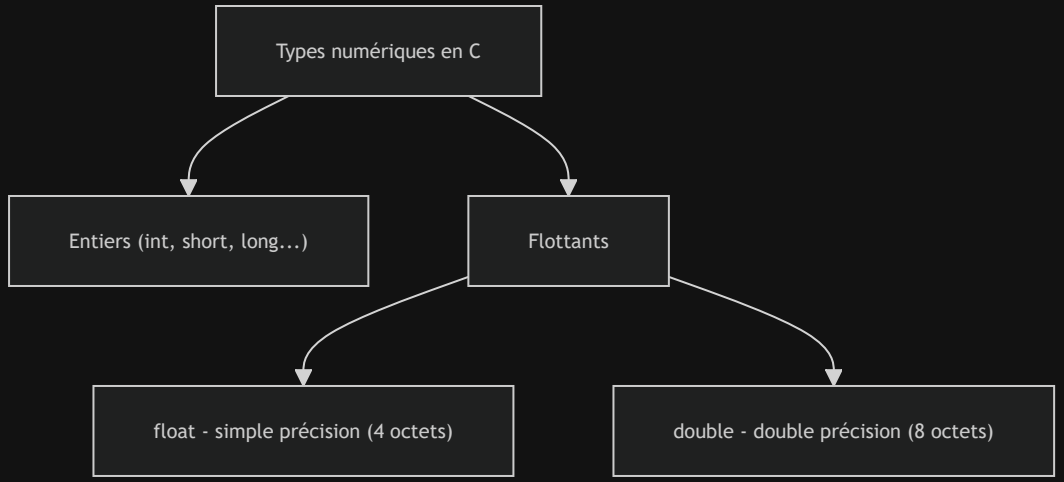
int main() {
    float pi_float = 3.1415927f;   // Simple précision
    double pi_double = 3.141592653589793; // Double précision

    printf("float pi = %.7f\n", pi_float);
    printf("double pi = %.15lf\n", pi_double);

    // Comparaison avec tolérance (epsilon)
    double a = 0.1 + 0.2;
    double b = 0.3;
    double epsilon = 0.00001;

    if (fabs(a - b) < epsilon) {
        printf("a et b sont égaux avec tolérance epsilon\n");
    } else {
        printf("a et b sont différents\n");
    }
    return 0;
}
```

Types Numériques en C et Conseils Pratiques



Conseils pratiques :

- **Choix du type :**
  - `float` : économie de mémoire, précision modérée.
  - `double` : précision élevée (préférable par défaut).
- **Attention :** aux dépassements et erreurs d'arrondi.
- **Fonctions mathématiques :** Utiliser des fonctions compatibles (`sin` , `cos` , `sqrt` prennent généralement des `double`).

Récapitulatif & Ressources

Maîtriser les types flottants et leurs limites est indispensable pour tout calcul nécessitant une représentation précise des valeurs réelles en programmation C.

Sources utilisées :

- ISO/IEC 9899:2018 – Langage C standard
- cppreference.com - Floating-point types
- TutorialsPoint - C Floating Point
- IEEE 754 Standard overview - https://ieeexplore.ieee.org/document/8766229

Type Caractère : `char`

Définition & Utilité

- **Stocke un caractère unique** (ex: 'A', 'b', '7').
- Occupe typiquement **1 octet (8 bits)**, permettant 256 valeurs.
- Principalement utilisé pour manipuler des **caractères ASCII**.
- Intrinsèquement un **petit entier**, peut être utilisé pour des valeurs numériques compactes.

Représentation & Initialisation

En mémoire

- Peut être **signé** ou **non signé** (dépend du compilateur/plateforme).
- La norme C ne définit pas la nature signée par défaut.
- Caractères ASCII standards : de 0 à 127.

Déclaration et initialisation

```
char lettre = 'A';      // caractère seul entre apostrophes
char code = 65;         // valeur ASCII entière équivalente à 'A'
```

Le caractère est toujours indiqué entre guillemets simples `'`.

Manipuler les `char` : Opérations

Le `char` comme entier

- Les opérations arithmétiques sont autorisées.
- On peut **incrémenter ou décrémenter** un `char`.
- Permet de parcourir facilement des alphabets ou des codes ASCII.

Exemples :

- `char c = 'a' + 1;` // `c` devient 'b'
- `char d = 'Z' - 2;` // `d` devient 'X'

Exemple Pratique : Opérations

```c
#include <stdio.h>

int main() {
    char c1 = 'Z';
    char c2 = 90;           // code ASCII de 'Z'
    char c3 = c1 + 1;       // caractère suivant (ici '[')

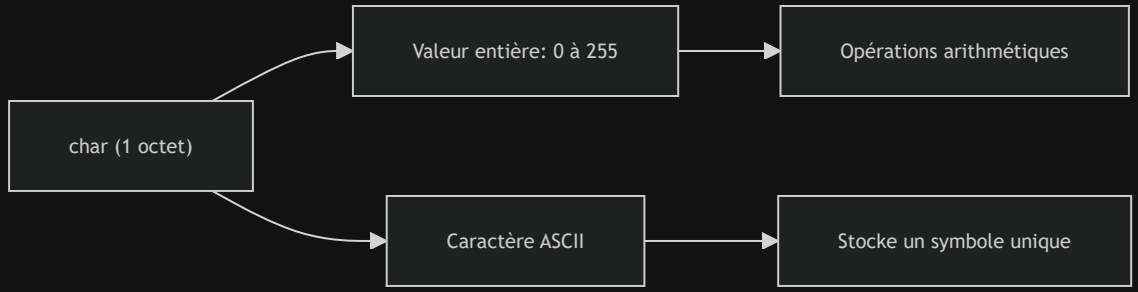
    printf("c1 = %c\n", c1);
    printf("c2 = %c\n", c2);
    printf("c3 = %c\n", c3);

    return 0;
}
```

Sortie :

```
c1 = Z
c2 = Z
c3 = [
```

Anatomie du `char` & Bonnes Pratiques



Particularités et bonnes pratiques

- Pour les chaînes de caractères : `char` dans des **tableaux terminés par '\0'**.
- Gérer les conversions entre `char` et `int` pour éviter des erreurs (liées au signe).
- Pour l'Unicode, envisager d'autres types ou bibliothèques.
- Toujours **initialiser** les variables `char` avant usage.

Synthèse & Références

Ce qu'il faut retenir

Le type `char` forme la base de la manipulation des symboles en C. Sa compréhension, tant comme caractère qu'entier compact, est fondamentale pour toute gestion à bas niveau des données textuelles.

Sources utilisées

- cppreference.com - char type
- ISO/IEC 9899:2018 – Langage C standard
- TutorialsPoint - C Char Data Type
- Stack Overflow - Is char signed or unsigned in C?

Le Type Booléen en C (C99/C23) : Clarté et Sémantique

Contexte & Introduction : Avant le `bool` natif

Avant la norme C99, le langage C ne disposait pas nativement d'un type booléen. Les programmeurs utilisaient souvent des entiers (`int`) avec la convention :

- `0 = faux`
- `Toute autre valeur ≠ 0 = vrai`

L'innovation C99/C23 Avec la norme C99 (maintenue en C23), un type booléen dédié a été introduit : `_Bool` . Une macro `bool` est définie dans `<stdbool.h>` pour simplifier son usage.

Déclaration et Caractéristiques du type `_Bool`

Déclaration :

- Le type natif du langage est `_Bool` (mot-clé réservé).
- L'inclusion de la bibliothèque `<stdbool.h>` permet d'utiliser `bool` comme alias de `_Bool` et les constantes `true` et `false`.

```
#include <stdbool.h>

bool est_actif = true;
_Bool est_valide = 0;  // false
```

Caractéristiques :

- Taille généralement 1 octet (mais peut varier).
- Valeur `0` représente `false`.
- Toute valeur non nulle est convertie en `1` (`true`) lors de l'affectation à une variable `_Bool`.

Exemple d’Utilisation Pratique

```c
#include <stdio.h>
#include <stdbool.h>

int main() {
    bool condition = true;
    _Bool autre_condition = 0;

    if (condition) {
        printf("condition est vraie\n");
    }

    if (!autre_condition) {
        printf("autre_condition est fausse\n");
    }

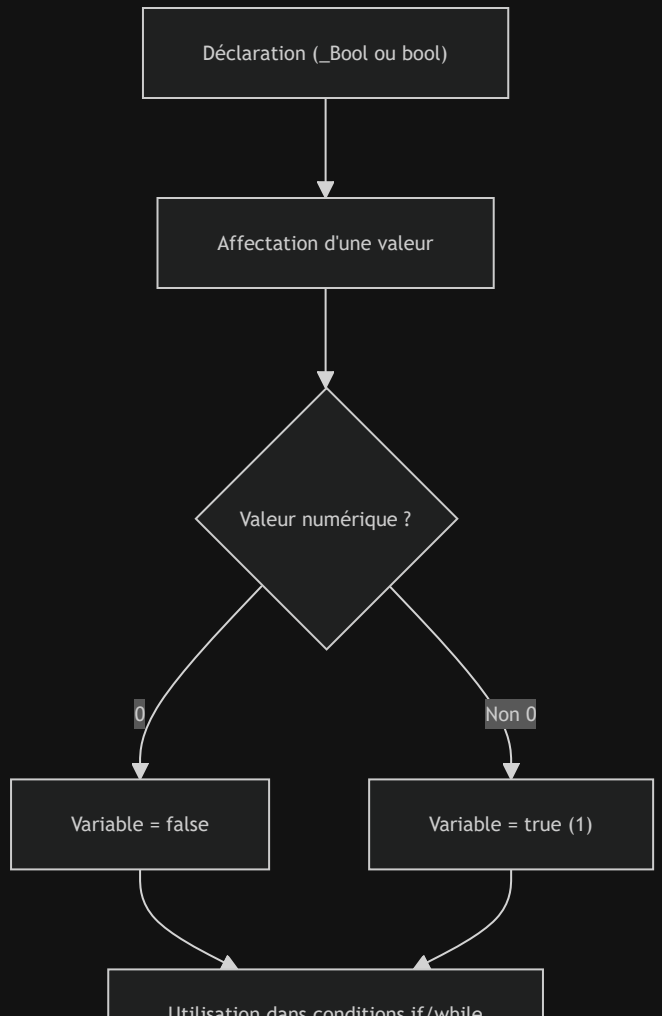
    // Affectation de valeurs entières
    autre_condition = 42; // automatiquement converti en true (1)

    printf("autre_condition vaut %d\n", autre_condition);

    return 0;
}
```

Comparaison avec les Pratiques Antérieures

Avant C99	Avec C99 et suivantes
`int flag = 0;`	`bool flag = false;`
`if (flag)` avec 0 ou non-0	`if (flag)` avec `true` / `false`
Pas de type booléen dédié	Type booléen natif clair et standardisé
Usage moins explicite, moins sûr	Meilleure lisibilité, sécurité sémantique


Déclaration (_Bool ou bool)
Affectation d'une valeur
Valeur numérique ?
0
Non 0
Variable = false
Variable = true (1)
Utilisation dans conditions if/while

Points Clés & Ressources

Ce qu'il faut retenir :

- Inclure `<stdbool.h>` pour définir correctement `bool`, `true`, `false`.
- `_Bool` est le mot-clé natif, `bool` est une macro pour faciliter la lecture.
- Conversion automatique lors de l'affectation : toute valeur non nulle devient `1`.
- Favoriser l'utilisation de `bool` pour clarifier l'intention du code.
- Programmation plus expressive et sûre dans les structures conditionnelles.

Sources utilisées :

- cppreference.com - _Bool and boolean type
- ISO/IEC 9899:2018 (C18) and draft C23 standards - sections sur `_Bool`
- TutorialsPoint - C Boolean Data Type
- GNU GCC documentation - stdbool.h

Le type booléen natif apporte une sémantique claire et simplifie la gestion des conditions logiques, en évitant les approximations liées à l'usage des entiers comme booléens.

Les Fondamentaux du C : Taille & Plage des Données

- **Taille :** L'espace mémoire (en octets) qu'une variable occupe.
- **Plage :** L'ensemble des valeurs possibles que ce type peut stocker.

En C, chaque type de variable se voit allouer une taille mémoire définie, ce qui détermine directement la plage de valeurs qu'elle peut contenir.

La norme C ne fixe que des bornes minimales, la taille réelle pouvant varier selon l'architecture et le compilateur. L'opérateur `sizeof()` est essentiel pour connaître la taille en octets d'un type ou d'une variable à l'exécution.

La Variabilité des Types C : Pourquoi est-ce important ?

- **Norme C et Bornes Minimales :** Le standard C impose seulement des bornes de valeurs *minimales* que chaque type doit pouvoir contenir, et non une taille fixe en octets.
- **Impact de l'Architecture et du Compilateur :**
  - La taille des types peut varier considérablement entre différentes architectures (ex: 32-bit vs. 64-bit).
  - Le compilateur que vous utilisez peut aussi influencer ces tailles.
- **L'Opérateur `sizeof()` : Votre Alliée**
  - Permet d'obtenir la taille *réelle* d'un type ou d'une variable en octets.
  - Indispensable pour écrire du code portable et éviter les erreurs liées aux hypothèses de taille.

Aperçu des Tailles et Plages des Types Standard

Sur une architecture commune 64-bit (Linux/Windows), voici les tailles et plages typiques :

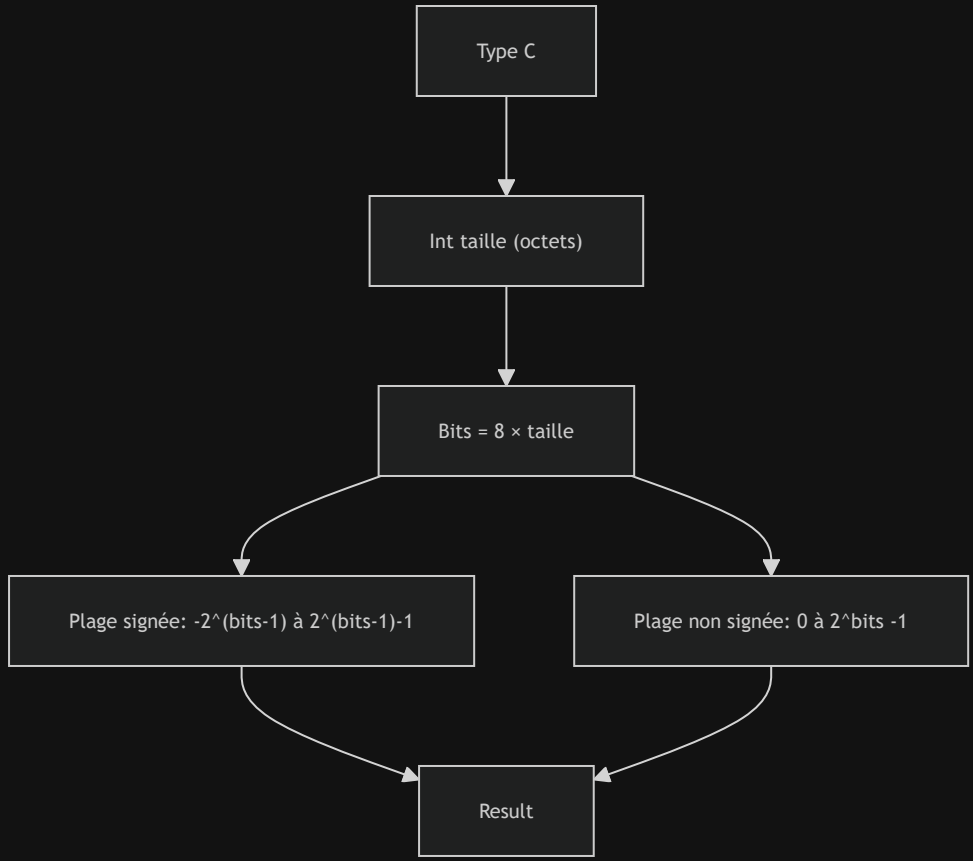
Type	Taille (octets)	Plage (signé)	Plage (non signé)
`char`	1	-128 à 127	0 à 255
`short`	2	-32,768 à 32,767	0 à 65,535
`int`	4	-2,147,483,648 à 2,147,483,647	0 à 4,294,967,295
`long`	8 (sur 64 bits)	-9,223,372,036,854,775,808 à 9,223,372,036,854,775,807	0 à 18,446,744,073,709,551,615
`long long`	8	même que `long`	même que `unsigned long`

Calculer les Plages des Types Entiers

Pour les types entiers (complément à deux) occupant *N* octets (soit 8 × N bits) :

- **Plage signée :** de -2^(8N-1) à 2^(8N-1) - 1
- **Plage non signée :** de 0 à 2^(8N) - 1

Ce diagramme illustre la relation directe entre la taille mémoire et les plages de valeurs :


Type C
Int taille (octets)
Bits = 8 × taille
Plage signée: -2^(bits-1) à 2^(bits-1)-1
Plage non signée: 0 à 2^bits -1
Result

Vérification Pratique avec `sizeof` et `limits.h`

Pour connaître la taille et les limites réelles sur votre système, utilisez `sizeof` et les constantes définies dans `<limits.h>` et `<float.h>`.

```c
#include <stdio.h>
#include <limits.h> // Pour INT_MIN, INT_MAX, etc.
#include <float.h>  // Pour FLT_MIN, FLT_MAX

int main() {
    printf("Taille de int: %zu octets\n", sizeof(int));
    printf("INT_MIN: %d, INT_MAX: %d\n", INT_MIN, INT_MAX);

    printf("Taille de char: %zu octets\n", sizeof(char));
    printf("CHAR_MIN: %d, CHAR_MAX: %d\n", CHAR_MIN, CHAR_MAX);

    printf("Taille de long: %zu octets\n", sizeof(long));
    printf("LONG_MIN: %ld, LONG_MAX: %ld\n", LONG_MIN, LONG_MAX);

    printf("Taille de float: %zu octets\n", sizeof(float));
    printf("FLT_MIN: %e, FLT_MAX: %e\n", FLT_MIN, FLT_MAX);

    return 0;
}
```

Synthèse et Ressources Clés

Points importants :

- **Vérifiez toujours :** Utilisez `sizeof` pour connaître la taille réelle et éviter les erreurs de dépassement (`overflow`).
- **Fiabilité :** Préférez les constantes de `<limits.h>` et `<float.h>` pour des plages garanties et portables.
- **Optimisation :** Adaptez le choix du type à la plage de valeurs nécessaire pour une gestion mémoire efficace.
- **Précision :** Ne partez pas du principe qu'un type a une taille fixe. Si besoin, utilisez des types à taille garantie comme `int32_t` ou `uint64_t` .

Comprendre la relation entre taille mémoire et plages de valeurs permet d'écrire des programmes fiables et efficaces, en évitant des erreurs d'overflow ou d'usage mémoire inadapté.

Sources utilisées :

- cppreference.com - Data types and limits
- ISO/IEC 9899:2018 – Langage C standard (chapitres sur types et portabilité)
- GNU GCC Documentation - Data Types

Introduction aux Opérateurs Arithmétiques

Les opérateurs arithmétiques en langage C permettent d'effectuer des calculs mathématiques de base sur des variables numériques. Ils sont fondamentaux pour toute manipulation de données numériques.

Opérateur	Description	Exemple
`+`	Addition	`a + b`
`-`	Soustraction	`a - b`
`*`	Multiplication	`a * b`
`/`	Division entière ou flottante	`a / b`
`%`	Modulo (reste de la division)	`a % b` (entiers)

Comportement en fonction des types

Le comportement des opérateurs varie selon les types des opérandes :

- `+`, `-`, `*` : S'appliquent aussi bien sur types **entiers** que **flottants**.
- `/` (Division) :
  - Si les opérandes sont **entiers**, la division est **entière** (partie décimale tronquée).
  - Si l'un des opérandes est **flottant** (`float` ou `double`), la division est **flottante**.
- `%` (Modulo) : Ne s'applique qu'aux **entiers**.

Exemples Pratiques

```c
#include <stdio.h>

int main() {
    int a = 10, b = 3;
    float x = 10.0, y = 3.0;

    printf("Addition : %d + %d = %d\n", a, b, a + b);
    printf("Soustraction : %d - %d = %d\n", a, b, a - b);
    printf("Multiplication : %d * %d = %d\n", a, b, a * b);

    printf("Division entière : %d / %d = %d\n", a, b, a / b);
    printf("Modulo : %d %% %d = %d\n", a, b, a % b);

    printf("Division flottante : %.2f / %.2f = %.2f\n", x, y, x / y);

    return 0;
}
```

Sortie :

```
Addition : 10 + 3 = 13
Soustraction : 10 - 3 = 7
Multiplication : 10 * 3 = 30
Division entière : 10 / 3 = 3
Modulo : 10 % 3 = 1
Division flottante : 10.00 / 3.00 = 3.33
```

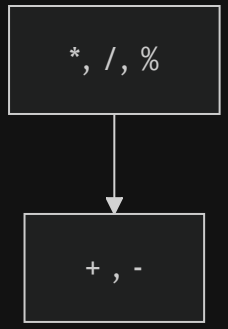
Particularités et Erreurs Courantes

Soyez vigilant aux pièges et erreurs :

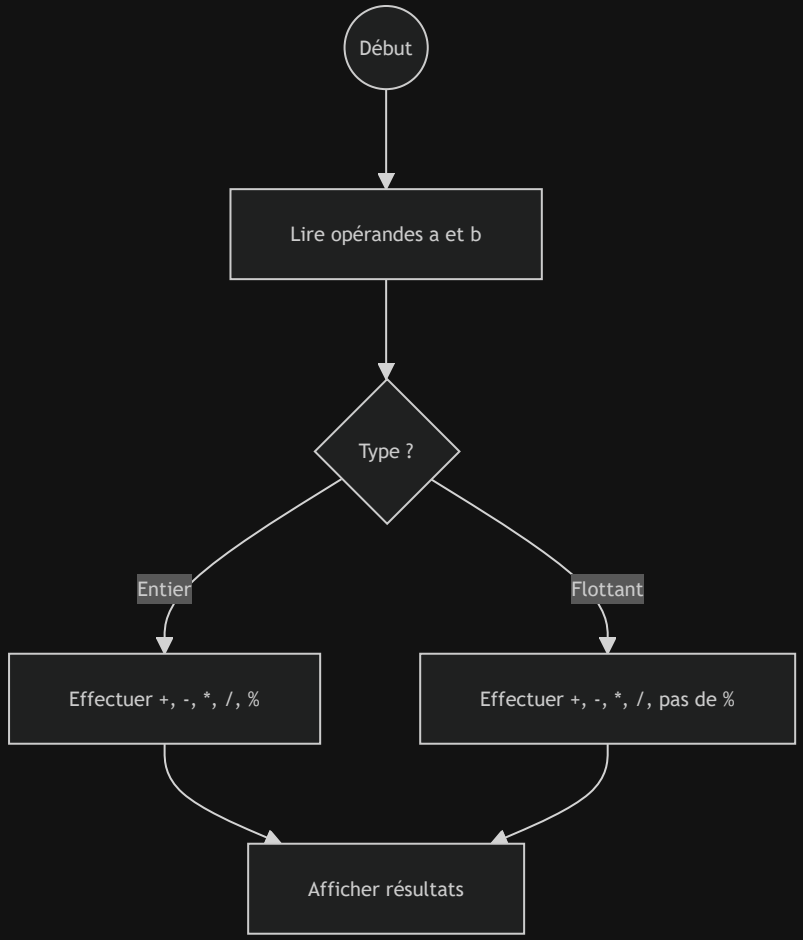
- **Division par zéro** : Cause d'erreur d'exécution, à éviter absolument.
- Le modulo `%` n'existe pas pour les flottants (utiliser `fmod` de `<math.h>` pour cela).
- Dans les calculs entiers, l'opération peut provoquer un **overflow** (dépassement de capacité) non détecté par défaut.

Précédence des Opérateurs

En C, les opérateurs arithmétiques ont un ordre de priorité :



- `*`, `/`, `%` sont évalués avant `+` et `-`.
- Les opérations du même niveau s’exécutent de gauche à droite.


Début
Lire opérandes a et b
Type ?
Entier
Flottant
Effectuer +, -, *, /, %
Effectuer +, -, *, /, pas de %
Afficher résultats

Ce qu’il faut retenir :

Les opérateurs arithmétiques sont le socle des calculs en C. Savoir les utiliser avec attention sur les types et la gestion des erreurs est indispensable pour écrire un code efficace et correct.

Sources utilisées :

- cppreference.com - Arithmetic operators
- ISO/IEC 9899:2018 – Langage C standard (section 6.5 Expressions)
- TutorialsPoint - C Arithmetic Operators
- GNU GCC Documentation - Expressions

Opérateurs d’Affectation en C

- **Modifier la valeur d’une variable**
- L’opérateur fondamental est `=`, qui **assigne** la valeur à droite à la variable à gauche.
- Les opérateurs composés combinent une opération arithmétique avec une affectation.
- Ils simplifient l’écriture et la lecture du code en rendant les expressions plus concises.

Maîtriser les Opérateurs Composés

Ces opérateurs sont essentiels pour des modifications efficaces des variables.

Opérateur	Fonction	Exemple équivalent
`=`	Affectation simple	`a = b`
`+=`	Addition puis affectation	`a = a + b`
`-=`	Soustraction puis affectation	`a = a - b`
`*=`	Multiplication puis affectation	`a = a * b`
`/=`	Division puis affectation	`a = a / b`
`%=`	Modulo puis affectation (entiers uniquement)	`a = a % b`

Comment ça marche ? Syntaxe et Principe

Comprendre la syntaxe aide à visualiser le comportement.

```
int a = 10; // 'a' est initialisé à 10
int b = 5;  // 'b' est initialisé à 5

a += b;  // équivalent à : a = a + b;  → a vaut 15
a -= b;  // équivalent à : a = a - b;  → a vaut 10 (15 - 5)
a *= b;  // équivalent à : a = a * b;  → a vaut 50 (10 * 5)
a /= b;  // équivalent à : a = a / b;  → a vaut 10 (50 / 5)
a %= b;  // équivalent à : a = a % b;  → a vaut 0 (reste de 10 / 5)
```

- L'opérande gauche doit être une variable modifiable (lvalue).
- Les opérateurs composés sont évalués de gauche à droite.
- Ils retournent la nouvelle valeur de la variable.

Mise en Pratique : Un Exemple Complet

Observons le comportement en temps réel avec un exemple détaillé.

```c
#include <stdio.h>

int main() {
    int compteur = 20;
    printf("Initial : compteur = %d\n", compteur);

    compteur += 5;   // compteur = 25
    printf("Après += 5 : compteur = %d\n", compteur);

    compteur -= 3;   // compteur = 22
    printf("Après -= 3 : compteur = %d\n", compteur);

    compteur *= 2;   // compteur = 44
    printf("Après *= 2 : compteur = %d\n", compteur);

    compteur /= 4;   // compteur = 11
    printf("Après /= 4 : compteur = %d\n", compteur);

    compteur %= 5;   // compteur = 1 (reste de 11 divisé par 5)
    printf("Après %%= 5 : compteur = %d\n", compteur);
```

Astuces et Pièges à Éviter

Quelques particularités et précautions à garder à l'esprit :

- L'opérateur `%=` n'est valide que pour les **types entiers**.
- Pour `/=` et `%=`, **ne jamais diviser par zéro** (erreur d'exécution).
- Les opérateurs composés effectuent une affectation **après** l'opération arithmétique.
- Utilisez-les pour rendre votre code plus **concis et lisible**.
- Applicables aussi aux types flottants pour `+=`, `-=`, `*=`, `/=`.

Ce qu’il faut retenir :

Les opérateurs d’affectation composés permettent d’exprimer des modifications sur des variables de manière concise et claire, tout en respectant la logique arithmétique sous-jacente.

Sources utilisées :

- cppreference.com - Assignment operators
- ISO/IEC 9899:2018 (C18 Standard) - section sur opérateurs d’affectation
- TutorialsPoint - C Assignment Operator
- GNU GCC Documentation - Expressions

Opérateurs `++` et `--` : Les fondamentaux

Ces opérateurs permettent de modifier une variable entière en ajoutant ou soustrayant 1, de manière concise.

- `++` : **Incrémentation**
  - Augmente la valeur d'une variable de 1.
- `--` : **Décrémentation**
  - Diminue la valeur d'une variable de 1.

Point clé : Uniquement applicables aux variables modifiables (lvalues).

Préfixe ou Suffixe : Une question de timing

La position de l'opérateur détermine le moment de la modification.

- **Pré-incrémentation / Pré-décrémentation (** `++var` **,** `--var` **)**
  - La variable est modifiée **avant** d'être utilisée dans l'expression.
- **Post-incrémentation / Post-décrémentation (** `var++` **,** `var--` **)**
  - La variable est utilisée dans l'expression **avant** d'être modifiée.

Illustration des différences : Préfixe vs Suffixe

```
int a = 5;
int b;

// Pré-incrémentation
b = ++a;
// 1. 'a' passe à 6
// 2. 'b' reçoit la nouvelle valeur de 'a' (6)
// Résultat : a = 6, b = 6

a = 5; // Réinitialisation pour l'exemple suivant

// Post-incrémentation
b = a++;
// 1. 'b' reçoit l'ancienne valeur de 'a' (5)
// 2. 'a' passe à 6
// Résultat : a = 6, b = 5
```

Mise en pratique : Exemples détaillés

```c
#include <stdio.h>

int main() {
    int x = 10;
    int y;

    y = x++; // Post-incrémentation
    printf("Post-incrémentation: x = %d, y = %d\n", x, y);

    x = 10;
    y = ++x; // Pré-incrémentation
    printf("Pré-incrémentation: x = %d, y = %d\n", x, y);

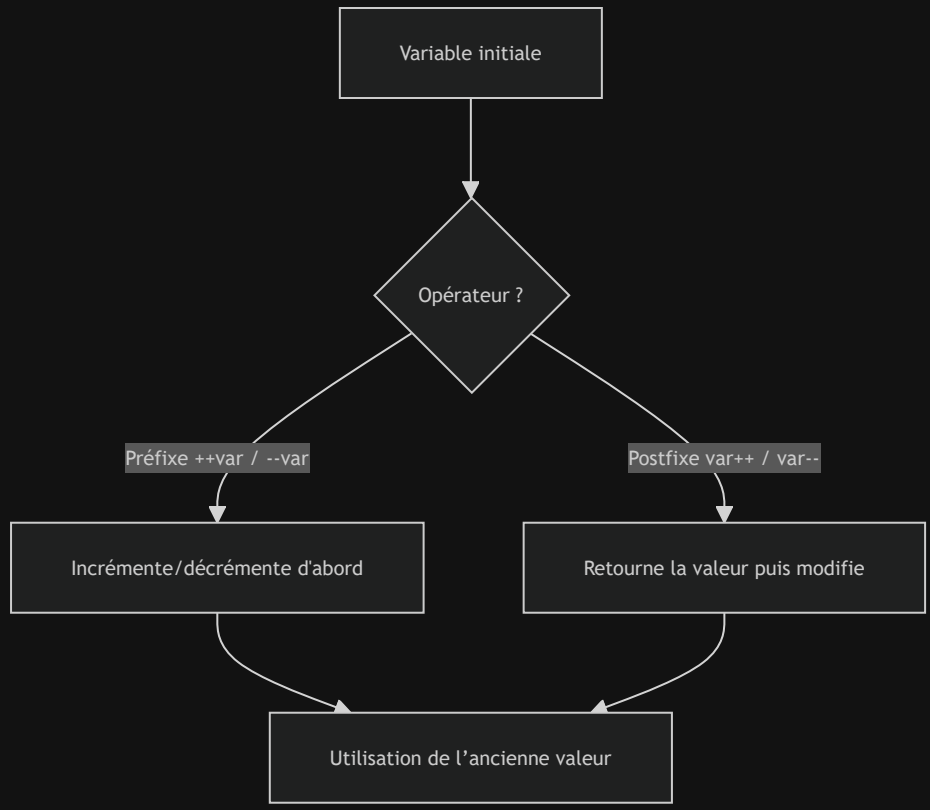
    y = x--; // Post-décrémentation
    printf("Post-décrémentation: x = %d, y = %d\n", x, y);

    x = 10;
    y = --x; // Pré-décrémentation
    printf("Pré-décrémentation: x = %d, y = %d\n", x, y);

    return 0;
}
```

Maîtriser les opérateurs : Bonnes pratiques et pièges à éviter

- **Comportement indéfini :** Éviter plusieurs opérations `++` / `--` sur la même variable dans une seule expression.
- **Lisibilité :** Privilégier des expressions simples pour un code clair et maintenable.
- **Performance / Convention :** Utiliser la forme préfixe (`++var`) dans les boucles lorsque la différence fonctionnelle n’a pas d’importance.
- **Limitations :** Ces opérateurs ne s’appliquent pas aux types flottants en C standard.


Variable initiale
Opérateur ?
Préfixe ++var / --var
Postfixe var++ / var--
Incrémente/décrémente d'abord
Retourne la valeur puis modifie
Utilisation de l’ancienne valeur

Synthèse & Approfondir

Les opérateurs `++` et `--` sont des raccourcis syntaxiques puissants. Comprendre la différence entre préfixe et suffixe est essentiel pour éviter des erreurs logiques fréquentes.

Ce qu'il faut retenir :

- **Préfixe :** Modifie, puis utilise.
- **Suffixe :** Utilise, puis modifie.
- À utiliser avec prudence dans les expressions complexes.

Sources :

- cppreference.com - Increment and decrement operators
- ISO/IEC 9899:2018 (C18 Standard) - Section 6.5.3
- TutorialsPoint - C Increment and Decrement Operators
- GNU GCC Documentation - Expressions

Les Opérateurs Relationnels en C

Comparer les valeurs pour piloter votre logique de programme

- **Rôle :** Comparer deux expressions (numériques, pointeurs) pour établir une relation.
- **Résultat :** Une valeur de type `_Bool` (0 pour faux, 1 pour vrai).
- **Importance :** Fondamentaux pour construire les structures conditionnelles (`if`) et les boucles (`while` , `for`).

Identifier les Opérateurs de Comparaison

Opérateur	Signification	Exemple	Résultat
`==`	Égal à	`a == b`	vrai si a égale b
`!=`	Différent de	`a != b`	vrai si a différent de b
`<`	Strictement inférieur à	`a < b`	vrai si a < b
`>`	Strictement supérieur à	`a > b`	vrai si a > b
`<=`	Inférieur ou égal à	`a <= b`	vrai si a ≤ b
`>=`	Supérieur ou égal à	`a >= b`	vrai si a ≥ b

Comment ça marche ?

- Ils comparent des valeurs numériques (entiers, flottants) ou des pointeurs.
- Le résultat est toujours `0` (faux) ou `1` (vrai).

Mises en Pratique : Exemples Code

```c
#include <stdio.h>

int main() {
    int a = 5, b = 7;

    printf("a == b : %d\n", a == b);  // 0 (faux)
    printf("a ≠ b : %d\n", a ≠ b);  // 1 (vrai)
    printf("a < b  : %d\n", a < b);   // 1 (vrai)
    printf("a > b  : %d\n", a > b);   // 0 (faux)
    printf("a ⩽ b : %d\n", a ⩽ b);  // 1 (vrai)
    printf("a ⩾ b : %d\n", a ⩾ b);  // 0 (faux)

    // Fonctionne aussi avec les flottants
    float x = 3.14, y = 3.14;
    printf("x == y : %d\n", x == y);   // 1 (vrai)

    return 0;
}
```

Observer les résultats `0` ou `1` selon la vérité de l'expression.

Intégration dans les Structures Conditionnelles

- Exemple avec `if` / `else` :

```c
int age = 18;

if (age ⩾ 18) {
    printf("Accès autorisé.\n");
} else {
    printf("Accès refusé.\n");
}
```

- Conversion en booléen (`_Bool` / `bool`) :
  - Le résultat (`0` ou `1`) est compatible avec le type `_Bool` (introduit en C99).
  - Pour une meilleure lisibilité, utilisez `bool` avec `<stdbool.h>` :

```c
#include <stdbool.h>

bool est_adulte = (age ⩾ 18); // est_adulte sera vrai ou faux
```

Ceci permet d'écrire un code plus clair et expressif.

Recommandations et Processus d’Évaluation

Bonnes pratiques pour éviter les pièges et comprendre le flux

- Précautions :
  - **Flottants :** Attention aux imprécisions de comparaison dues à leur représentation.
  - **`==` (égalité) vs `=` (affectation) :** Une confusion fréquente pouvant entraîner des bugs.
  - **Lisibilité :** Préférez toujours un espace autour des opérateurs.

- Processus d’évaluation d’une expression relationnelle :

Maîtriser la Comparaison en C

Ce qu'il faut retenir : Les opérateurs relationnels sont des outils indispensables pour comparer des valeurs et construire la logique conditionnelle de tout programme C. Ils fournissent un moyen clair et simple de prendre des décisions et de contrôler le flux d'exécution.

Ressources complémentaires :

- cppreference.com - Relational operators
- ISO/IEC 9899:2018 (C18 Standard) - section 6.5.8
- TutorialsPoint - C Relational Operators
- GNU GCC Documentation - Expressions

Opérateurs Logiques en C

Les opérateurs logiques sont essentiels pour créer des conditions complexes. Ils permettent de combiner ou d'inverser des expressions booléennes (vraies ou fausses), un mécanisme fondamental pour les structures de contrôle comme les conditions (`if` , `else`) et les boucles.

Les Opérateurs Logiques : `&&` , `||` , `!`

Opérateur	Nom	Fonction	Exemple
`&&`	ET logique	Résulte en vrai (1) si **les deux** opérandes sont vrais	`(a > 0) && (b < 10)`
`||`	OU logique	Résulte en vrai si **au moins un** des opérandes est vrai	`(x == 5) || (y ≠ 3)`
`!`	NON logique	Inverse la valeur booléenne (vrai devient faux, et vice versa)	`!(a == b)`

Comportement Avancé et Évaluation Paresseuse

- En C, `0` représente *faux* et toute valeur non nulle représente *vrai*.
- `&&` et `||` effectuent une **évaluation paresseuse** ("short-circuit") :
  - Pour `&&`, si le premier opérande est faux, le deuxième n'est pas évalué.
  - Pour `||`, si le premier opérande est vrai, le deuxième n'est pas évalué.
- L'opérateur `!` ne prend qu'un seul opérande et inverse sa valeur booléenne.

Exemples Pratiques

```c
#include <stdio.h>

int main() {
    int a = 5, b = 10, c = 0;

    printf("a > 0 && b < 20 : %d\n", (a > 0) && (b < 20)); // 1 (vrai)
    printf("a > 0 && c ≠ 0 : %d\n", (a > 0) && (c ≠ 0)); // 0 (faux)
    printf("a == 5 || b == 5 : %d\n", (a == 5) || (b == 5)); // 1 (vrai)
    printf("! (a == b) : %d\n", !(a == b));                  // 1 (vrai)

    // Illustration évaluation paresseuse
    int x = 0;
    if ((x ≠ 0) && (10 / x > 1)) { // La division par zéro est évitée
        printf("Pas d'erreur, second test non exécuté.\n");
    } else {
        printf("Premier test faux, second non évalué.\n");
    }

    return 0;
}
```

Précédence et Associativité

La précédence détermine l'ordre d'évaluation des opérateurs dans une expression, et l'associativité gère les opérateurs de même précédence.

Opérateur	Description	Précédence (plus haut d'abord)	Associativité
`!`	NON logique	3	Droite à gauche
`&&`	ET logique	2	Gauche à droite
`||`	OU logique	1	Gauche à droite

Bonnes Pratiques & Ressources

Bonnes Pratiques :

- Utiliser les opérateurs logiques pour combiner clairement les conditions.
- Exploiter l'évaluation paresseuse pour éviter des calculs coûteux ou des erreurs (exemple division par zéro).
- Toujours utiliser les parenthèses pour clarifier la priorité des opérations logiques.
- Attention à ne pas confondre `&` (ET bit-à-bit) avec `&&` (ET logique).

Ce qu'il faut retenir : Les opérateurs logiques sont cruciaux pour contrôler le flux de vos programmes en C. Une compréhension précise de leur fonctionnement, en particulier de l'évaluation paresseuse, est essentielle pour éviter les erreurs et écrire du code robuste.

Sources utilisées :

- cppreference.com - Logical AND, OR, NOT operators
- ISO/IEC 9899:2018 (C18 Standard) - section 6.5.13-15
- TutorialsPoint - C Logical Operators
- GNU GCC Documentation - Expressions

Décrypter l’Ordre des Opérations

Lorsque plusieurs opérateurs sont présents dans une expression, leur ordre d’évaluation est crucial et déterminé par deux concepts :

- **Priorité (ou précédence)** : Définit quel opérateur est évalué en premier. *(Ex: la multiplication avant l’addition)*
- **Associativité** : Définit le sens d’évaluation (gauche-à-droite ou droite-à-gauche) pour les opérateurs de *même* priorité. *(Ex: `a - b - c` est évalué de gauche à droite)*

Pourquoi est-ce important ? Comprendre ces règles permet d’éviter les erreurs logiques et d’assurer que votre code est évalué comme prévu. Pour forcer un ordre, utilisez toujours les parenthèses `()`.

Le Tableau des Priorités et Associativités Clés

Priorité	Opérateurs	Associativité	Exemple
1	`()` (parenthèses)	-	`(a + b) * c`
2	`++` (pré-inc), `--` (pré-déc), `!`, `~`, `+` (unaire), `-` (unaire)	Droite à gauche	`-a`, `++i`
3	`*`, `/`, `%`	Gauche à droite	`a * b / c`
4	`+`, `-`	Gauche à droite	`a + b - c`
5	`<`, `⩽`, `>`, `⩾`	Gauche à droite	`a < b && b ⩽ c`
6	`⩵`, `≠`	Gauche à droite	`a ⩵ b`
7	`&&`	Gauche à droite	`a && b`
8	`		`

La Priorité en Action : `a + b * c`

Comprendre comment une expression est évaluée pas à pas :

```c
int a = 5, b = 10, c = 15;
int resultat;

resultat = a + b * c;
```

Évaluation :

1. `*` a une priorité plus élevée que `+`.
2. Donc `b * c` est calculé en premier : `10 * 15 = 150`.
3. Ensuite `a + 150` est évalué : `5 + 150 = 155`. `resultat` sera `155`.

Forcer l’ordre avec les parenthèses :

```
resultat = (a + b) * c;
```

Évaluation :

1. Les parenthèses forcent `a + b` à s’évaluer en premier : `5 + 10 = 15`.
2. Ensuite, la multiplication `15 * c` est effectuée : `15 * 15 = 225`. `resultat` sera `225`.

L’Associativité Démontrée

Opérateurs de Gauche à Droite :

```
int x = 10, y = 5, z = 2;
int valeur;

valeur = x - y - z; // L'opération se fait de gauche à droite
                    // Équivalent à : ((10 - 5) - 2)
                    // (5 - 2) = 3
```

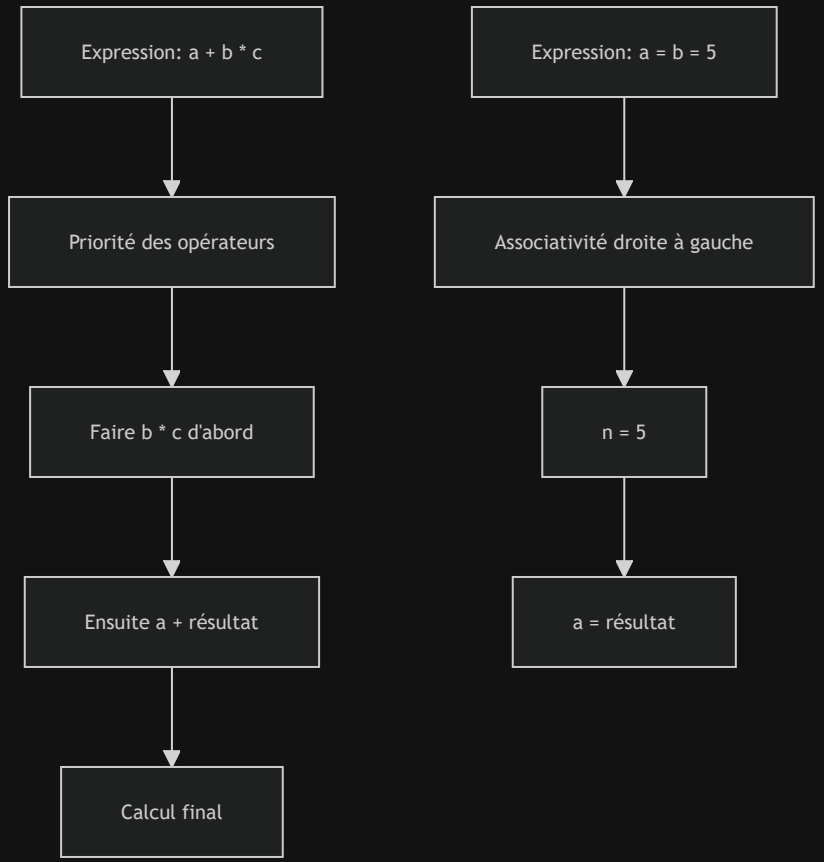
`valeur` sera `3`.

Opérateurs de Droite à Gauche (Affectation) :

```
int m, n;
m = n = 7; // L'affectation se fait de droite à gauche
           // Équivalent à : m = (n = 7)
           // 1. n reçoit 7
           // 2. m reçoit la valeur de l'affectation (7)
```

`m` sera `7` et `n` sera `7`.

Visualiser la Logique d’Évaluation



En Bref et Pour Aller Plus Loin

Ce qu'il faut retenir :

- Les opérations entre parenthèses `( )` ont toujours la priorité absolue.
- Les opérateurs unaires (`!` , `++` , `--`) sont évalués avant les binaires (`+` , `*`).
- Multiplication (`*`), division (`/`), et modulo (`%`) précèdent l'addition (`+`) et la soustraction (`-`).
- Les opérateurs logiques (`&&` , `||`) et relationnels (`==` , `≠`) ont des priorités plus faibles.
- Les affectations (`=` , `+=` , etc.) sont généralement évaluées en dernier, avec une associativité de droite à gauche.
- Utilisez systématiquement les parenthèses pour clarifier les expressions complexes et éviter toute ambiguïté.

Sources utilisées :

- cppreference.com - Operator precedence
- ISO/IEC 9899:2018 (C18 Standard) - Section 6.5
- TutorialsPoint - C Operator Precedence and Associativity

`printf()` - La porte de sortie de votre programme C

Qu'est-ce que `printf()` ?

Une fonction standard du langage C (définie dans `<stdio.h>`) pour afficher des données formatées sur la sortie standard (généralement la console).

Son prototype principal :

```c
int printf(const char *format, ...);
```

- `format` : Chaîne de caractères spécifiant le format de sortie, incluant texte brut et spécificateurs de format.
- `...` : Liste variable d'arguments correspondant aux spécificateurs.

Les Spécificateurs de Format Essentiels

Comment ça marche ?

- Les spécificateurs indiquent à `printf` comment interpréter les arguments.
- L'ordre des arguments doit correspondre *strictement* aux spécificateurs.
- La fonction retourne le nombre de caractères affichés.

Spécificateur	Type de données attendu	Description	Exemple
`%d`	`int`	Affiche un entier en base décimale	`printf("%d", 42);`
`%f`	`float` ou `double`	Affiche un nombre à virgule flottante	`printf("%f", 3.14);`
`%c`	`char`	Affiche un caractère unique	`printf("%c", 'A');`

`printf()` en Action : Exemples concrets

```c
#include <stdio.h>

int main() {
    int age = 25;
    float taille = 1.75;
    char initiale = 'J';
    char nom[] = "Dupont";

    printf("Nom: %s\n", nom);
    printf("Initiale: %c\n", initiale);
    printf("Âge: %d ans\n", age);
    printf("Taille: %f mètres\n", taille);

    return 0;
}
```

Séquences d'échappement communes :

- `\n` : saut de ligne
- `\t` : tabulation horizontale
- `\\` : antislash littéral
- `\"` : guillemet double littéral

Maîtriser l'Affichage et Comprendre le Processus

Contrôle de l'affichage des nombres à virgule flottante :

- Par défaut, `%f` affiche 6 chiffres après la virgule.
- Utilisez `%.nf` pour spécifier `n` décimales.

```
printf("Taille avec 2 décimales: %.2f mètres\n", taille); // Sortie: 1.75 mètres
```

Adopter les Bonnes Pratiques avec `printf()`

- **Vérification stricte :** Toujours s'assurer de la correspondance exacte entre les types d'arguments et les spécificateurs pour éviter les comportements indéfinis.
- **Précision :** Utiliser la précision (`%.nf`) quand c'est nécessaire pour un affichage contrôlé des flottants.
- **Documentation :** Pour d'autres types de formatage (hexadécimal `%x` , octal `%o` , pointeurs `%p` , etc.), se référer à la documentation.
- **Sécurité (code avancé) :** Privilégier les fonctions de formatage dans des buffers sécurisés (comme `snprintf`) pour prévenir les risques d'erreurs ou de débordements.

Synthèse et Ressources

Ce qu'il faut retenir : La fonction `printf` est une des principales interfaces de sortie en C. Son système de formatage puissant permet d'afficher aussi bien du texte simple que des données diverses dans des formats contrôlés, ce qui est essentiel pour le débogage et l'interaction utilisateur.

Sources utilisées :

- cppreference.com - printf
- ISO/IEC 9899:2018 (C18 Standard) - Chapitre 7.21.6.1
- TutorialsPoint - C - printf() Function
- GNU GCC Documentation - printf Format

`scanf()` : Lecture d'Entrée Utilisateur

- Qu'est-ce que `scanf()` ?
  - Fonction C qui permet la lecture **formatée** depuis l'entrée standard (généralement le clavier).
  - Définie dans la bibliothèque `<stdio.h>`.
  - Récupère des données saisies par l'utilisateur, en les typant et convertissant selon des spécificateurs.
- Prototype simplifié :

```c
int scanf(const char *format, ...);
```

  - `format` : Chaîne de format avec des spécificateurs adaptés à la lecture.
  - `...` : Adresses des variables où stocker les valeurs lues.

Spécificateurs Clés et Adresses (`&`)

Spécificateur	Type attendu	Exemples
`%d`	Entier (`int`)	`scanf("%d", &age);`
`%f`	Nombre à virgule flottante (`float`)	`scanf("%f", &taille);`
`%c`	Caractère (`char`)	`scanf(" %c", &lettre);`
`%s`	Chaîne de caractères (`char[]`)	`scanf("%s", nom);`

- **Note sur `%s` :** Lit une chaîne jusqu'au premier espace, tabulation ou saut de ligne.
- **Importance de l'opérateur `&` (adresse) :**
  - `scanf()` a besoin de l'**adresse** des variables pour pouvoir modifier leur contenu.
  - Pour les types simples (`int` , `float` , `char`), on utilise `&var` .

```c
#include <stdio.h>

int main() {
    int age;
    float taille;
    char initiale;
    char nom[30];

    printf("Entrez votre âge : ");
    scanf("%d", &age);

    printf("Entrez votre taille en mètres : ");
    scanf("%f", &taille);

    printf("Entrez la première lettre de votre prénom : ");
    scanf(" %c", &initiale); // L'espace avant %c est important

    printf("Entrez votre nom : ");
    scanf("%s", nom);

    printf("\nRésumé :\n");
    printf("Nom : %s\n", nom);
    printf("Initiale : %c\n", initiale);
    printf("Âge : %d\n", age);
    printf("Taille : %.2f m\n", taille);
```

Pièges Courants et Conseils

1. **L'espace blanc avant `%c`**
   - Crucial pour consommer les caractères blancs (ex: retour à la ligne) laissés dans le buffer par une saisie précédente.
2. **Ne pas oublier `&`**
   - Essentiel pour les variables scalaires (`int` , `float` , `char`). L'oublier peut causer des erreurs graves.
   - **Rappel :** Pas besoin de `&` pour les tableaux de caractères (`char[]`).
3. **Attention au format de lecture**
   - La correspondance exacte entre le spécificateur de `scanf()` et le type de la variable est impérative pour une lecture correcte.
   - Une mauvaise correspondance entraîne des lectures erronées ou des comportements indéfinis.
4. **Gestion des retours à la ligne et du buffer d'entrée**
   - Le buffer d'entrée n'est pas automatiquement vidé, ce qui peut impacter les lectures subséquentes.

L’Essentiel à Retenir

Ce qu’il faut retenir :

- `scanf()` : un outil puissant pour lire des données typées depuis l’entrée standard.
- **Adresses (`&`) :** Toujours utiliser `&` pour les variables scalaires, sauf pour les chaînes de caractères (tableaux).
- `%c` : Pensez à l’espace blanc (`" %c"`) pour gérer les caractères résiduels.
- **Concordance :** Assurez-vous que les spécificateurs de format (`%d` , `%f` , etc.) correspondent aux types des variables cibles.
- **Robustesse :** Soyez conscient des potentielles complications liées au buffer d’entrée.

Sources utilisées :

- cppreference.com - `scanf`
- ISO/IEC 9899:2018 (C18 Standard) - Chapitre 7.21.6.2
- TutorialsPoint - C - scanf() Function
- GNU GCC Documentation - Input/output

Gestion des retours de `scanf()`

La fonction `scanf()` renvoie un **entier** qui indique le **nombre d'éléments** correctement lus et affectés. Cette valeur est votre premier indicateur de succès ou d'échec de l'opération.

- **Succès total :** Le retour est égal au nombre de spécificateurs de format attendus.
- **Échec partiel ou total :** Le retour est inférieur au nombre attendu (par exemple, entrée incompatible).
- **Erreur critique :** Le retour peut être `EOF` (End Of File, souvent -1), signalant une fin de fichier ou un problème majeur d'entrée.

Pourquoi cette vigilance est-elle indispensable ?

- **Éviter l'utilisation de données non valides** qui peuvent provoquer un comportement indéfini.
- **Contrôler la validité de l'entrée utilisateur** et réagir de manière appropriée (messages d'erreur, nouvelle saisie).
- **Construire un programme robuste**, capable de gérer les imprévus liés aux entrées.

Exemple simple : Détection d’une entrée invalide

Un test conditionnel basique suffit pour vérifier si `scanf()` a rempli sa mission pour une seule donnée.

```c
#include <stdio.h>

int main() {
    int age;
    printf("Entrez votre âge : ");

    int ret = scanf("%d", &age); // Tentative de lire un entier

    if (ret == 1) { // Attendue 1 lecture réussie
        printf("Âge enregistré : %d\n", age);
    } else {
        printf("Erreur : entrée invalide.\n");
    }
    return 0;
}
```

Explication :

- `scanf("%d", &age)` tente de lire un entier.
- Si `ret` vaut 1, la lecture s’est déroulée comme prévu.

Garantir une saisie valide : la boucle de validation

Pour des programmes interactifs, il est souvent nécessaire de redemander la saisie tant que l'entrée est incorrecte. La boucle `do-while` est idéale.

```c
#include <stdio.h>

int main() {
    int age;
    int ret;

    do {
        printf("Entrez votre âge : ");
        ret = scanf("%d", &age);

        if (ret ≠ 1) {
            printf("Entrée invalide. Veuillez recommencer.\n");
            // Nettoyage du buffer pour éviter une boucle infinie
            int c;
            while ((c = getchar()) ≠ '\n' && c ≠ EOF);
        }
    } while (ret ≠ 1); // Répéter tant que la lecture n'est pas correcte

    printf("Âge enregistré : %d\n", age);
    return 0;
}
```

Points clés :

- Le `do-while` assure au moins une tentative de saisie et répète en cas d'erreur.
- `while ((c = getchar()) ≠ '\n' && c ≠ EOF);` est crucial pour **vider le buffer d'entrée** et éliminer les caractères erronés, empêchant ainsi `scanf()` de relire la même erreur indéfiniment.

Cas multiples et visualisation du processus

Lecture de plusieurs valeurs : Le principe reste le même : le retour de `scanf()` doit correspondre au nombre d'éléments attendus.

```c
int a, b;
int ret = scanf("%d %d", &a, &b); // Attend 2 entiers

if (ret == 2) { // Si 2 entiers ont été lus
    printf("Vous avez saisi %d et %d\n", a, b);
} else {
    printf("Erreur lors de la saisie de deux entiers\n");
}
```

Bonnes pratiques pour une robustesse accrue

- **Toujours tester la valeur de retour** de `scanf()`. Ne jamais présumer que l'entrée utilisateur est parfaite.
- **Nettoyer le buffer d'entrée** après une erreur pour éviter de relire la même donnée incorrecte, ce qui pourrait conduire à une boucle infinie.
- **Ne jamais supposer** que `scanf()` lira parfaitement l'entrée utilisateur. L'utilisateur peut toujours saisir n'importe quoi.
- Pour des interfaces plus complexes ou des exigences de sécurité élevées, envisager des méthodes de lecture plus sécurisées comme `fgets` combiné à un parsing manuel (`sscanf` , `strtol` , etc.).

En synthèse & Références

Gérer le retour de `scanf()` est la clé pour valider les données entrantes et garantir la fiabilité de la lecture, un élément indispensable pour tout programme interactif écrit en C. C'est la base pour des applications robustes et prévisibles.

Sources utilisées :

- cppreference.com - scanf
- ISO/IEC 9899:2018 (C18 Standard) - Section 7.21.6.2
- TutorialsPoint - C - scanf() Function
- Learn-C.org - Input/Output

Structures de Contrôle en C : L’instruction `if`

Introduction : Le `if` pour une exécution conditionnelle

L'instruction `if` en langage C est fondamentale pour contrôler le flux d'exécution. Elle permet d'exécuter un bloc de code **uniquement si une condition est vraie**.

Syntaxe de base :

```
if (condition) {
    // bloc de code exécuté si condition vraie
}
```

- La `condition` est une expression entière.
- En C, une condition est **vraie** si sa valeur est non nulle, et **fausse** si elle vaut zéro.
- Le bloc entre accolades contient les instructions à exécuter.

Les Syntaxes Courantes de `if`

L'instruction `if` offre plusieurs formes pour s'adapter à divers scénarios de décision.

1. `if` simple :

```
if (x > 0) {
    printf("x est positif\n");
} // Exécute si Vrai, sinon ignore
```

2. `if` ... `else` :

```
if (x % 2 == 0) {
    printf("x est pair\n");
} else {
    printf("x est impair\n");
} // Exécute un bloc ou l'autre
```

3. `if` … `else if` … `else` :

```
if (score ⩾ 90) {
    printf("Excellent\n");
} else if (score ⩾ 75) {
    printf("Bien\n");
} else if (score ⩾ 50) {
    printf("Passable\n");
} else {
    printf("Échec\n");
} // Teste plusieurs conditions alternatives en cascade
```

Bonnes Pratiques et Précautions

Pour un code clair et sans erreur avec `if`.

- **Accolades :** Indispensables pour les blocs de plusieurs instructions. Recommandées même pour une seule instruction pour la clarté.
- **Opérateurs :**
  - **Relationnels :** `==`, `!=`, `<`, `>`, `<=`, `>=`.
  - **Logiques :** `&&` (ET), `||` (OU), `!` (NON).
- **Attention `==` vs `=` :** L'opérateur d'égalité est `==`, l'opérateur d'affectation est `=`. Une erreur fréquente !
  - `if (x = 0)` affecte 0 à `x`, puis évalue la condition (fausse).
  - `if (x == 0)` compare `x` à 0.
- **Priorité des opérateurs :** Utilisez des parenthèses pour clarifier les expressions complexes.

Exemples Illustratifs

Exemple 1 : Vérification d'un nombre (positif, négatif ou nul)

```c
#include <stdio.h>

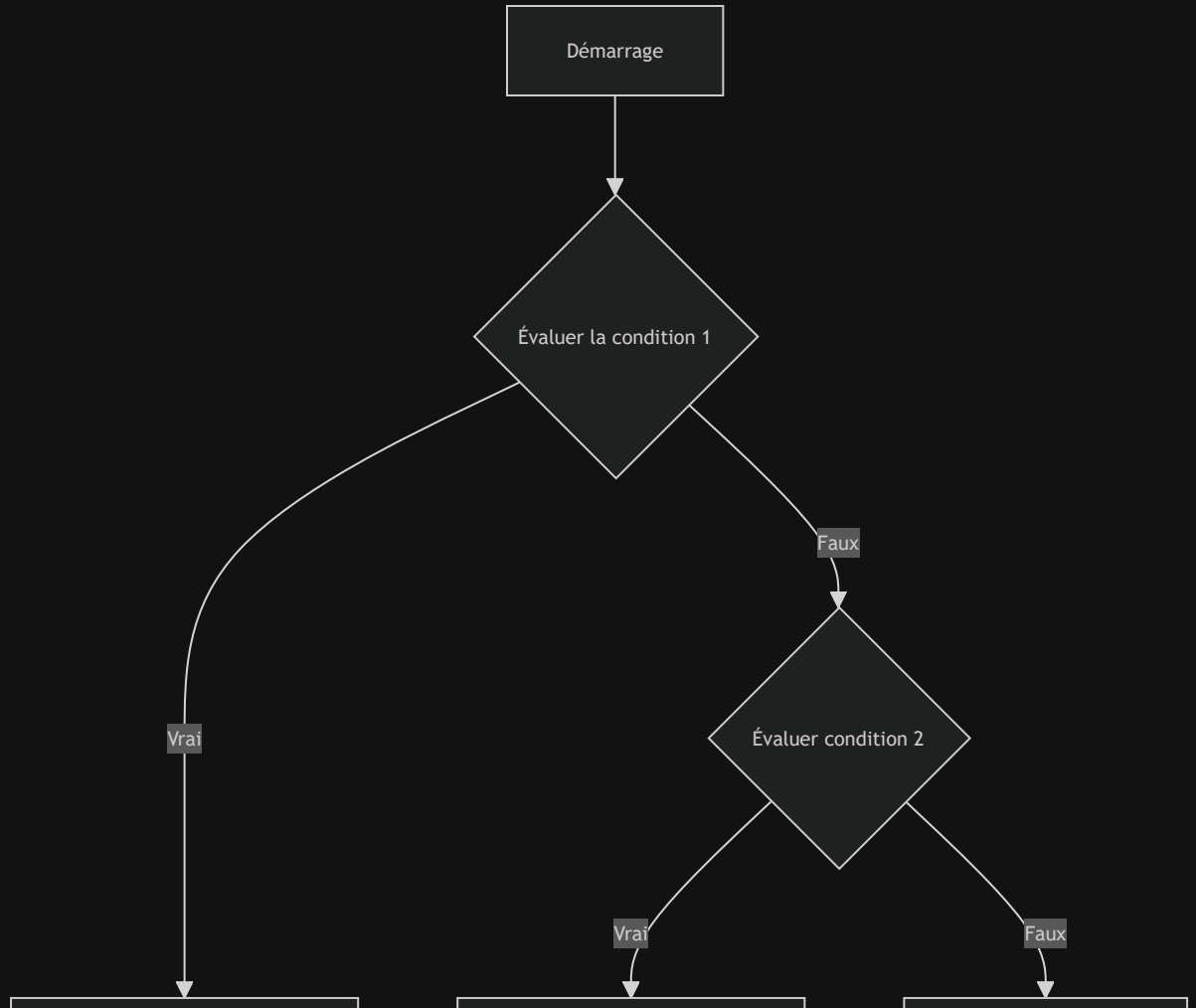
int main() {
    int nombre;
    printf("Entrez un nombre : ");
    scanf("%d", &nombre);

    if (nombre > 0) {
        printf("Le nombre est positif.\n");
    } else if (nombre < 0) {
        printf("Le nombre est négatif.\n");
    } else {
        printf("Le nombre est nul.\n");
    }
    return 0;
}
```

Exemple 2 : Validation d’âge pour l’accès

```c
int age = 20;

if (age ⩾ 18) {
    printf("Accès autorisé.\n");
} else {
    printf("Accès refusé.\n");
}
```


Démarrage
Évaluer la condition 1
Vrai
Faux
Évaluer condition 2
Vrai
Faux

Ce qu'il faut retenir & Sources

L'essentiel à retenir :

- L'instruction `if` est la base de la prise de décision, permettant une exécution conditionnelle.
- Elle peut être combinée avec `else` pour gérer des alternatives binaires.
- `else if` permet de chaîner plusieurs conditions exclusives.
- Vérifier attentivement la syntaxe, en particulier `==` pour la comparaison et l'utilisation des accolades.
- Privilégier la lisibilité et la clarté du code.

Sources :

- cppreference.com - if statement
- ISO/IEC 9899:2018 - Chapitre 6.8.4.1
- TutorialsPoint - C - if else Statement
- GeeksforGeeks - if-else Statement in C

Séance 3 : Structures de Contrôle

Partie 1 : Les Conditions - L'instruction `if-else`

L'instruction `if-else` est un mécanisme fondamental pour introduire la prise de décision dans vos programmes. Elle permet de gérer des décisions binaires simples, dirigeant l'exécution vers l'un de deux chemins alternatifs en fonction de la validité d'une condition.

Le `if-else` : Principe et Syntaxe

Principe : Choisir entre deux alternatives L'instruction `if-else` permet de **choisir entre deux blocs d'instructions** selon la validité d'une condition.

- Si la condition est vraie (non nulle), le bloc situé après `if` est exécuté.
- Si la condition est fausse (zéro), c'est le bloc situé après `else` qui est exécuté.

Syntaxe de base

```
if (condition) {
    // Instructions exécutées si condition vraie
} else {
    // Instructions exécutées si condition fausse
}
```

- La condition est une expression évaluée comme vraie ou fausse (vrai ≠ 0, faux = 0).
- Les blocs sont entourés d'accolades `{}` .

Exemples Pratiques de `if-else`

Exemple 1 : Vérifier la parité d'un nombre

```c
#include <stdio.h>

int main() {
    int nombre;
    printf("Saisissez un nombre : ");
    scanf("%d", &nombre);

    if (nombre % 2 == 0) { // La condition teste si le reste de la division par 2 vaut 0.
        printf("Le nombre est pair.\n");
    } else {
        printf("Le nombre est impair.\n");
    }
    return 0;
}
```

Exemple 2 : Déterminer la majorité

```c
int age = 17;

if (age ⩾ 18) {
    printf("Vous êtes majeur.\n");
} else {
```

Bonnes Pratiques et Conditions Étendues

Pour une meilleure lisibilité et fiabilité :

- **Accolades toujours :** Utiliser des accolades `{}` même pour une seule instruction. Cela évite des erreurs lors de modifications futures.

```
if (x > 0)
    printf("Positif\n"); // Possible mais risqué avec ajout futur
```

- **Égalité vs Affectation :** Vérifier la condition avec l'opérateur `==` (égalité) et non avec `=` (affectation).

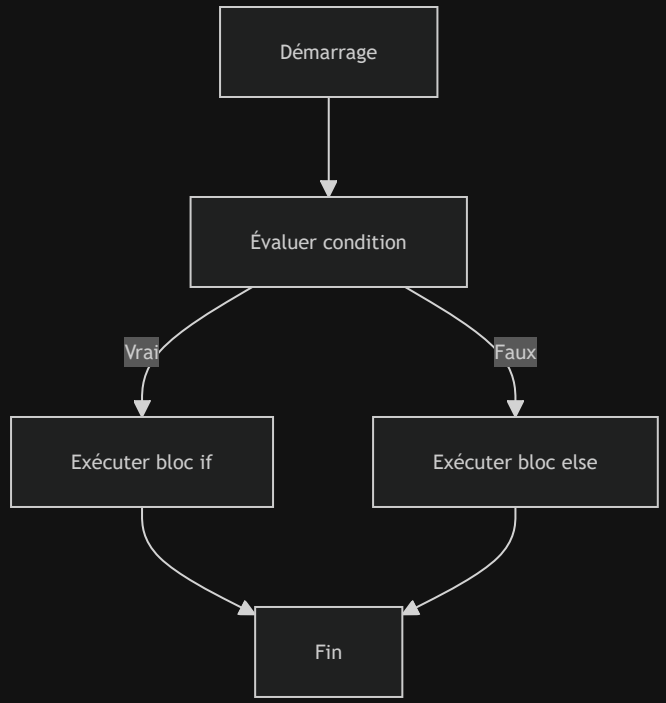
Étendue des conditions : Les opérateurs logiques Les conditions peuvent combiner plusieurs expressions avec :

- `&&` : ET logique (toutes les parties doivent être vraies)
- `||` : OU logique (au moins une partie doit être vraie)
- `!` : NON logique (inverse la valeur de vérité)

```
if (x > 0 && x < 10) {
    printf("x est entre 1 et 9\n");
} else {
```

Visualiser le flux d’exécution du `if-else`

Le diagramme de flux ci-dessous illustre le parcours de votre programme selon que la condition est vraie ou fausse.



Ce qu’il faut retenir et Ressources

L’essentiel à retenir : L’instruction `if-else` est un composant essentiel pour effectuer des choix dans un programme. Elle permet d’exécuter deux chemins d’exécution alternatifs selon le résultat d’une condition évaluée, rendant vos applications dynamiques et adaptatives.

Ressources complémentaires :

- cppreference.com - `if` statement
- ISO/IEC 9899:2018 - Chapitre 6.8.4.1
- TutorialsPoint - C `if else` Statement
- GeeksforGeeks - `if-else` Statement in C

`if-else if-else` : Choix Multiples

Concept et Fonctionnement

- Permet d'effectuer un **choix parmi plusieurs alternatives**, en testant plusieurs conditions successives.
- Le programme évalue chaque condition **dans l'ordre**.
- Dès qu'une condition est vraie, le bloc associé est exécuté, puis la structure est quittée.
- Si aucune condition n'est vraie, le bloc `else` (optionnel) est exécuté.

Syntaxe de `if-else if-else`

```
if (condition1) {
    // instructions si condition1 vraie
} else if (condition2) {
    // instructions si condition2 vraie
} else if (condition3) {
    // instructions si condition3 vraie
} else {
    // instructions si aucune condition vraie
}
```

- Le nombre de blocs `else if` est variable.
- Le bloc `else` est optionnel et sert de valeur par défaut.

Exemple : Évaluation d'une Note

```c
#include <stdio.h>

int main() {
    int note;
    printf("Entrez la note : ");
    scanf("%d", &note);

    if (note ⩾ 90) {
        printf("Mention : Excellent\n");
    } else if (note ⩾ 75) {
        printf("Mention : Très bien\n");
    } else if (note ⩾ 50) {
        printf("Mention : Passable\n");
    } else {
        printf("Mention : Échec\n");
    }
    return 0;
}
```

- Ce programme teste la note d'un étudiant et affiche une mention selon une plage de valeurs.

Logique d’Exécution et Points Clés

Optimisation et Alternatives : `switch`

- Pour des tests sur la **même variable avec des constantes**, l'instruction `switch` peut être plus adaptée.
- `switch` offre souvent une meilleure **lisibilité** et est plus performant après compilation que des `if-else if` multiples.

Ce Qu’il Faut Retenir & Ressources

Conclusion La structure `if-else if-else` offre un mécanisme flexible pour gérer des choix multiples, permettant d’orienter le flux d’exécution vers différentes réponses en fonction des conditions évaluées.

Sources utilisées

- cppreference.com - if statement
- TutorialsPoint - C if-else-if Statement
- GeeksforGeeks - if-else ladder in C
- ISO/IEC 9899:2018 (C18 standard) - Chapitre 6.8.4.1

L’opérateur ternaire `? :` - Une alternative concise à `if-else`

Introduction aux structures de contrôle : L’opérateur ternaire

L’opérateur ternaire `? :` en langage C est une **forme abrégée de l’instruction conditionnelle** `if-else`. Il permet d’écrire une expression qui retourne une valeur selon une condition, en une seule ligne.

Syntaxe générale

```
(condition) ? expression_si_vrai : expression_si_faux;
```

- **condition** : expression évaluée comme vraie (non nulle) ou fausse (zéro).
- **expression_si_vrai** : valeur ou expression retournée si la condition est vraie.
- **expression_si_faux** : valeur ou expression retournée si la condition est fausse.

Pourquoi et quand l’utiliser ?

Utilisation et avantages clés

L’opérateur ternaire est idéal pour :

- **Simplifier et raccourcir** un test simple.
- Être utilisé lorsque le résultat conditionnel est une **valeur ou une expression**.
- Éviter des blocs `if-else` courts et gagner en concision.
- S’intégrer dans des expressions plus complexes (ex : affectations, arguments de fonction).

Exemple 1 : Affectation conditionnelle

Trouver le maximum entre deux nombres

```c
int x = 10, y = 20;
int max;

// Utilisation de l'opérateur ternaire
max = (x > y) ? x : y;
printf("Le maximum est %d\n", max); // Affiche : Le maximum est 20
```

Équivalent avec `if-else` traditionnel

```c
if (x > y) {
    max = x;
} else {
    max = y;
}
```

Autres exemples d’utilisation

Exemple 2 : Affichage conditionnel

```c
int age = 18;

// Affichage d'une chaîne de caractères selon l'âge
printf("Vous êtes %s majeur.\n", (age ≥ 18) ? "un" : "un non");
// Affiche : Vous êtes un majeur.
```

Exemple 3 : Ternaire imbriqué (pour des choix multiples)

```c
int note = 75;
const char *mention;

// Détermination de la mention selon la note
mention = (note ⩾ 90) ? "Excellent" :
          (note ⩾ 75) ? "Bien" :
          (note ⩾ 50) ? "Passable" : "Échec";

printf("Mention : %s\n", mention); // Affiche : Mention : Bien
```

Points importants à retenir

- L’opérateur ternaire est une **expression**, utilisable partout où une expression est valide.
- Sa priorité est faible : utilisez des **parenthèses** pour garantir l’ordre d’évaluation.
- Pour des conditions complexes ou plusieurs instructions, **privilégiez** `if-else` pour la clarté.
- Évitez l’abus du ternaire imbriqué, qui peut nuire à la lisibilité.

En résumé & Références

Ce qu'il faut retenir

L'opérateur ternaire `? :` apporte une écriture compacte et efficace pour les choix simples dans les expressions. Il réduit la verbosité du langage C tout en respectant la logique conditionnelle fondamentale. C'est un outil précieux pour la concision lorsque bien utilisé.

Sources utilisées

- cppreference - Conditional operator (?:)
- GeeksforGeeks - Ternary Operator in C
- TutorialsPoint - C Ternary Operator
- ISO/IEC 9899:2018 (C18) – Chapitre 6.5.15 (Conditional operator)

La boucle `for` : Itérations Contrôlées

1. Qu’est-ce que la boucle `for` ?

La boucle `for` est une structure de contrôle itérative en langage C.

Elle est conçue pour exécuter un bloc d’instructions :

- Un **nombre précis** de fois.
- Lorsque le **nombre d’itérations est déterminé à l’avance**.

Idéale pour les scénarios où le compte est connu.

2. Comprendre la Syntaxe `for`

```
for (initialisation; condition; incrémentation) {
    // instructions à répéter
}
```

- `initialisation` :
  - Initialise la/les variable(s) de la boucle.
  - Exécutée **une seule fois** au début.
- `condition` :
  - Évaluée avant chaque itération.
  - La boucle continue tant qu'elle est **vraie** (non zéro).
- `incrémentation` :
  - Met à jour la/les variable(s) de contrôle.
  - Exécutée **après chaque itération**.

3. Exemples Pratiques

Afficher les nombres de 0 à 9 :

```c
#include <stdio.h>

int main() {
    for (int i = 0; i < 10; i++) { // i est 0, boucle tant que i<10, i est incrémenté
        printf("%d\n", i);
    }
    return 0;
}
```

Afficher les nombres pairs de 0 à 18 :

```c
for (int i = 0; i < 20; i += 2) { // i est 0, boucle tant que i<20, i est incrémenté de 2
    printf("%d ", i);
}
```

4. Fonctionnement Détaillé

À chaque nouvelle itération, la boucle `for` suit ces étapes :

1. La **condition** est évaluée.
2. Si elle est **vraie**, le bloc d'instructions est exécuté.
3. L'instruction d'**incrémentation** est appliquée.
4. Retour à l'étape 1.

Si la condition est fausse, la boucle se termine.

5. Optimiser l'Usage de `for`

Bonnes pratiques :

- Déclarer la variable d'itération dans le `for` (C99+).
- Bien définir la condition pour éviter les boucles infinies.
- Utiliser des types adaptés (`int` , `size_t`).
- Éviter de modifier la variable d'itération dans le corps de la boucle.

Exemples avancés :

- Somme des entiers de 1 à N :

```c
int N = 100, somme = 0;
for (int i = 1; i ⩽ N; i++) {
    somme += i;
}
```

- Plusieurs variables dans la boucle :

```c
for (int i = 0, j = 10; i < j; i++, j--) {
    printf("i = %d, j = %d\n", i, j);
}
```

6. Conclusion et Ressources

La boucle `for` est un outil puissant en C pour itérer de façon claire et contrôlée lorsque le nombre de répétitions est déterminé. Son utilisation correcte favorise un code lisible et maintenable.

Sources utilisées :

- cppreference.com - for loop
- TutorialsPoint - C for loop
- GeeksforGeeks - for loop in C
- ISO/IEC 9899:2018 (C18) – Chapitre 6.8.5.3 (for statement)

La boucle `while` : Principe et Syntaxe

1. Principe de la boucle `while`

- Exécute un bloc d'instructions **tant qu'une condition reste vraie**.
- La condition est évaluée **avant l'exécution du bloc** à chaque itération.
- Si la condition est fausse initialement, le corps de la boucle ne sera pas exécuté.
- Utile lorsque le nombre d'itérations n'est pas connu à l'avance, ou dépend de l'évolution du programme.

2. Syntaxe générale

```
while (condition) {
    // instructions répétées
}
```

* `condition` : expression évaluée comme vraie (non zéro) ou fausse (zéro). * Le corps de boucle s'exécute tant que la condition est vraie.

Exemple Simple : Compteur Croissant

```c
#include <stdio.h>

int main() {
    int compteur = 0;

    while (compteur < 5) {
        printf("Compteur = %d\n", compteur);
        compteur++;  // Important de modifier la variable pour éviter boucle infinie
    }

    return 0;
}
```

- La boucle s'arrête lorsque `compteur` vaut 5 (condition fausse).
- La variable est incrémentée à chaque tour.

Exemple Pratique : Saisie Sécurisée

```c
#include <stdio.h>

int main() {
    int age;

    printf("Entrez votre âge (≥ 0) : ");
    scanf("%d", &age);

    while (age < 0) {
        printf("Erreur. Entrez un âge valide (≥ 0) : ");
        scanf("%d", &age);
    }

    printf("Âge validé : %d\n", age);

    return 0;
}
```

* La boucle `while` permet de répéter la demande jusqu'à une saisie correcte.

Points Clés & Ressources

Points clés et bonnes pratiques

- **Prévenir une boucle infinie** : Assurer que la condition devienne fausse à un moment. La variable testée doit être modifiée dans le corps de la boucle.
- Préférer `while` quand le nombre d'itérations est **inconnu** ou dépend d'événements.
- Pour des itérations à nombre fixe, la boucle `for` est généralement plus lisible.

En résumé : La boucle `while` offre une manière naturelle de répéter un bloc d'instructions tant qu'une condition contrôlant la répétition reste vraie, particulièrement utile pour des boucles dont la durée dépend de l'évolution des données pendant l'exécution.

Sources utilisées

- cppreference.com - while statement
- TutorialsPoint - C While Loop
- GeeksforGeeks - while Loop in C
- ISO/IEC 9899:2018 (C18 standard) – Chapitre 6.8.5.2 (while statement)

Le `do-while` : Exécution au moins une fois

- La boucle `do-while` garantit que le bloc d'instructions s'exécute **au moins une fois**.
- La condition est évaluée **après l'exécution du bloc**.
- Le corps de la boucle s'exécute d'abord, puis la condition est testée.
- Elle est appropriée pour les situations où le bloc doit s'exécuter une ou plusieurs fois avant de tester la condition.

Syntaxe et Mécanisme

Syntaxe générale :

```
do {
    // instructions à répéter
} while (condition); // Le point-virgule est obligatoire
```

La condition est évaluée après chaque exécution du bloc.

Cas d’usage : Saisie utilisateur

Exemple : Saisie d’un nombre positif

```c
#include <stdio.h>

int main() {
    int nombre;

    do {
        printf("Entrez un nombre positif : ");
        scanf("%d", &nombre);
    } while (nombre < 0);

    printf("Nombre validé : %d\n", nombre);
    return 0;
}
```

- Le message est affiché et la saisie est faite **au moins une fois**.
- La vérification se fait après chaque saisie, répétant la demande en cas de valeur négative.

Cas d’usage : Menu interactif

Exemple : Menu simple avec sortie

```c
#include <stdio.h>

int main() {
    int choix;

    do {
        printf("Menu:\n1. Option A\n2. Option B\n3. Quitter\n");
        printf("Choisissez une option : ");
        scanf("%d", &choix);

        switch (choix) {
            case 1: printf("Option A sélectionnée.\n"); break;
            case 2: printf("Option B sélectionnée.\n"); break;
            case 3: printf("Fin du programme.\n"); break;
            default: printf("Option invalide.\n");
        }
    } while (choix ≠ 3); // Le menu s'affiche au moins une fois
    return 0;
}
```

`do-while` vs `while` & Bonnes Pratiques

Comparaison des boucles :

Aspect	`while`	`do-while`
Condition évaluée	Avant l'exécution	Après l'exécution
Exécution minimum	0 (peut ne jamais s'exécuter)	1 (toujours exécutée au moins une fois)
Usage typique	Boucle contrôlée par condition	Boucle nécessitant une exécution initiale obligatoire

Bonnes pratiques :

- Utiliser quand le traitement doit être effectué au moins une fois (ex: saisies utilisateurs).
- Veiller à modifier la condition pour éviter les boucles infinies.
- Éviter d'imbriquer trop de boucles `do-while` pour maintenir la lisibilité.

Ce qu’il faut retenir & Sources

La boucle `do-while` permet d’assurer une première exécution du bloc de code avant de vérifier une condition, utile notamment dans les situations où une action doit être réalisée au moins une fois avant de décider de poursuivre ou non.

Sources utilisées :

- cppreference.com - do-while statement
- TutorialsPoint - C do-while Loop
- GeeksforGeeks - do-while loop in C
- ISO/IEC 9899:2018 (C18 standard) – Chapitre 6.8.5.1 (do-while statement)

Boucles Infinies : Comprendre et Éviter

1. Qu’est-ce qu’une boucle infinie ?

- Une boucle qui ne se termine jamais car sa condition de sortie n’est jamais atteinte ou ne peut jamais être fausse.
- **Conséquences :** Blocage du programme, consommation inutile des ressources du processeur.

Les causes des boucles infinies

1. Comment cela se produit-il ?

- La condition de la boucle **reste toujours vraie**.
- Les variables contrôlant la condition **ne sont pas modifiées** à l'intérieur de la boucle.
- Erreur logique dans la condition empêchant la sortie.
- Oubli d'un mécanisme d'arrêt.
- Peut être volontaire pour des usages spécifiques, mais doit être maîtrisée.

Exemples concrets de boucles infinies

1. Oubli de mise à jour de la variable de contrôle (boucle `while`)

```c
int i = 0;
while (i < 10) {
    printf("%d\n", i);
    // i++ est oublié, la valeur reste 0 → boucle infinie
}
```

2. Condition faussement toujours vraie (boucle `for`)

```c
for (int j = 0; j ≥ 0; j++) {
    // j ≥ 0 est toujours vrai pour un entier signé (sauf débordement non garanti)
    printf("%d\n", j);
}
```

Prévenir les boucles infinies

Techniques essentielles :

- **Modifier systématiquement** la variable d'itération ou la condition durant l'exécution de la boucle.
- Tester la boucle avec des **cas d'arrêt clairs et atteignables**.
- Privilégier une **structure claire** (ex : `for` pour un nombre d'itérations connu).
- Ajouter des **instructions d'arrêt explicites (** `break` **)** si besoin, sous conditions bien définies.
- Simplifier les conditions : éviter la complexité.
- Prévoir un **système de sécurité** (nombre maximal d'itérations, temporisation).

Corriger et maîtriser les boucles

1. Correction de l'exemple `while` :

```c
int i = 0;
while (i < 10) {
    printf("%d\n", i);
    i++;  // Incrémentation nécessaire
}
```

2. Utilisation judicieuse d'un `break` :

```c
int compteur = 0;
while (1) {  // boucle infinie intentionnelle
    printf("Compteur = %d\n", compteur);
    compteur++;
    if (compteur ≥ 10) {
        break;  // sortie de la boucle
    }
}
```

3. Quand une boucle infinie est-elle intentionnelle ?

- Programmation système (processus serveurs).
- Attente d’un événement externe.
- Implémentations d’attente active ou de pollings.
- Toujours associée à un mécanisme de sortie (signal, interruption).

Points clés et ressources complémentaires

Ce qu'il faut retenir : Une vigilance constante sur la condition d'arrêt et la modification correcte de la variable(s) de contrôle évite les boucles infinies non désirées. Cette bonne habitude permet d'assurer la robustesse et la stabilité des programmes.

Sources utilisées :

- cppreference.com - While loop
- GeeksforGeeks - Infinite Loop in C
- TutorialsPoint - C Infinite loops
- ISO/IEC 9899:2018 (C18) – Chapitre 6.8.5 (boucles)

L’instruction `break` : Maîtriser les sorties anticipées

Séance 3 : Structures de Contrôle - Instructions de Saut

- **Qu’est-ce que `break` ?**
  - Une instruction clé en langage C.
  - Permet d’**interrompre immédiatement** l’exécution d’une structure de contrôle.
- **Son objectif principal**
  - Sortir prématurément d’une boucle (`for` , `while` , `do-while`).
  - Terminer l’exécution d’un bloc `switch` .
  - Passe directement à la première instruction située après la structure.

Contextes d’utilisation et Syntaxe simple

- **Où utiliser `break` ?**
  - **Dans les boucles :** Pour stopper l’itération avant sa fin naturelle, si une condition spécifique est remplie.
  - **Dans les `switch` :** Essentiel pour empêcher la poursuite dans les cas suivants (le "fall-through") après qu’un cas a été exécuté.
- **Syntaxe de l’instruction `break`**

```
break;
```

  - Ne prend aucun argument.
  - Se place à l’intérieur des blocs `for`, `while`, `do-while` ou `switch`.

`break` dans les boucles - Exemple pratique

Exemple 1 : Trouver le premier multiple de 7

```c
#include <stdio.h>

int main() {
    int i;

    for (i = 1; i <= 100; i++) {
        if (i % 7 == 0) {
            printf("Premier multiple de 7 trouvé : %d\n", i);
            break;  // sortie immédiate dès la première trouvaille
        }
    }

    printf("Boucle terminée après break.\n");
    return 0;
}
```

- Dès que `i` est un multiple de 7, le message est affiché et `break` arrête la boucle **immédiatement**.
- Le programme continue ensuite avec `printf("Boucle terminée ...")`.

`break` dans un `switch` et son flux d'exécution

Exemple 2 : Gestion des options avec `switch`

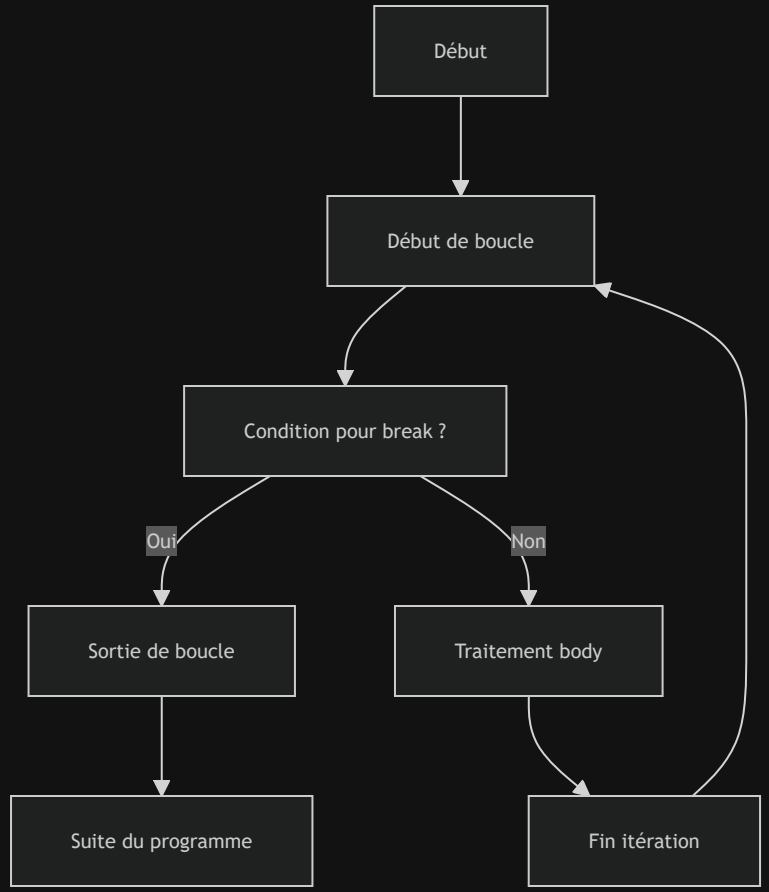
```c
#include <stdio.h>

int main() {
    char option = 'b';

    switch (option) {
        case 'a':
            printf("Option A sélectionnée\n");
            break; // Empêche l'exécution de 'b'
        case 'b':
            printf("Option B sélectionnée\n");
            break; // Empêche l'exécution du 'default'
        default:
            printf("Option inconnue\n");
    }

    return 0;
}
```

- Sans `break` dans les `case`, toutes les instructions des cas suivants seraient exécutées.


Début
Début de boucle
Condition pour break ?
Oui
Non
Sortie de boucle
Traitement body
Suite du programme
Fin itération

Points clés et bonnes pratiques avec `break`

- **Simplification :** Utiliser `break` peut rendre les boucles avec conditions d'arrêt multiples plus simples et plus lisibles.
- **Contrôle fin :** Offre une maîtrise explicite du flux d'exécution, évitant des conditions imbriquées complexes.
- **Modération :** Ne pas en abuser excessivement, car un usage excessif pourrait parfois nuire à la lisibilité du code.
- **Indispensable dans `switch` :** Dans un `switch`, le `break` est crucial pour éviter le "fall-through" et garantir que seul le cas souhaité est exécuté.

Conclusion et Ressources

- **En bref :** L'instruction `break` permet de sortir immédiatement d'une boucle ou d'un bloc `switch`, offrant une maîtrise explicite du déroulement du programme. Utilisée à bon escient, elle optimise la gestion du contrôle de flux dans le code C.

- **Sources pour aller plus loin :**

  - cppreference.com - break statement
  - GeeksforGeeks - break statement in C
  - TutorialsPoint - C break Statement
  - ISO/IEC 9899:2018 (C18) – Chapitre 6.8.6.1 (break statement)

L’instruction `continue` en C : Maîtriser le flux des boucles

L’instruction `continue` en langage C :

- Interrompt l’exécution **du reste du corps de la boucle en cours**.
- Force le programme à passer immédiatement à l’itération suivante.
- S’utilise uniquement à l’intérieur des boucles (`for` , `while` , `do-while`).
- Modifie le flux normal d’exécution **sans sortir de la boucle**.

Fonctionnement & Syntaxe

Comment `continue` agit-il ?

- Lorsqu'un `continue` est rencontré, le programme saute les instructions restantes du bloc courant.
- **Dans une boucle `for`** : Le mécanisme d'incrémentation ou de mise à jour est exécuté, avant de revenir à l'évaluation de la condition.
- **Dans une boucle `while` ou `do-while`** : La condition est évaluée directement après.

Syntaxe simple

```
continue;
```

- Doit être utilisé à l'intérieur d'une boucle.
- Ne prend pas d'arguments.

Exemples Pratiques

Exemple 1 : Ignorer les nombres pairs lors de l'affichage

```c
#include <stdio.h>

int main() {
    for (int i = 0; i < 10; i++) {
        if (i % 2 == 0) {
            continue;  // passer à l'itération suivante
        }
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

Résultat : `1 3 5 7 9` Les nombres pairs ne sont pas affichés car `continue` empêche l'exécution du `printf` dans ces cas.

Exemple 2 : Sauter une valeur spécifique dans une boucle `while`

```c
#include <stdio.h>

int main() {
    int i = 0;

    while (i < 10) {
        i++;
        if (i == 5) {
            continue;  // la valeur 5 ne sera pas affichée
        }
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

La valeur `5` est incrémentée puis le `continue` est déclenché, empêchant son affichage.

Comportement de `continue`

Points Clés & Ressources

Ce qu'il faut retenir de `continue`

- `continue` **n'interrompt pas la boucle**, seulement l'itération en cours.
- Permet d'éviter une imbrication excessive et d'augmenter la lisibilité.
- Diffère de `break` qui **quitte définitivement** la boucle.
- Utilisé avec précaution, il peut clarifier le flux de contrôle dans des boucles complexes.

Ressources utilisées

- cppreference.com - continue statement
- GeeksforGeeks - continue statement in C
- TutorialsPoint - C continue Statement
- ISO/IEC 9899:2018 (C18) – Chapitre 6.8.6.2 (continue statement)

Introduction à la synergie

Combiner les **conditions** (`if` , `switch`) et les **boucles** (`for` , `while` , `do-while`) est essentiel pour des traitements complexes et des itérations contrôlées.

Cette synergie permet de :

- Exécuter des actions uniquement dans des cas précis.
- Contrôler finement le flux d’exécution à chaque étape d’une boucle.
- Adapter le comportement répétitif d’un programme en fonction des données.

Exemple Pratique 1 : Saisie Contrôlée

Saisie d'un nombre valide (1-10) avec `do-while` et `if`

Contexte : Assurer qu'un utilisateur entre un nombre entre 1 et 10.

```c
#include <stdio.h>

int main() {
    int nombre;
    do {
        printf("Entrez un nombre entre 1 et 10 : ");
        scanf("%d", &nombre);
        if (nombre < 1 || nombre > 10) {
            printf("Nombre invalide. Réessayez.\n");
        }
    } while (nombre < 1 || nombre > 10);
    printf("Vous avez entré %d.\n", nombre);
    return 0;
}
```

- La boucle `do-while` assure la **répétition** de la saisie.
- La condition `if` à l'intérieur de la boucle **vérifie la validité** du nombre.

Exemple Pratique 2 : Filtrage et Agrégation

Somme des nombres pairs jusqu'à 100 avec `for` et `if`

Contexte : Calculer la somme de tous les nombres pairs de 1 à 100.

```c
#include <stdio.h>

int main() {
    int somme = 0;
    for (int i = 1; i ⩽ 100; i++) {
        if (i % 2 == 0) { // Si le nombre est pair
            somme += i;
        }
    }
    printf("La somme des nombres pairs de 1 à 100 est %d.\n", somme);
    return 0;
}
```

- La boucle `for` **itère** de 1 à 100.
- La condition `if` **filtre** les nombres pairs pour les ajouter à la somme.

Exemple Pratique 3 : Interaction Utilisateur

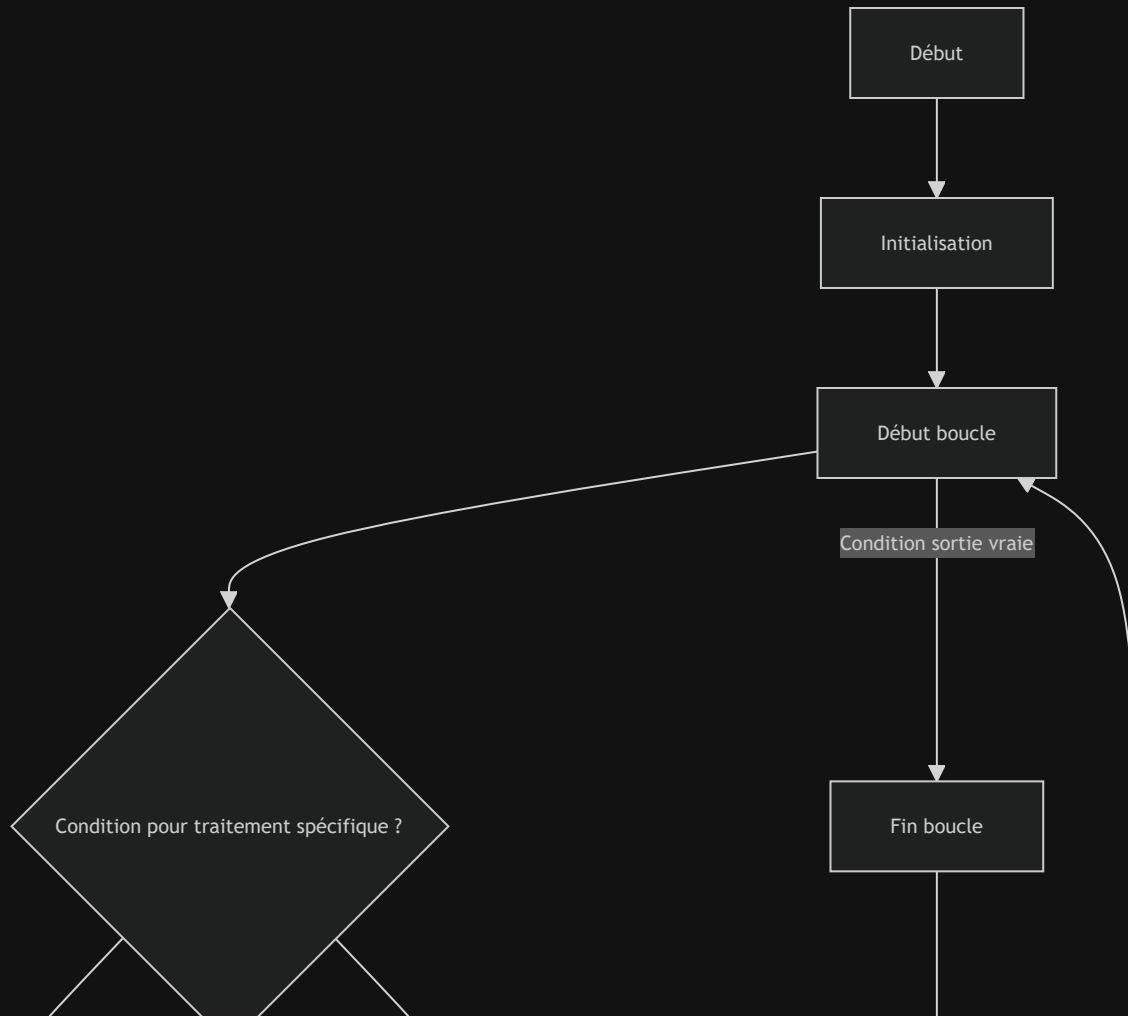
Menu interactif simple avec `do-while` et `switch`

Contexte : Proposer un menu d'options à l'utilisateur jusqu'à ce qu'il quitte.

```c
#include <stdio.h>

int main() {
    int choix;
    do {
        printf("Menu:\n1. Dire bonjour\n2. Dire au revoir\n3. Quitter\n");
        printf("Choisissez une option: ");
        scanf("%d", &choix);
        switch (choix) {
            case 1: printf("Bonjour !\n"); break;
            case 2: printf("Au revoir !\n"); break;
            case 3: printf("Fin du programme.\n"); break;
            default: printf("Option invalide.\n");
        }
    } while (choix ≠ 3);
    return 0;
}
```

- La boucle `do-while` permet des **choix successifs**.


Début
Initialisation
Début boucle
Condition sortie vraie
Condition pour traitement spécifique ?
Fin boucle

Conseils pratiques

- Vérifiez que la **condition d'arrêt de la boucle** est claire et atteignable.
- Utilisez les conditions pour **filtrer et orienter** le traitement.
- Privilégiez la **lisibilité** ; évitez les conditions trop imbriquées.
- **Commentez** les cas particuliers dans les blocs conditionnels.

Ce qu’il faut retenir & Ressources pour aller plus loin

Conclusion : Maîtriser le Flux de Contrôle

La combinaison de boucles et de conditions est fondamentale. Elle permet de gérer efficacement un grand nombre de problèmes en C, notamment pour :

- Les interactions utilisateur.
- Le traitement de données.
- La gestion des choix multiples.

C’est une compétence clé pour le contrôle du flux dans tout programme.

Sources utilisées

- cplusplus.com - Loops and Conditions
- GeeksforGeeks - Loops and Conditionals in C
- TutorialsPoint - C - Loop Control Statements
- ISO/IEC 9899:2018 (C18) – Chapitres sur les boucles et conditions

Introduction au Pseudo-code

Qu’est-ce que le pseudo-code ?

Méthode informelle et indépendante d’un langage de programmation pour décrire un algorithme de manière claire et lisible.

À quoi sert-il ?

- **Concevoir la logique :** Réfléchir au flux avant de coder.
- **Faciliter la communication :** Compréhension partagée entre programmeurs.
- **Prévenir les erreurs :** Structurer la pensée logique.

Principes Fondamentaux du Pseudo-code

Pour une description efficace d'un algorithme :

- **Structures de contrôle :** Utiliser des classiques comme `Si ... Alors`, `Tant que`, `Pour`.
- **Vocabulaire naturel :** Employer un langage proche du parlé.
- **Simplicité et précision :** Être concis mais clair, éviter l'excès de détails ou d'abstraction.
- **Décomposition :** Organiser le problème en étapes logiques et séquentielles.

Exemple Simple : Parité d’un nombre

Problème : Afficher si un nombre est pair ou impair.

```
Début
    Lire nombre
    Si nombre modulo 2 = 0 Alors
        Afficher "Le nombre est pair"
    Sinon
        Afficher "Le nombre est impair"
    Fin Si
Fin
```

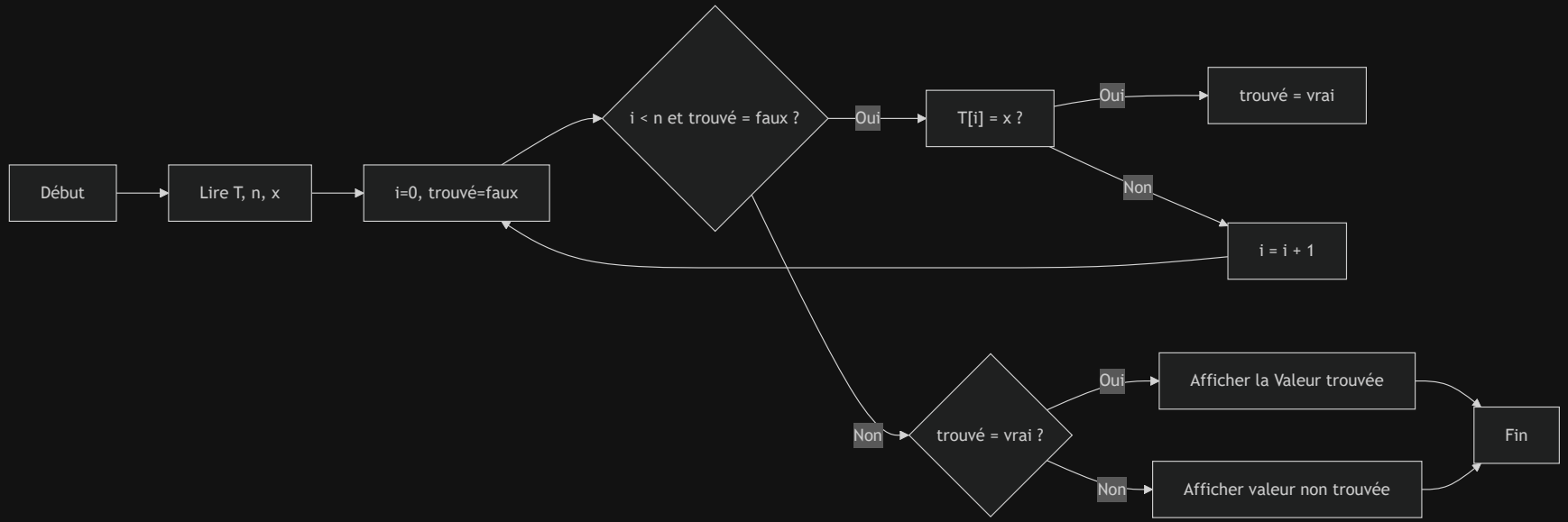
Exemple Avancé : Recherche dans un tableau

Problème : Rechercher la présence d'un élément dans un tableau.

```
Début
    Lire tableau T de taille n
    Lire valeur x
    trouvé ← faux
    i ← 0
    Tant que i < n et trouvé = faux faire
        Si T[i] = x Alors
            trouvé ← vrai
        Sinon
            i ← i + 1
        Fin Si
    Fin Tant que

    Si trouvé = vrai Alors
        Afficher "Valeur trouvée"
    Sinon
        Afficher "Valeur non trouvée"
    Fin Si
Fin
```

Représentation visuelle du flux



Rédiger un Pseudo-code Efficace

Quelques bonnes pratiques pour des algorithmes clairs :

- **Clarté avant concision :** La lisibilité prime.
- **Définir les variables :** Préciser les données et structures utilisées.
- **Identifier la logique :** Mettre en évidence les conditions et les boucles.
- **Mots-clés cohérents :** Utiliser des termes uniformes (`Si` , `Sinon` , `Pour` , `Tant que`).
- **Décomposer les problèmes :** Aborder les tâches complexes par sous-problèmes.

L’Essentiel à Retenir & Ressources

Ce qu’il faut retenir

Le pseudo-code est un outil fondamental pour :

- Structurer la logique avant de coder.
- Exprimer clairement les algorithmes.
- Faciliter la traduction vers n’importe quel langage de programmation. Il doit être simple, lisible et refléter fidèlement la logique complète du problème.

Sources utilisées

- Wikipedia - Pseudocode
- OpenClassrooms - Algorithmes : Faites vos premiers pas
- GeeksforGeeks - Pseudocode Introduction
- Tutorialspoint - Introduction to Algorithms and Pseudocode

Tableaux et `struct` en langage C

1 Les tableaux en C

1.1 Définition

Un **tableau** est une **collection d'éléments du même type**, stockés **de manière contiguë en mémoire**.

```
type nom_tableau[taille];
```

Exemple

```
int notes[5];
```

1.2 Initialisation d’un tableau

Initialisation complète

```
int notes[5] = {12, 15, 9, 18, 14};
```

Initialisation partielle

```
int notes[5] = {10, 20}; // les autres valeurs valent 0
```

Initialisation automatique

```
int notes[] = {12, 15, 9}; // taille = 3
```

1.3 Accès aux éléments

Les indices commencent à 0

```
notes[0] = 12;
printf("%d\n", notes[2]); // affiche 9
```

1.4 Parcours d’un tableau

```
for (int i = 0; i < 5; i++) {
    printf("%d ", notes[i]);
}
```

1.5 Taille d’un tableau

```
int taille = sizeof(notes) / sizeof(notes[0]);
```

Fonctionne uniquement dans le même scope

1.6 Tableaux et fonctions

ATTENTION : Il impossible de connaître la taille automatiquement

```
void afficher(int tab[]) { } // taille inconnue
```

Toujours passer la taille en argument

```
void afficher(int tab[], int taille) {
    for (int i = 0; i < taille; i++) {
        printf("%d ", tab[i]);
    }
}
```

1.7 Tableaux de caractères (chaînes)

```c
char nom[20] = "Alice";
```

Une chaîne est **terminée par** `'\0'`

Fonctions utiles

```c
strlen(nom);
strcpy(dest, src);
strcmp(a, b);
```

2 Les structures (`struct`)

2.1 Pourquoi utiliser une `struct` ?

Une `struct` permet de **regrouper plusieurs variables de types différents** sous un même nom.

Équivalent d'un **objet simple** sans méthodes.

2.2 Déclaration d'une structure

```
struct Produit {
    int id;
    char nom[50];
    float prix;
    int stock;
};
```

2.3 Déclaration de variables

```
struct Produit p1;
```

Avec `typedef` (recommandé)

```
typedef struct {
    int id;
    char nom[50];
    float prix;
    int stock;
} Produit;

Produit p1;
```

2.4 Initialisation d'une `struct`

```
Produit p1 = {101, "Souris", 25.99, 50};
```

2.5 Accès aux champs

```
p1.prix = 29.99;
printf("%s\n", p1.nom);
```

2.6 Structures et fonctions

Passage par valeur

```
void afficher(Produit p) {
    printf("%s %.2f\n", p.nom, p.prix);
}
```

Copie complète

Passage par adresse (recommandé)

```
void afficher(Produit *p) {
    printf("%s %.2f\n", p->nom, p->prix);
}
```

2.7 Tableaux de structures

Très courant en C.

```
Produit catalogue[100];
```

Initialisation

```
Produit catalogue[] = {
    {101, "PC", 1200.0, 10},
    {102, "Clavier", 80.0, 25}
};
```

Parcours

```
for (int i = 0; i < taille; i++) {
    printf("%s %.2f\n", catalogue[i].nom, catalogue[i].prix);
}
```

3. Tableaux + `struct` = base de 80% des programmes C

Exemple complet

```c
#define MAX 3

Produit produits[MAX] = {
    {1, "Livre", 12.5, 10},
    {2, "Stylo", 1.2, 100},
    {3, "Sac", 45.0, 5}
};

void afficher_catalogue(Produit tab[], int taille) {
    for (int i = 0; i < taille; i++) {
        printf("%d - %s (%.2f€)\n",
               tab[i].id,
               tab[i].nom,
               tab[i].prix);
    }
}
```

4. Erreurs classiques à éviter

Erreur	Conséquence
Dépassement de tableau	Segmentation fault
Oublier `'\0'`	Chaîne corrompue
Ne pas passer la taille	Comportement indéfini
Modifier une struct passée par valeur	Modification perdue
`scanf("%s", nom)` sans limite	Buffer overflow

5. Bonnes pratiques

- ✔ Toujours définir des **constantes** (`#define`)
- ✔ Toujours passer la **taille des tableaux**
- ✔ Préférer le **passage par pointeur**
- ✔ Commenter les structures
- ✔ Tester les cas limites
- ✔ Initialiser les variables

Fonctions et Modularité : Les Fondamentaux

Pourquoi utiliser des fonctions ?

- Réutilisabilité
- Modularité
- Lisibilité

Qu’est-ce qu’une fonction en C ? Un bloc de code nommé, conçu pour accomplir une tâche précise. Elle peut être appelée depuis d’autres parties du programme pour exécuter cette tâche sans reproduction de code.

Avantages des Fonctions

1. La Réutilisabilité

- Permet de **réutiliser** un même morceau de code dans différentes parties d'un programme ou même dans différents projets.
- Évite la duplication et facilite la maintenance.

Exemple : Calcul du carré

```c
#include <stdio.h>

int carre(int x) {
    return x * x;
}

int main() {
    int a = 5;
    printf("Carré de %d = %d\n", a, carre(a));
    printf("Carré de %d = %d\n", 10, carre(10));
    return 0;
}
```

Avantages des Fonctions

2. La Modularité

- **Découpe** un programme complexe en modules ou unités fonctionnelles.
- Chaque unité a une responsabilité claire.
- Facilite la conception et le test unitaire.
- Favorise la collaboration : chaque développeur peut travailler sur une fonction distincte.
- Encourage la répartition logique et la gestion des tâches.

3. La Lisibilité

- Réduit la taille des blocs de code dans la fonction `main` ou autres fonctions principales.
- Clarifie le but du programme à travers le nom des fonctions.
- Facilite la lecture et la compréhension du flux d’exécution.

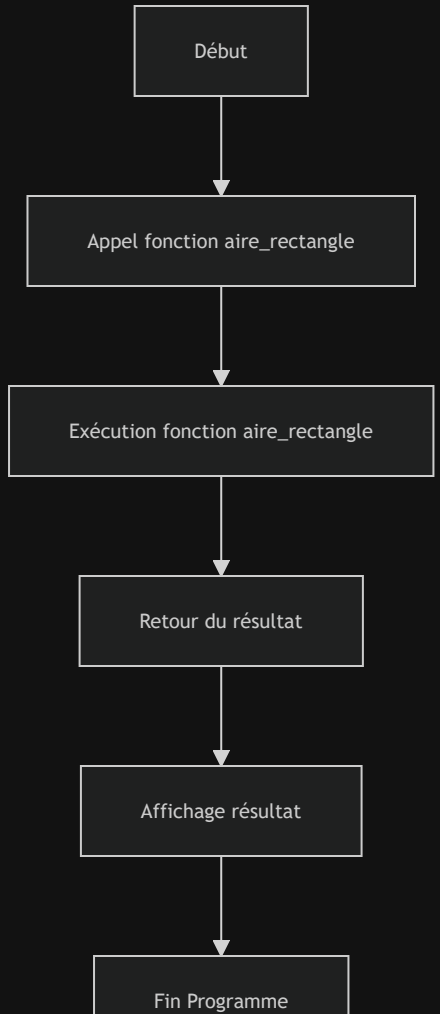
Fonctions en Action : Découpage et Appel

Illustration : Calcul de l'aire d'un rectangle

```c
#include <stdio.h>

// Fonction qui calcule l'aire
int aire_rectangle(int largeur, int hauteur) {
    return largeur * hauteur;
}

int main() {
    int l = 4, h = 5;
    int aire = aire_rectangle(l, h);
    printf("Aire du rectangle : %d\n", aire);
    return 0;
}
```


Début
Appel fonction aire_rectangle
Exécution fonction aire_rectangle
Retour du résultat
Affichage résultat
Fin Programme

Fonctions en C : Les Points Essentiels

Avantage	Description
Réutilisabilité	Évite la redondance de code en appelant plusieurs fois une fonction
Modularité	Découpage clair en unités fonctionnelles
Lisibilité	Améliore la compréhension et la structure du programme

Pour un code performant et maintenable

Ce qu'il faut retenir : Les fonctions sont la clé pour écrire un code propre, clair, évolutif et facile à maintenir. Elles offrent la structure et la flexibilité indispensables au développement logiciel efficace.

Sources Utilisées :

- GeeksforGeeks - Why Use Functions?
- cppreference.com - Functions
- TutorialsPoint - C Functions
- ISO/IEC 9899:2018 (C18) – Chapitre 6.7 Functions

Fonctions en C : Comprendre leur Anatomie

Les 4 piliers d’une brique logicielle essentielle

1. Introduction aux Fonctions en C

Une fonction en langage C est un bloc de code conçu pour accomplir une tâche spécifique. Elle est fondamentale pour la modularité et la réutilisabilité du code.

Chaque fonction est structurée autour de quatre éléments fondamentaux :

1. Type de retour
2. Nom de la fonction
3. Liste des paramètres (optionnelle)
4. Corps de la fonction

2. Type de Retour & Nom : L'Identité de la Fonction

- **Type de retour**
  - Indique le type de la valeur que la fonction renvoie à son appelant.
  - Peut être un type primitif (`int` , `float`), un pointeur, une structure, ou `void` si la fonction ne retourne rien.
  - Le type doit correspondre à la valeur retournée par l'instruction `return` .

```c
int maFonction() {
    return 42;   // retourne un entier
}
```

- **Nom de la fonction**
  - Identificateur unique utilisé pour appeler la fonction.
  - Doit respecter les conventions de nommage en C (lettres, chiffres, underscore, ne pas commencer par un chiffre).
  - Le nom doit être explicite pour refléter la tâche de la fonction.

3. Paramètres & Corps : Entrées et Logique

- **Liste des paramètres**
  - Enveloppe les variables d'entrée que la fonction utilise pour effectuer son traitement.
  - Chaque paramètre est déclaré avec son type et son nom.
  - Une fonction peut ne pas prendre de paramètres (`void` ou liste vide).
  - Les paramètres sont passés par valeur en C (sauf si ce sont des pointeurs).

```c
void afficherMessage(char message[]) {
    printf("%s\n", message);
}
```

- **Corps de la fonction**
  - Bloc d'instructions entre accolades `{ ... }`.
  - Contient les déclarations locales et le code à exécuter.
  - Peut contenir des structures de contrôle, appels à d'autres fonctions, etc.
  - Doit se terminer par un `return` si le type de retour n'est pas `void`.

4. Exemple Complet : Calculer une Somme

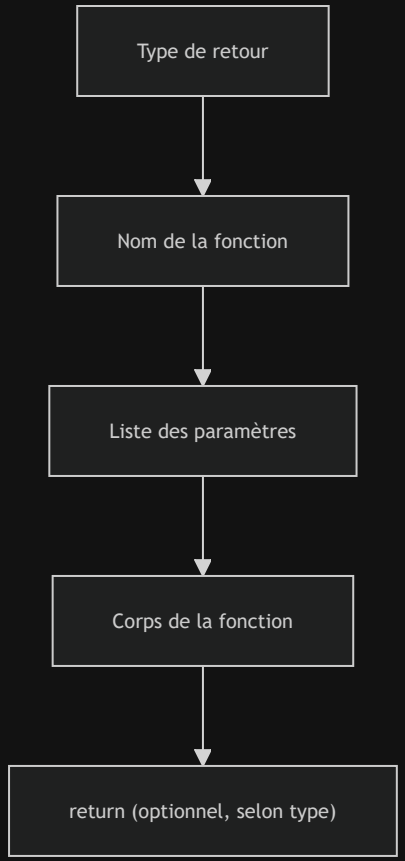
Cet exemple illustre l'intégration de tous les éléments d'une fonction pour réaliser une tâche simple.

```c
#include <stdio.h>

// Fonction qui calcule la somme de deux entiers et renvoie le résultat
int addition(int a, int b) { // Type de retour: int, Nom: addition, Paramètres: int a, int b
    int resultat = a + b;   // Corps de la fonction
    return resultat;        // Instruction de retour
}

int main() {
    int x = 5, y = 3;
    int somme = addition(x, y); // Appel de la fonction 'addition'
    printf("La somme de %d et %d est %d\n", x, y, somme);
    return 0;
}
```

5. Synthèse Visuelle : Les Composants Clés



Points clés en résumé

Élément	Description	Exemple
Type de retour	Type de la valeur retournée, ou `void` si rien rendu	`int`, `void`
Nom de la fonction	Identifiant pour l'appel	`addition`, `afficher`
Paramètres	Variables d'entrée (possiblement vide)	`(int a, int b)`, `(void)`
Corps	Suite d'instructions qui réalise la fonction	`{ return a + b; }`

6. En Résumé & Pour Aller Plus Loin

- **Ce qu'il faut retenir :** Cette décomposition détaillée permet de maîtriser la déclaration et la définition des fonctions en C, en structurant clairement leurs entrées, sorties et logique interne. Elles sont la base de la modularité en programmation C.

- **Sources utilisées :**

  - cppreference.com - Functions in C
  - GeeksforGeeks - Anatomy of a Function in C
  - TutorialsPoint - C Functions
  - ISO/IEC 9899:2018 (C18) – Section 6.7

Fonctions et Modularité : Déclaration et Définition

Déclaration et Définition : Les Fondamentaux

En C, la **déclaration** et la **définition** d'une fonction sont distinctes :

- **Déclaration (Prototype)**
  - Informe le compilateur de l'existence de la fonction.
  - Précise son nom, son type de retour et ses paramètres.
  - **Ne contient pas** le corps de la fonction.
- **Définition**
  - Fournit l'implémentation complète.
  - Contient le corps de la fonction avec toutes les instructions.
  - Décrit ce que la fonction réalise concrètement.

La Déclaration de Fonction (Prototype)

- **Rôle :** Permet au compilateur de vérifier la validité des appels à la fonction avant même de connaître son implémentation.
- **Emplacement :**
  - Généralement en début de fichier source.
  - Dans un fichier d'en-tête (`.h`).
  - Avant la fonction `main()` .
- **Syntaxe :**

```
type_de_retour nom_de_fonction(type_param1, type_param2, ...);
```

- **Exemple :**

```
int addition(int a, int b);
```

La Définition de Fonction

- **Rôle :** Fournit le code exécutable qui détermine le comportement de la fonction.
- **Contenu :** Le corps de la fonction, délimité par des accolades, contenant toutes les instructions.
- **Syntaxe :**

```
type_de_retour nom_de_fonction(type_param1 param1, type_param2 param2, ...) {
    // corps de la fonction
}
```

- **Exemple :**

```
int addition(int a, int b) {
    return a + b;
}
```

Exemple Complet et Cas Particulier

1. Déclaration et Définition Séparées :

```c
#include <stdio.h>

// Déclaration du prototype
int multiplication(int x, int y);

int main() {
    int resultat = multiplication(4, 5); // Appel à la fonction
    printf("Résultat : %d\n", resultat);
    return 0;
}

// Définition de la fonction
int multiplication(int x, int y) {
    return x * y;
}
```

- La déclaration permet à `main()` d'appeler `multiplication()` même si sa définition est placée après.
- **Important :** Sans la déclaration, si la définition est après `main()`, le compilateur signale une erreur.

2. Définition sans Déclaration Préalable :

```c
#include <stdio.h>

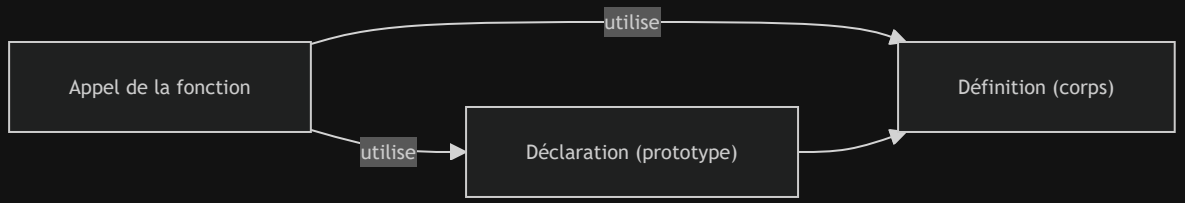
int somme(int a, int b) { // Définition placée avant l'utilisation
    return a + b;
}

int main() {
    printf("%d\n", somme(3, 4));
    return 0;
}
```

- Valide uniquement si la définition est placée *avant* la première utilisation de la fonction.

Liens et Conseils Pratiques

Relations entre Déclaration, Définition et Appel :



Conseils :

- **Toujours déclarer** les fonctions avant leur premier appel pour une meilleure portabilité et lisibilité.
- Utiliser des **fichiers d'en-tête (** `.h` **)** pour regrouper les prototypes et garantir la cohérence dans les projets multi-fichiers.
- Les prototypes facilitent la **détection précoce d'erreurs** (typographiques, mauvais paramètres) lors de la compilation.

Ce qu’il faut Retenir & Sources

En Bref :

Le prototype (déclaration) et la définition sont essentiels pour la gestion des fonctions en C. Ils garantissent que le compilateur dispose des informations nécessaires pour :

1. Vérifier les appels de fonction.
2. Générer un code correct.
3. Éviter les erreurs d’appel durant la compilation.

Sources :

- cppreference.com - Function Declaration
- GeeksforGeeks - Function declaration and prototype in C
- TutorialsPoint - C Function Declaration & Definition
- ISO/IEC 9899:2018 (C18) – Section 6.7

Passage d’Arguments par Valeur en C

Comprendre la copie des données

En langage C, les arguments passés à une fonction le sont par **valeur** par défaut. Ce principe fondamental implique que :

- Une **copie** de la valeur de l’argument est transmise à la fonction.
- La fonction travaille sur cette copie, sans modifier la variable d’origine dans la fonction appelante.
- Toute modification effectuée sur le paramètre dans la fonction n’a **pas d’effet** sur la variable passée depuis l’appelant.

Illustration par l'Exemple

```c
#include <stdio.h>

// Fonction qui tente de modifier la valeur d'un entier
void incrementer(int x) {
    x = x + 1;
    printf("Dans la fonction : x = %d\n", x);
}

int main() {
    int a = 5;
    incrementer(a); // Appel avec la valeur de 'a'
    printf("Après appel de la fonction : a = %d\n", a);
    return 0;
}
```

Résultat :

```
Dans la fonction : x = 6
Après appel de la fonction : a = 5
```

`a` dans `main` reste inchangé car `x` est une copie locale, modifiée indépendamment.

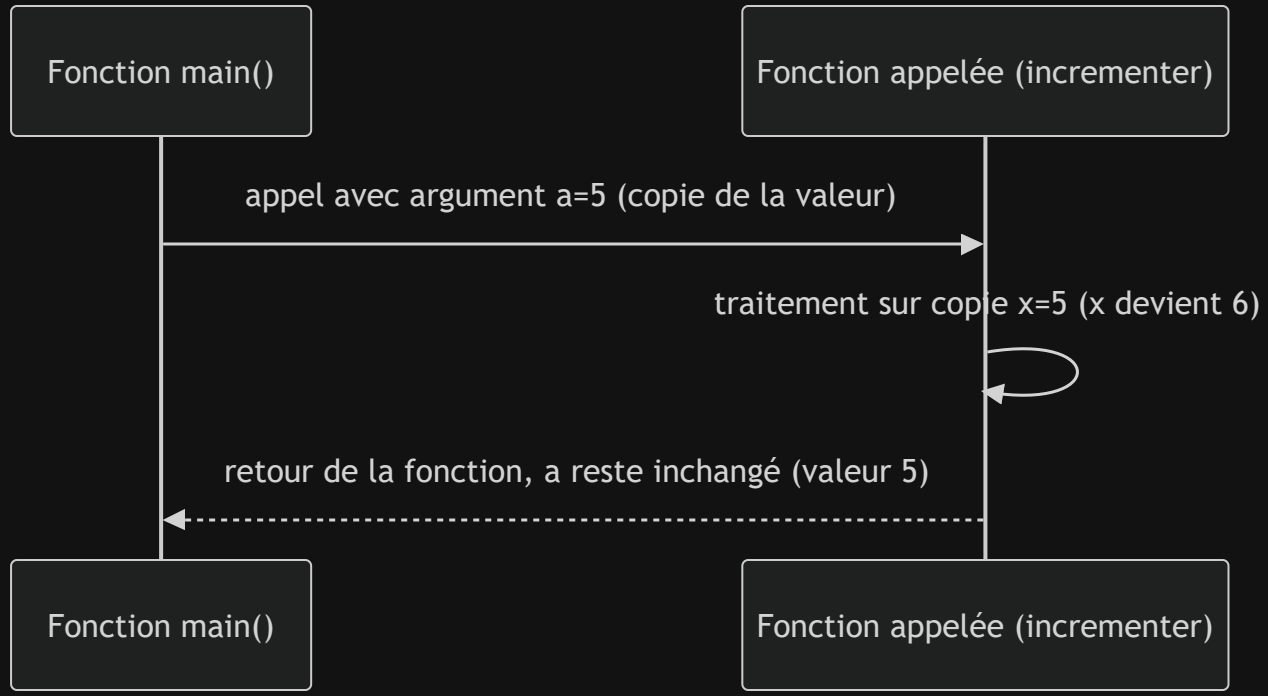
Implications du Passage par Valeur

Le choix du passage par valeur n’est pas anodin et comporte des conséquences importantes :

Avantage	Inconvénient
Sécurité : pas de modification accidentelle des variables externes	Pas possible de modifier directement les données de la fonction appelante
Simplicité de compréhension du flux de données	Pour modifier les données de l’appelant, nécessité de passer par des mécanismes plus avancés (ex: pointeurs)

Visualisation du Mécanisme

Le diagramme ci-dessous illustre le flux de données lors d'un appel par valeur :



Résumé Technique et Cas d’Usage

Quelques points clés à retenir techniquement :

- Le passage par valeur correspond à un **copiage bit à bit** des données de l’argument.
- Applicable à tous les types simples (int, float, char) ainsi qu’aux structures.
- Pour les structures volumineuses, le passage par valeur peut être **coûteux** en termes de mémoire et de cycles CPU en raison du copiage intégral.
- En cas de données complexes ou si le besoin de modifier les variables d’appel est avéré, il est nécessaire d’utiliser le **passage par adresse** (via des pointeurs).

Ce qu’il faut retenir & Références

Le passage par valeur est la méthode standard en C pour transmettre des arguments aux fonctions. Comprendre ce mécanisme clarifie le comportement du programme face aux modifications des données et permet de choisir la méthode la plus adaptée selon le contexte.

Sources utilisées :

- GeeksforGeeks - Call by Value in C
- TutorialsPoint - C Pass By Value
- cppreference.com - Function arguments
- ISO/IEC 9899:2018 (C18) – Passage des arguments

L’instruction `return` en C : Passage d’Arguments et Valeurs de Retour

L’instruction `return` dans une fonction en langage C remplit deux fonctions essentielles :

- **Terminer l’exécution** de la fonction immédiatement.
- **Retourner une valeur** (optionnelle) au point d’appel de la fonction.

Syntaxe Générale et Utilisation

Syntaxe

```
return expression;
```

- `expression` est une valeur compatible avec le type de retour de la fonction.

Cas particulier des fonctions `void`

- Dans une fonction ayant un type de retour `void`, l'instruction `return;` peut être utilisée pour sortir prématurément, sans valeur.

`return` avec une Valeur : La Fonction `carre`

```c
#include <stdio.h>

int carre(int n) {
    return n * n; // Retourne le résultat du calcul
}

int main() {
    int x = 4;
    int result = carre(x); // La valeur retournée est assignée à 'result'
    printf("Carré de %d = %d\n", x, result);
    return 0;
}
```

- La fonction `carre` retourne le résultat qui est assigné à `result` dans `main`.
- L'exécution quitte la fonction dès le `return`.

`return` dans une Fonction `void` : Sortie Anticipée

```c
#include <stdio.h>

void afficherMessage(int n) {
    if (n ⩽ 0) {
        printf("Valeur non positive\n");
        return;  // Sortie anticipée de la fonction sans valeur
    }
    printf("Valeur positive : %d\n", n);
}

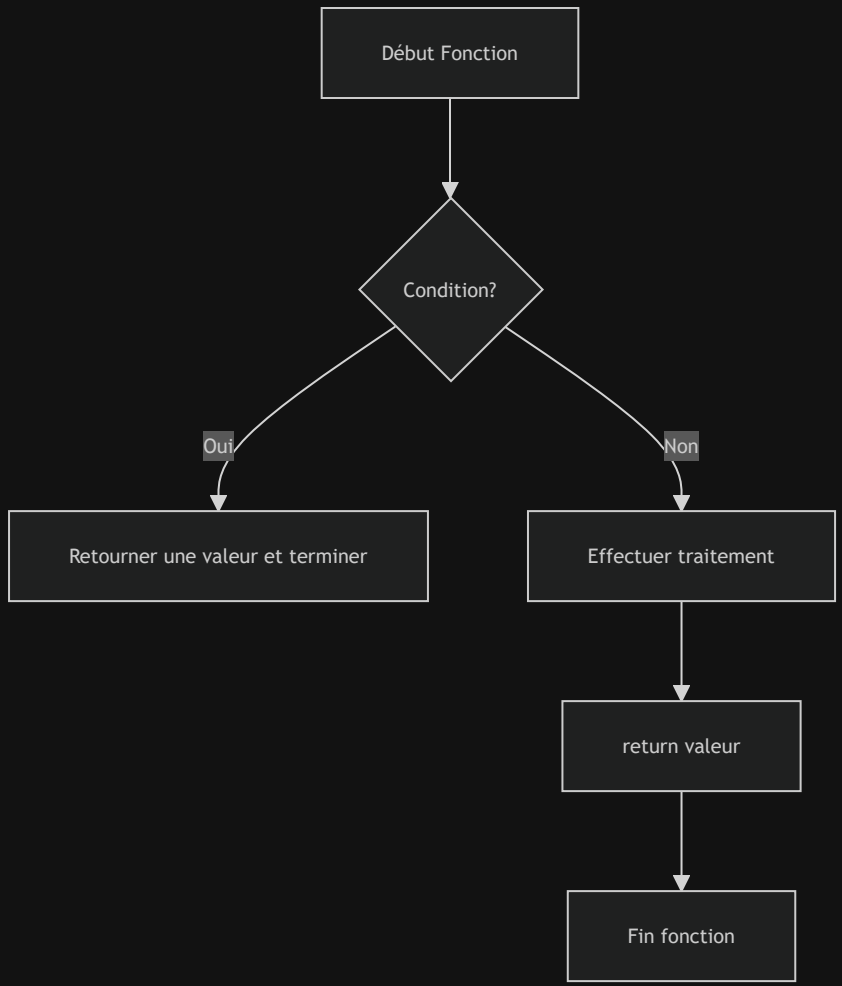
int main() {
    afficherMessage(5);
    afficherMessage(-1);
    return 0;
}
```

- Le `return;` arrête l'exécution de la fonction `afficherMessage` si la condition est remplie.
- Pas de valeur retournée car la fonction est `void`.

Gestion du Flux et Importance du `return`

L'instruction `return` est cruciale pour :

- Permettre de **sortir immédiatement** d'une fonction, utile dans les tests conditionnels.
- Assurer que la **valeur retournée** suit un chemin clair vers le programme appelant.
- Être **obligatoire** dans toutes les fonctions (sauf celles de type `void` si elles terminent naturellement).


Début Fonction
Condition?
Oui
Non
Retourner une valeur et terminer
Effectuer traitement
return valeur
Fin fonction

Points Clés et Pour Aller Plus Loin

Synthèse

Aspect	Description
Terminaison	`return` termine la fonction immédiatement
Valeur retournée	Permet de renvoyer une donnée au programme appelant
Utilité dans `void`	Utilisé pour sortie anticipée, sans valeur
Obligatoire	Doit correspondre au type de retour de la fonction

L'instruction `return` est l'instrument fondamental pour passer une valeur calculée et contrôler la fin d'une fonction. Son usage approprié rend les programmes plus clairs, efficaces et maitrisables.

Sources utilisées

- cppreference.com - return statement
- TutorialsPoint - Return Statement in C
- GeeksforGeeks - return statement in C
- ISO/IEC 9899:2018 (C18) – Section 6.8.6.4

Introduction au `void` en C

- Le mot-clé `void` en C signifie **absence de type** ou **absence de valeur**.
- Utilisé pour indiquer qu'une fonction :
  - **ne retourne aucune valeur**.
  - **ne prend pas de paramètres**.

Fonctions sans paramètres

- Syntaxe :

```
void fonctionSansParam(void) {
    // code de la fonction
}
```

  - Le `(void)` indique explicitement que la fonction ne prend **aucun argument**.
  - Ceci est la forme recommandée par rapport à une liste vide `()`.

- Exemple :

```
#include <stdio.h>

void afficherBonjour(void) {
    printf("Bonjour!\n");
}

int main() {
    afficherBonjour(); // Appel sans argument
    return 0;
}
```

Fonctions sans valeur de retour (`void`)

- Syntaxe :

```
void fonctionSansRetour(int param) {
    // traitement sans retour
}
```

  - La fonction effectue des opérations mais **ne retourne rien** à son appelant.

- **Usage typique :** Actions d'affichage, modification via pointeurs, manipulation d'I/O.

- Exemple :

```
#include <stdio.h>

void afficherValeur(int n) {
    printf("La valeur est %d\n", n);
}

int main() {
    afficherValeur(10); // Appel avec argument, mais sans valeur de retour
    return 0;
}
```

Fonctions sans paramètres et sans valeur de retour

- Combinaison des deux cas : la fonction ne reçoit aucune entrée et ne retourne aucune sortie.
- Exemple :

```c
#include <stdio.h>

void afficherMessage(void) {
    printf("Message fixe\n");
}

int main() {
    afficherMessage(); // Appel simple, sans entrée ni sortie
    return 0;
}
```

Synthèse et Ressources

- **Récapitulatif des cas d'usage :**

Fonction	Syntaxe	Usage typique
Avec paramètres et retour	`int f(int x)`	Calculs, transformations
Sans paramètres, avec retour	`int f(void)`	Renvoi constantes ou infos
Avec paramètres, sans retour	`void f(int x)`	Actions sans résultat attendu
Sans paramètres, sans retour	`void f(void)`	Tâches fixes, affichages, alertes

- **Ce qu'il faut retenir :** Utiliser les fonctions sans paramètres et/ou sans valeur de retour contribue à structurer clairement les responsabilités dans un programme : exécuter une action sans nécessiter d'entrées ou sans renvoyer de résultat. Cela simplifie l'interface fonctionnelle selon le besoin.

- **Sources :**

  - cppreference.com - `void` in functions
  - GeeksforGeeks - Void Functions in C
  - TutorialsPoint - C Void Functions
  - ISO/IEC 9899:2018 (C18) – Chapitre 6.7

Bibliothèques Standards C : Un Aperçu

Séance 4 : Fonctions et Modularité

Cette session explore les fonctions des bibliothèques standards en C, des outils essentiels pour des applications robustes et efficaces.

Nous allons nous concentrer sur :

- `stdio.h` : Pour les opérations d'entrée/sortie.
- `stdlib.h` : Pour les fonctions utilitaires générales.
- `math.h` : Pour les calculs mathématiques avancés.

`stdio.h` : Gérer les Entrées/Sorties Essentielles

Cette bibliothèque est la pierre angulaire des interactions console et de la gestion de fichiers.

Fonctions clés pour l'Entrée/Sortie console :

- `printf()`, `scanf()` : affichage et lecture formatée de données.
- `putchar()`, `getchar()` : gestion des caractères uniques.

Pour la gestion des fichiers :

- `fopen()`, `fclose()`, et autres pour manipuler des fichiers.

`stdio.h` est souvent la première bibliothèque que l'on rencontre en programmation C.

`stdlib.h` : Fonctions Utilitaires Générales

`stdlib.h` offre des outils polyvalents pour la gestion de la mémoire, le contrôle du programme et la génération de nombres aléatoires.

Fonctions courantes :

Fonction	Description
`exit(int status)`	Termine immédiatement le programme avec un code de sortie.
`rand()`	Génère un entier pseudo-aléatoire.
`srand(unsigned int seed)`	Initialise le générateur de nombres aléatoires.
`malloc()`, `free()`	Allocation et libération dynamique mémoire (hors sujet immédiat).

Ces fonctions permettent un contrôle plus fin et des comportements dynamiques dans vos programmes.

`stdlib.h` en Action : Aléatoire et Contrôle Programme

Exemple d'utilisation de `exit()` et `rand()` pour une gestion conditionnelle du programme :

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {
    srand(time(NULL)); // Initialisation du générateur

    int alea = rand() % 100; // Génère nombre entre 0 et 99
    printf("Nombre aléatoire généré : %d\n", alea);

    if (alea < 10) {
        printf("Valeur trop petite, sortie du programme.\n");
        exit(EXIT_FAILURE);  // Fin immédiate avec code erreur
    }

    printf("Programme continue...\n");
    return 0;
}
```

- `srand(time(NULL))` initialise le générateur avec l'horloge système.
- `rand()` génère un nombre pseudo-aléatoire.

`math.h` : Maîtriser les Calculs Mathématiques

La bibliothèque `math.h` est essentielle pour intégrer des opérations mathématiques complexes dans vos applications.

Fonctions mathématiques présentées :

Fonction	Description	Prototype
`sqrt(double x)`	Racine carrée	`double sqrt(double x)`
`pow(double x, double y)`	Puissance (x^y)	`double pow(double x, double y)`
`sin(double x)`	Sinus (angle x en radians)	`double sin(double x)`

Exemple d’utilisation :

```c
#include <stdio.h>
#include <math.h>

int main() {
    double x = 9.0;
    printf("sqrt(%.1f) = %.2f\n", x, sqrt(x));

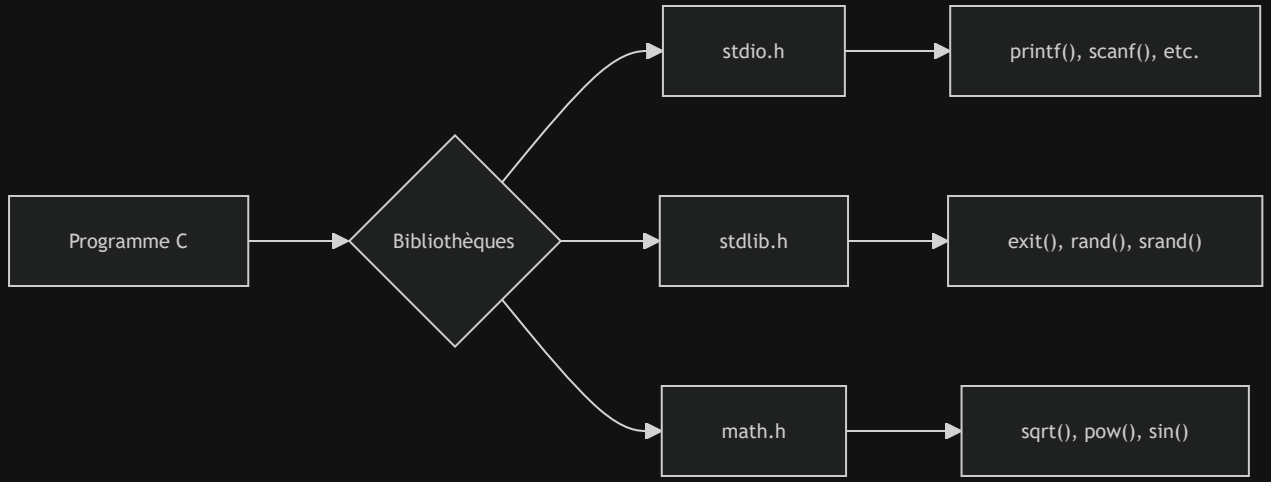
    double base = 2.0, exposant = 3.0;
    printf("%.1f^%.1f = %.2f\n", base, exposant, pow(base, exposant));

    double angle = 3.14159 / 2;  // ~ π/2 radians
    printf("sin(π/2) = %.2f\n", sin(angle));

    return 0;
}
```

Synthèse, Bonnes Pratiques et Ressources

Relations entre bibliothèques et fonctions :



Notes importantes :

- Pour `math.h` sous Linux, la compilation peut nécessiter l'option `-lm` (ex: `gcc fichier.c -o prog -lm`).
- Toujours initialiser `srand()` avant `rand()` pour des séquences aléatoires variées.
- `exit()` arrête le programme immédiatement, sans passer par la fin normale du `main()`.

Ce panorama montre comment les bibliothèques standard enrichissent la programmation C, offrant des solutions fiables pour l'affichage, le contrôle du programme et le calcul numérique.

Sources utilisées :

- cppreference.com - stdio.h
- cppreference.com - stdlib.h
- cppreference.com - math.h
- TutorialsPoint - C Standard Library
- ISO/IEC 9899:2018 (C18) – Section 7.22

L’instruction `#include` : Fondation de la Modularité

- Directive de préprocesseur qui sert à **inclure le contenu d’un fichier source ou d’en-tête** avant la compilation.
- Permet d’utiliser les **déclarations et définitions** présentes dans ces fichiers (prototypes de fonctions, macros, constantes, types).
- Facilite la modularité du code en réutilisant des bibliothèques standards ou personnalisées.

Deux Manières d'Inclure : `< >` vs. `" "`

Inclusion avec chevrons `< >`

```
#include <nom_de_fichier.h>
```

- Recherche le fichier dans les **répertoires systèmes et bibliothèques standards**.
- Utilisée pour inclure des bibliothèques **standards**.
- **Exemples :** `#include <stdio.h>`, `#include <stdlib.h>`, `#include <math.h>`

Inclusion avec guillemets `" "`

```
#include "mon_fichier.h"
```

- Recherche d'abord dans le **répertoire courant**, puis dans les répertoires système.
- Utilisée surtout pour les fichiers **personnalisés** (headers locaux).

Le Rôle Clé du Préprocesseur

- Le préprocesseur **copie littéralement le contenu** du fichier inclus à la place de l'instruction `#include`.
- Cette action se déroule **avant la compilation** proprement dite.
- Elle permet au compilateur d'avoir accès à toutes les déclarations indispensables pour la compilation correcte du programme.
- C'est une étape fondamentale pour assembler les différentes pièces de votre code source.

Modularité en Action : Un Exemple Concret

- `math_utils.h` (Déclaration de l'interface)

```c
#ifndef MATH_UTILS_H
#define MATH_UTILS_H
double carre(double x); // Prototype de fonction
#endif
```

- `math_utils.c` (Implémentation de la fonction)

```c
#include "math_utils.h" // Inclut la déclaration
double carre(double x) {
    return x * x;
}
```

- `main.c` (Utilisation de la fonction)

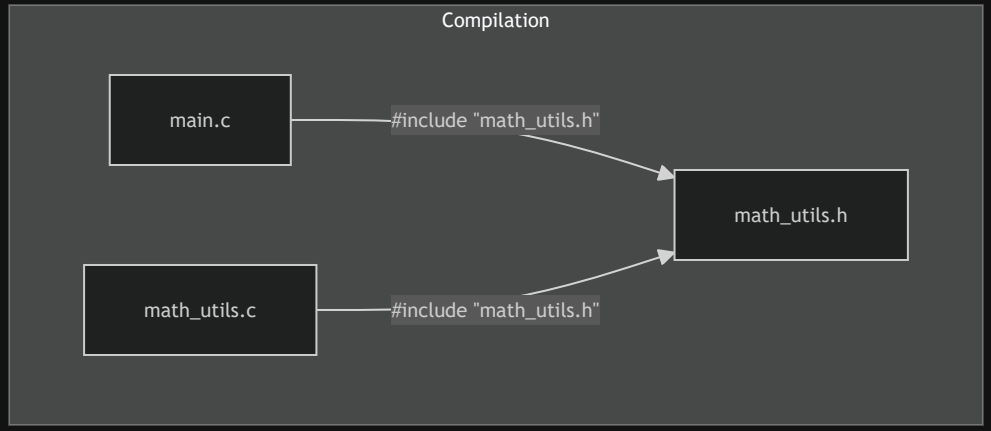
```c
#include <stdio.h>
#include "math_utils.h" // Accès à la déclaration de 'carre'

int main() {
    double val = 5.0;
    printf("Carré de %.2f = %.2f\n", val, carre(val));
    return 0;
}
```

- `main.c` peut appeler `carre` grâce à l'inclusion de `math_utils.h`, illustrant une séparation claire entre interface (`.h`) et implémentation (`.c`).

Flux d’Inclusion et Bonnes Pratiques

Diagramme de Compilation



Bonnes Pratiques d’Inclusion

- Utiliser les **gardes d’inclusion** (`#ifndef` , `#define` , `#endif`) pour éviter les doubles inclusions qui peuvent causer des erreurs.
- Préférer les fichiers `.h` pour déclarer les prototypes et les constantes, et les fichiers `.c` pour les définitions (implémentations).

`#include` : L'Essentiel & Ressources

Ce qu'il faut retenir

- L'utilisation de `#include` est la **pierre angulaire de la modularité en C**.
- Elle permet de composer un programme à partir de composants séparés.
- Elle donne accès aux définitions nécessaires, favorisant une **organisation rationnelle du code**.
- Comprendre son fonctionnement est crucial pour une programmation C efficace.

Sources pour aller plus loin

- cppreference.com - #include Directive
- TutorialsPoint - C Header Files
- GeeksforGeeks - #include in C
- ISO/IEC 9899:2018 (C18) – Chapitre 6.10

Fonctions et Modularité

Conception de Fonctions Simples

Les Fonctions en C : Concepts Clés

- Une fonction en C **encapsule des instructions** pour réaliser une tâche précise.
- Elle favorise la **réutilisation**, la **lisibilité** et la **modularité** du code.
- Les fonctions peuvent recevoir des **paramètres** et retourner une **valeur** selon leur rôle.

Exemple 1 : Calculer le Carré d'un Nombre

Objectif

Créer une fonction qui reçoit un nombre et retourne son carré.

Code simplifié

```c
// Fonction calculant le carré d'un entier
int carre(int n) {
    return n * n;
}

int main() {
    int x = 7;
    printf("Le carré de %d est %d\n", x, carre(x)); // Appel de la fonction
    return 0;
}
```

Explications

- `int carre(int n)` : prend un entier `n` en paramètre.
- Elle retourne le produit de ce nombre par lui-même (`n * n`).
- Dans `main` , l'appel `carre(x)` fournit le résultat affiché.

Exemple 2 : Afficher un Message Personnalisé

Objectif

Créer une fonction qui affiche un message comprenant un nom passé en paramètre.

Code simplifié

```c
// Fonction affichant un message personnalisé
void afficherMessage(char nom[]) {
    printf("Bonjour, %s !\n", nom);
}

int main() {
    char nom[] = "Alice";
    afficherMessage(nom); // Appel de la fonction
    return 0;
}
```

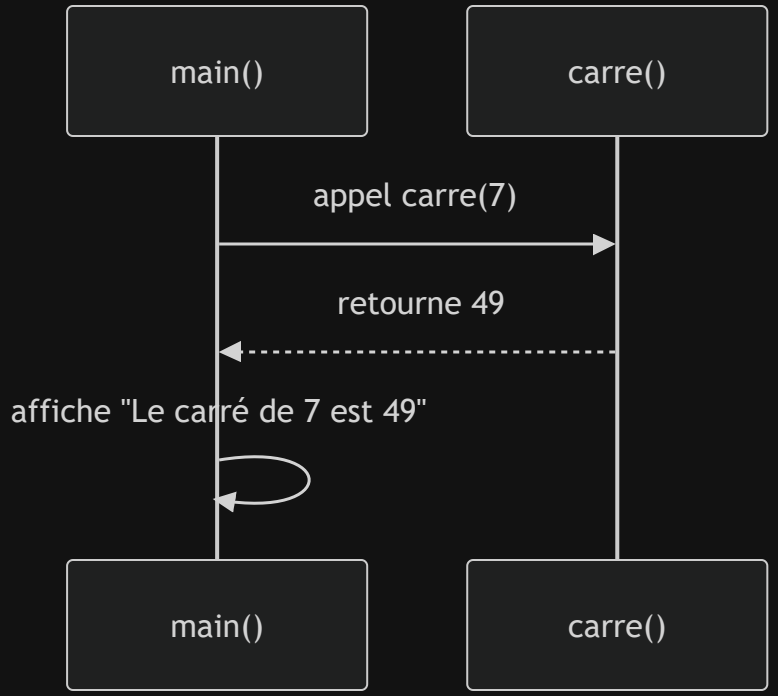
Explications

- `void afficherMessage(char nom[])` : reçoit une chaîne de caractères (tableau de `char`).
- Elle utilise `printf` pour afficher un message personnalisé.
- Fonction de type `void` : **aucune valeur n'est retournée**, l'action est l'affichage.

Bonnes Pratiques pour des Fonctions Efficaces

- **Noms explicites** : Donnez des noms clairs aux fonctions pour comprendre leur rôle.
- **Tâche unique** : Privilégiez une tâche simple et bien définie par fonction.
- **Typage correct** : Utilisez les types appropriés pour les paramètres et les valeurs de retour.
- **Documentation** : Commentez brièvement chaque fonction pour expliquer son objectif.

Visualiser l’Appel de Fonction : Flux d’Exécution



Synthèse & Pour Aller Plus Loin

Ce qu'il faut retenir

- **Modulariser** l'écriture en fonctions simples permet de mieux organiser le code.
- Cela facilite la **maintenance** et clarifie l'**intention** lors du développement.
- Les fonctions sont un concept fondamental en C, offrant **simplicité et utilité directe**.

Ressources Supplémentaires

- GeeksforGeeks - Functions in C
- TutorialsPoint - C Functions
- ISO/IEC 9899:2018 (C18) – Chapitre 6.7 Fonctions

Décomposer pour mieux concevoir : Fonctions et Modularité

- Un programme complexe peut être difficile à comprendre, déboguer et maintenir.
- La **décomposition** en sous-problèmes simplifie le raisonnement et la tâche globale.
- Désarticuler un problème en fonctions spécifiques permet de tester, réutiliser et faire évoluer chaque partie plus facilement.

Principe de décomposition fonctionnelle :

1. Identifier les étapes indépendantes ou cohérentes d'un problème.
2. Concevoir pour chaque étape une fonction avec un rôle bien défini.
3. Interconnecter ces fonctions pour réaliser la solution globale.

Les Avantages Clairs de la Décomposition Fonctionnelle

La décomposition d'un programme en fonctions apporte des bénéfices majeurs :

- **Simplification :** Réduit la complexité en traitant des portions plus petites et gérables.
- **Clarté :** Améliore la lisibilité et la compréhension de la logique du programme.
- **Maintenance facilitée :** Les corrections et évolutions sont ciblées sur des fonctions spécifiques.
- **Réutilisabilité :** Les fonctions bien conçues peuvent être réutilisées dans d'autres parties du code ou d'autres projets.
- **Testabilité accrue :** Chaque fonction peut être testée isolément, simplifiant la détection d'erreurs.
- **Évolutivité :** Permet d'ajouter de nouvelles fonctionnalités ou de modifier des existantes sans impacter l'ensemble du système.

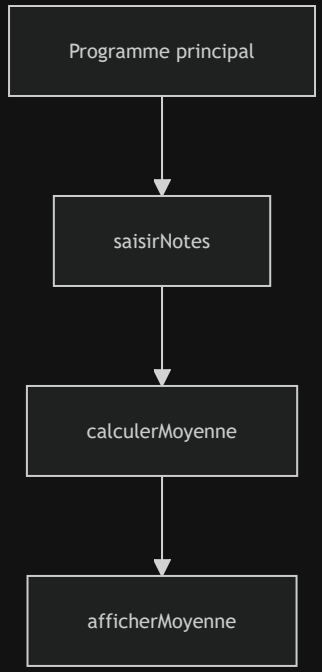
Cas Pratique : Le Calcul de Moyenne Étape par Étape

Prenons l'exemple d'un programme simple qui doit calculer et afficher la moyenne de plusieurs notes.

Le problème peut être naturellement décomposé en trois étapes distinctes :

1. **Lecture des notes :**
   - Saisir un ensemble de notes fournies par l'utilisateur.
   - Exemple : `saisirNotes()`
2. **Calcul de la moyenne :**
   - Additionner toutes les notes et diviser par leur nombre.
   - Exemple : `calculerMoyenne()`
3. **Affichage du résultat :**
   - Présenter la moyenne calculée à l'utilisateur de manière claire.
   - Exemple : `afficherMoyenne()`

Chaque étape du problème est gérée par une fonction dédiée, et le programme principal orchestre leur exécution.



- Le programme principal (`main`) déclare les variables nécessaires (nombre de notes, tableau de notes).
- Il appelle séquentiellement `saisirNotes` , `calculerMoyenne` puis `afficherMoyenne` .
- Cette approche illustre comment des fonctions distinctes collaborent pour résoudre un problème global.

  Chaque fonction traite un sous-problème, menant à une structure claire et modulaire.

Principes Clés d’une Conception Fonctionnelle Réussie

Pour une décomposition efficace et des fonctions de qualité :

- **Une fonction = une tâche :** Chaque fonction doit avoir une seule responsabilité, limitée et bien définie.
- **Paramétrage :** Concevez les fonctions pour qu’elles soient génériques et réutilisables en utilisant des paramètres d’entrée appropriés.
- **Compacité :** Limitez la taille et la complexité de chaque fonction pour en faciliter la compréhension et le test.
- **Documentation claire :** Décrivez précisément l’interface de la fonction (rôle, paramètres d’entrée, valeurs de retour, effets de bord).

Maîtriser la Modularité : Synthèse et Approfondissement

Ce qu'il faut retenir :

Décomposer un problème en fonctions gérables apporte **clarté**, facilite la **conception**, le **test** et la **maintenance** d'un logiciel. Ce principe s'applique dans tout développement sérieux et constitue la base d'une programmation modulaire efficace.

Pour aller plus loin :

- GeeksforGeeks - Modular Programming in C
- TutorialsPoint - Modular Programming
- ISO/IEC 9899:2018 (C18) – principe de modularité, chapitres 6.7 Fonctions

Gestion de fichiers en langage C

Objectifs du cours

À la fin de ce cours, vous serez capable de :

- Comprendre le principe des fichiers en C
- Ouvrir et fermer un fichier correctement
- Lire et écrire dans des fichiers texte
- Manipuler les erreurs liées aux fichiers
- Utiliser les fonctions standards de la bibliothèque `<stdio.h>`

1. Rappels : qu’est-ce qu’un fichier ?

Un **fichier** est une zone de stockage persistante sur un support (disque dur, SSD, clé USB…). Contrairement aux variables en mémoire, les données stockées dans un fichier :

- persistent après l’arrêt du programme
- peuvent être relues plus tard
- peuvent être partagées entre plusieurs programmes

En C, la gestion des fichiers se fait principalement à l’aide de la bibliothèque standard :

```c
#include <stdio.h>
```

2. Le type `FILE`

En C, un fichier est manipulé via un **pointeur de type** `FILE*`.

```
FILE *fichier;
```

Ce pointeur permet au programme de savoir :

- où se trouve le fichier
- dans quel mode il est ouvert
- où se situe le curseur de lecture/écriture

3. Ouvrir un fichier : `fopen`

3.1 Syntaxe

```
FILE *fopen(const char *nomFichier, const char *mode);
```

3.2 Modes d'ouverture les plus courants

Mode	Description
`"r"`	Lecture seule (le fichier doit exister)
`"w"`	Écriture (crée ou écrase le fichier)
`"a"`	Ajout à la fin du fichier
`"r+"`	Lecture et écriture
`"w+"`	Lecture et écriture (écrase)

3.3 Exemple

```
FILE *f = fopen("donnees.txt", "r");
```

Attention : si l'ouverture échoue, `fopen` retourne `NULL`.

4. Vérifier l'ouverture du fichier

Il est indispensable de vérifier que le fichier a bien été ouvert :

```
if (f == NULL) {
    printf("Erreur lors de l'ouverture du fichier\n");
    return 1;
}
```

5. Fermer un fichier : `fclose`

Tout fichier ouvert doit être fermé.

```
fclose(f);
```

Pourquoi ?

- libération des ressources mémoire
- écriture effective des données sur le disque
- évite les corruptions de fichiers

6. Écriture dans un fichier

6.1 `fprintf`

Fonction très proche de `printf`, mais qui écrit dans un fichier.

```
fprintf(f, "Bonjour %s %d\n", nom, age);
```

Exemple complet

```
#include <stdio.h>

int main() {
    FILE *f = fopen("message.txt", "w");

    if (f == NULL) {
        printf("Impossible d'ouvrir le fichier\n");
        return 1;
    }

    fprintf(f, "Bonjour le monde !\n");

    fclose(f);
    return 0;
}
```

7. Lecture dans un fichier

7.1 Lecture caractère par caractère : `fgetc`

```
char c = fgetc(f);
```

Retourne `EOF` (End Of File) à la fin du fichier.

Exemple

```
char c;
while ((c = fgetc(f)) ≠ EOF) {
    printf("%c", c);
}
```

7.2 Lecture ligne par ligne : `fgets`

```
fgets(buffer, taille, f);
```

Exemple

```
char ligne[100];

while (fgets(ligne, 100, f) ≠ NULL) {
    printf("%s", ligne);
}
```

7.3 Lecture formatée : `fscanf`

```c
int age;
char nom[50];

fscanf(f, "%s %d", nom, &age);
```

Attention : `fscanf` est sensible au format et peut être source d'erreurs si le fichier est mal structuré.

8. Détection de fin de fichier : `feof`

```
while (!feof(f)) {
    // lecture
}
```

Bonne pratique : préférer tester directement le retour de la fonction de lecture (`fgets` , `fgetc` , etc.).

9. Exemple complet : lecture d’un fichier texte

```c
#include <stdio.h>

int main() {
    FILE *f = fopen("texte.txt", "r");
    char ligne[100];

    if (f == NULL) {
        printf("Erreur d'ouverture\n");
        return 1;
    }

    while (fgets(ligne, 100, f) != NULL) {
        printf("%s", ligne);
    }

    fclose(f);
    return 0;
}
```

10. Bonnes pratiques

- Toujours vérifier le retour de `fopen`
- Toujours fermer un fichier avec `fclose`
- Choisir le bon mode d’ouverture
- Ne jamais lire ou écrire sur un fichier non ouvert
- Utiliser des tailles de buffer suffisantes

11. Exercices

Exercice 1

Créer un programme qui écrit votre nom et votre âge dans un fichier `infos.txt`.

Exercice 2

Lire le fichier `infos.txt` et afficher son contenu à l'écran.

Exercice 3 (bonus)

Créer un programme qui copie le contenu d'un fichier source vers un fichier destination.